# ccom: compilador de Calser para una arquitectura VLIW de cuatro slots

Luis Castro Montenegro, Saddy Guzmán Rojas, Gabriel Núñez Morales y Kevin Ruiz Rodríguez

***Resumen*—Este informe presenta `ccom`, un compilador que traduce programas escritos en el lenguaje Calser a código binario para una arquitectura VLIW de cuatro slots, es decir, con cuatro posiciones para instrucciones dentro de cada *bundle* de 128 bits. `ccom` está implementado en C con Flex y Bison y realiza seis fases: análisis léxico; análisis sintáctico con construcción del árbol sintáctico abstracto; análisis semántico con tablas de símbolos y de etiquetas y asignación de direcciones; generación de ensamblador; cálculo de saltos y resolución de referencias; y generación del binario. Produce un archivo binario con encabezado, código y datos iniciales, sus versiones `.hex` y `.mem` para inicializar las memorias del procesador y un ensamblador anotado. Se especifican la arquitectura y el lenguaje, se describen los algoritmos clave y el manejo de los casos especiales, y se presentan los tres ejercicios con su ensamblador, su binario y su traza. Los resultados se validaron en un simulador del ensamblador, no en hardware físico.**

***Index Terms*—Compiladores, VLIW, Flex, Bison, tabla de símbolos, backpatching, generación de código, Feistel.**

## I. Introducción

El proyecto pide construir un compilador que traduzca un lenguaje propio a código binario ejecutable por una arquitectura también propia, con énfasis en la traducción de las estructuras de control, el manejo de tablas de símbolos, el cálculo de saltos y la resolución de referencias [2]. Los algoritmos siguen el libro de texto del curso [1]. El código fuente está en https://github.com/Kevnrz/calser_compiler.

La Sección II especifica la arquitectura; la III, el lenguaje; la IV, los cambios respecto al diseño de setiembre; la V, los algoritmos clave; la VI, los casos especiales; la VII, los ejercicios; y la VIII, la evidencia de pruebas. El Apéndice A contiene el conjunto de instrucciones; el B, la gramática; el C, los cambios al diseño original; y los D a F, el código fuente, el ensamblador, el binario y la traza de cada ejercicio.

## II. Especificación de la arquitectura

El procesador destino es VLIW con paralelismo estático: el compilador agrupa hasta cuatro instrucciones independientes en un *bundle*, y el procesador las ejecuta en el mismo ciclo. Esta sección especifica lo que pide la sección 8.1 del enunciado [2]: tamaño de palabra, registros, organización de memoria y conjunto de instrucciones.

Instituto Tecnológico de Costa Rica, Escuela de Ingeniería en Computadores. CE 1108 Compiladores e Intérpretes, II semestre 2026. Profesor: Jason Leitón Jiménez. Entrega: 14 de octubre de 2026.

### *II-A. Tamaño de palabra e instrucción*

Cada instrucción mide 32 bits y cada bundle, 128 bits: cuatro slots, uno por unidad funcional. Los registros y los datos son de 32 bits, los inmediatos de 18 bits con signo y la memoria de datos es *little endian*. La Tabla I resume estos parámetros y qué unidad ocupa cada slot.

Tabla I
Parámetros de la arquitectura

| Parámetro | Valor |
|---|---|
| Instrucción | 32 bits |
| Bundle | 4 slots, 128 bits |
| Registros | 16 de 32 bits |
| Inmediato | 18 bits con signo |
| Memoria de instrucciones | 4096 bundles (64 KB) |
| Memoria de datos | 64 KB, por byte, *little endian* |
| Contador de programa | 22 bits, cuenta bundles |
| Slot 0 (bits 31..0) | ALU0 |
| Slot 1 (bits 63..32) | ALU1 o CRYPTO |
| Slot 2 (bits 95..64) | BRU (saltos) |
| Slot 3 (bits 127..96) | LSU (memoria) |

El slot 1 elige entre la segunda ALU y la unidad criptográfica con el bit 31 (`sel`). Los campos ocupan la misma posición en todos los formatos (Tabla II). La instrucción vacía es `nopi` (`funct = 00000`), de modo que un bundle vacío son 128 bits en cero.

Tabla II
Formatos de instrucción

| Tipo | 31 | 30..26 | 25..22 | 21..18 | 17..0 |
|---|---|---|---|---|---|
| R | sel | funct | rd | rs1 | rs2 (17..14), reservado |
| I | sel | funct | rd | rs1 | imm (18, con signo) |
| B | sel | funct | rs2 | rs1 | off (18, con signo) |
| J | sel | funct | rd | target (21..0, 22 con signo) | |
| CRYPTO | 1 | funct | rd | rs1 | rs2, k (13..12), i (11..10) |

Dos instrucciones dependientes deben estar separadas por un bundle, porque el procesador no detecta riesgos de datos. Un salto tomado descarta el bundle siguiente; los demás slots del bundle del salto sí se ejecutan.

### *II-B. Registros*

La Tabla III lista los 16 registros con su código y su uso en `ccom`. Los parámetros y el valor de retorno no viajan en registros: cada uno tiene una casilla fija en memoria (Sección VI-B).

### *II-C. Organización de memoria*

La memoria de instrucciones se direcciona por bundle y la de datos por byte. Como el inmediato de 18 bits cubre

Tabla III
BANCO DE REGISTROS Y USO EN `ccom`

| Nombre | Código | Uso |
|---|---|---|
| `rz` | 0 | Cero cableado. Base de toda dirección absoluta (`lw t0, 64(rz)`) y comparación con cero |
| `rl` | 1 | Enlace de retorno: lo escribe `jas` y lo lee `jefR rz, rl` |
| `rx` | 2 | Puntero de la pila de llamadas |
| `rg` | 3 | Puntero global por convención; `ccom` no lo usa |
| `t0`–`t11` | 4–15 | Temporales para expresiones, argumentos, rutinas aritméticas y cifrado |

los 64 KB de datos, toda variable se accede con base `rz` y su dirección absoluta. La Tabla IV muestra el mapa de la memoria de datos.

Tabla IV
MAPA DE LA MEMORIA DE DATOS

| Rango | Contenido |
|---|---|
| `0x0000`– | Variables globales, en orden de declaración |
| siguientes | Un área estática por función |
| fin de áreas–`0x0FFF` | Pila de llamadas (`rx`), crece hacia arriba; mínimo 256 B |
| `0x1000`–`0xFFFF` | Datos externos cargados en memoria (archivos para cifrar) |

No hay *heap*: Calser no tiene memoria dinámica. Las variables tienen asignación estática, lo que es posible porque el semántico prohíbe la recursión ordinaria y la indirecta. El área de cada función se reparte como indica la Tabla V.

Tabla V
ÁREA ESTÁTICA DE UNA FUNCIÓN

| Desplazamiento | Contenido |
|---|---|
| Inicio | Variable de retorno, si no es `void` |
| siguientes | Parámetros, en orden (un arreglo ocupa 4 B: su dirección) |
| siguientes | Locales de todos sus bloques, incluida la variable de cada `S` y su casilla oculta `fin` |
| últimos 48 B | 12 casillas para guardar temporales (`temp_base`) |

`int` y `bool` miden 4 bytes; `int[n]` mide $4n$ y `int[f][c]` mide $4fc$, por filas, con $dir(i,j) = base + (i \cdot c + j) \cdot 4$. Las direcciones se asignan con un contador que avanza el ancho de cada declaración. En el ejercicio 1, `factorial` ocupa `0x0000`–`0x003F` (retorno en `0x0004`, `n` en `0x0008`, `i` en `0x000C`), `main` ocupa `0x0040`–`0x0077` (`x` en `0x0040`, `f` en `0x0044`) y la pila empieza en `0x0078`.

### *II-D. Conjunto de instrucciones*

La Tabla IX (Apéndice A) lista las 33 instrucciones, más `nopi`, con su opcode, formato, operación y un ejemplo de uso. El opcode es el campo `funct`, independiente por unidad: el mismo valor significa cosas distintas en slots distintos. Los saltos se expresan en bundles, relativos al bundle del salto. Las operaciones que el ISA no tiene, como la multiplicación o la división, las genera el compilador (Sección V-F).

### *II-E. Formato del ensamblador y del binario*

El ensamblador tiene un bundle por línea, `{ s0 | s1 | s2 | s3 }`, etiquetas con `nombre:` y comentarios con `#`. El compilador produce:

- `.bin`: encabezado de cuatro palabras de 32 bits (cantidad de bundles, punto de entrada 0, tamaño de la imagen de datos y su dirección, 0), seguido del código y de los valores iniciales de las globales (sección 6.1 del enunciado).
- `.hex`: un bundle de 128 bits por línea, slot 3 a la izquierda, para inicializar la memoria de instrucciones.
- `.mem`: `@0` y una palabra por línea con las globales, para la memoria de datos.
- `.asm` (con `-s`): ensamblador con la dirección de cada bundle, la línea del fuente, la tabla de etiquetas y la de variables (sección 6.2 del enunciado).

## III. ESPECIFICACIÓN DEL LENGUAJE

Calser es estructurado, imperativo y de tipado estático. Su nombre son sus seis primitivas de control: `C` (*check*), `A` (*alt*), `L` (*loop*), `S` (*step*), `E` (*emit*) y `R` (*rewind*). El `;` termina toda sentencia y toda cabecera; el `.` cierra todo bloque. No hay símbolo de apertura, lo que elimina el *dangling-else*. Esta sección describe la versión implementada; la Sección IV indica en qué difiere del diseño original [3].

### *III-A. Tokens*

La Tabla VI resume los tokens.

Tabla VI
TOKENS DE CALSER

| Categoría | Lexemas |
|---|---|
| Declaración | `F` función, `Vm` mutable, `Vi` inmutable |
| Control | `C`, `A`, `L`, `S` |
| Retorno | `E` emitir, `R` recursión de cola |
| Tipos | `int` (32 bits con signo), `bool`, `void` |
| Literales | `verdadero`, `falso`, enteros `[0-9]+` |
| Cifrado | `autenticar`, `cerrar_sesion`, `cargar_llave`, `sellar`, `cambiar_ancla`, `cifrar`, `descifrar` |
| Delimitadores | `; . [ ] ( ) : , =` |
| Aritméticos | `+ - * / % ^` |
| Relacionales | `== != < > <= >=` |
| Lógicos | `& \| !` |
| Identificador | `[a-z_][a-zA-Z0-9_]*`, hasta 32 caracteres |
| Comentarios | `#` hasta fin de línea; `#[ ... ]#` de bloque, sin anidar |

Las palabras de control van en mayúscula y los identificadores empiezan en minúscula o `_`, así que no chocan. Un entero va de 0 a $2^{31}$; el signo es el operador unario (`-5` es `-` aplicado a `5`).

### *III-B. Elementos básicos*

```
# Comentario de linea
#[ Comentario
   de bloque ]#
Vi limite : int = 100 ;   # inmutable
Vm total : int ;          # vale 0 (regla del valor cero)
Vm activo : bool ;        # vale falso
Vm m : int [3][3] = [[1,2,3],[4,5,6],[7,8,9]] ;
total = total + limite * 2 ;
m[1][2] = m[0][0] ;
```

Toda variable sin inicializador vale el cero de su tipo. Las globales van al inicio del archivo, antes de las funciones. La asignación es una sentencia, no una expresión. Los arreglos tienen una o dos dimensiones; asignar un arreglo a otro copia su contenido, y pasarlo a una función lo pasa por referencia.

### *III-C. Expresiones*

Las expresiones combinan literales, variables, elementos de arreglo, llamadas y los operadores de la Tabla VII. Los aritméticos operan sobre `int`; los relacionales comparan `int` y dan `bool`; `==` y `!=` aceptan dos `int` o dos `bool`; los lógicos operan sobre `bool`. No hay conversiones implícitas entre `int` y `bool`. La división y el resto truncan hacia cero (`-7 / 2 = -3`, `-7 % 2 = -1`), y `&` y `|` se evalúan en corto circuito.

Tabla VII
OPERADORES, DE MAYOR A MENOR PRECEDENCIA

| Nivel | Operadores | Asociatividad | Tipos |
|---|---|---|---|
| 1 | `( )` | – | – |
| 2 | `^` | derecha | `int → int` |
| 3 | `-` unario, `!` | derecha | `int → int`; `bool → bool` |
| 4 | `* / %` | izquierda | `int → int` |
| 5 | `+ -` | izquierda | `int → int` |
| 6 | `< > <= >=` | izquierda | `int → bool` |
| 7 | `== !=` | izquierda | mismo tipo `→ bool` |
| 8 | `&` | izquierda | `bool → bool` |
| 9 | `\|` | izquierda | `bool → bool` |

Por ejemplo, `a + b * 2 > 10 & !listo` se agrupa como `((a + (b * 2)) > 10) & (!listo)`, y `2 ^ 3 ^ 2` vale $2^9 = 512$.

### *III-D. Gramática*

La Fig. 3 (Apéndice B) contiene la gramática completa, en la notación del libro [1]: los no terminales van en minúscula, **id** y **num** son el identificador y el literal entero, y `'|'` es el operador O, para distinguirlo del separador de alternativas. La precedencia de la Tabla VII está codificada en nueve niveles: los recursivos por la izquierda asocian a la izquierda y los recursivos por la derecha, a la derecha, sin declaraciones de precedencia. La gramática es LALR(1) sin conflictos.

### *III-E. Estructuras de control y funciones*

```
C [ x > 0 ] ;        # if
    total = total + x ;
A [ x == 0 ] ;       # else if (repetible)
    total = 0 ;
A ;                  # else (unica y ultima)
    total = total - 1 ;
.                    # un solo cierre para la cadena
L [ total > 0 ] ;    # while
    total = total - 1 ;
.
S [ i , 0 , 10 , 1 ] ;   # for: i de 0 a 9
    total = total + i ;
.
```

En `S [var, inicio, fin, paso]` la variable se declara implícitamente, `inicio` y `fin` se evalúan una vez y `paso` es un literal distinto de cero; con paso negativo la condición es `var > fin`.

```
F sumar [ int a , int b ] : int res ;
    res = a + b ;
    E ;              # emite res y termina
.
F fact [ int n , int acc ] : int r ;
    C [ n <= 1 ] ;
        r = acc ;
        E ;
    A ;
        R fact [ n - 1 , acc * n ] ;  # recursion de cola
    .
.
F main [] : void ;
    Vm s : int = sumar [ 3 , 5 ] ;
    E ;
.
```

El valor de retorno tiene nombre y arranca en cero, así que no hay que verificar que todo camino retorne. `E` es obligatorio. `R` solo puede llamar a la propia función y reutiliza su área; la recursión ordinaria y la indirecta son error semántico. Llamada e índice comparten la forma `id [ ...]`: el semántico decide cuál es según la tabla de símbolos. El programa debe tener una función `main` sin parámetros, de tipo `void`, que no se puede invocar.

### *III-F. Operaciones de cifrado*

Siete sentencias usan la unidad CRYPTO: `autenticar [t]`, `cerrar_sesion []`, `cargar_llave [k, w, valor]`, `sellar []`, `cambiar_ancla [a]`, y `cifrar/descifrar [origen, destino, bloques, k]`, que procesan bloques de 8 bytes en memoria. `k` y `w` deben ser literales porque son campos de la instrucción.

### *III-G. Ámbito*

Cada bloque (`F`, `C`, `A`, `L`, `S`) abre un ámbito hijo y sus nombres se liberan en su `.`. Un nombre se resuelve con la declaración más cercana. Una local puede ocultar una global, pero no un parámetro de su función.

```
F f [ int base ] : int salida ;   # ambito 1
    Vm factor : int = base * 2 ;
    C [ factor > 20 ] ;           # ambito 2
        Vm bono : int = 50 ;
        factor = factor + bono ;
    .                             # bono se libera
    # bono = 10 ;  error: fuera de alcance
    salida = factor ;
    E ;
.
```

## IV. CAMBIOS AL DISEÑO ORIGINAL

El diseño original del lenguaje [3] se escribió antes de implementar. Los cambios principales son: se eliminó la importación entre archivos, que el enunciado no exige en la primera versión; las fases y banderas pasaron a ser las del enunciado; la declaración quedó como `Vm x : int`, la forma que usaban los ejemplos; la segunda dimensión de los arreglos quedó en la gramática; se agregaron las siete sentencias de cifrado; y cada función usa un área estática en lugar de marcos de activación. La Tabla X (Apéndice C) detalla cada cambio y su motivo; lo que no aparece ahí sigue vigente.

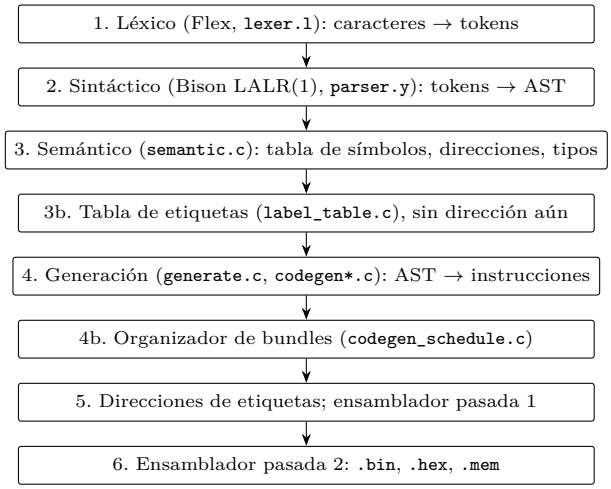



Figura 1. Fases de `ccom` en el orden de `main.c`. Si una fase reporta errores, las siguientes no se ejecutan.

## V. Algoritmos clave

### V-A. Estructura del compilador

La Fig. 1 muestra el flujo. Con `-s`, el `.asm` anotado se escribe después de resolver las direcciones y antes de ensamblar; `-x` toma un `.asm` y ejecuta solo las fases 5 y 6.

### V-B. Análisis léxico

El analizador es generado por Flex [5] a partir de `grammar/lexer.l`. Cada token se describe con una expresión regular; Flex construye el autómata y, ante varias coincidencias, aplica la regla del prefijo más largo y, a igual longitud, la primera regla escrita [1]. Por eso las palabras reservadas van antes del patrón de identificador: `int` sale como palabra reservada, pero `cifrar_total` sale como identificador porque es más largo que `cifrar`. Los comentarios y espacios se descartan sin producir tokens. Cada token lleva su línea y columna (`yylloc`) para los mensajes de error.

Ejemplo con `Vm i : int = 2 ;`:

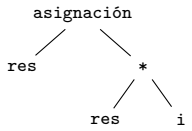
| Lexema | Token | Regla |
|---|---|---|
| `Vm` | `KW_VAR_M` | palabra reservada |
| `i` | `ID` | `[a-z_][a-zA-Z0-9_]*` |
| `:` | `COLON` | delimitador |
| `int` | `TYPE_INT` | palabra reservada (antes que `ID`) |
| `=` | `OP_ASSIGN` | `=` y no `==`: el siguiente es espacio |
| `2` | `LIT_INT` | `[0-9]+`, valor 2 |
| `;` | `DELIM_STMT` | delimitador |

Errores léxicos (sección 8.4 del diseño original): un carácter fuera del alfabeto se reporta y se descarta; un identificador de más de 32 caracteres se trunca; un literal fuera de 32 bits se reporta y se sustituye por 0; un comentario de bloque sin cerrar es fatal.

### V-C. Análisis sintáctico y construcción del AST

La gramática tiene 43 no terminales y está en `grammar/parser.y`. Bison [4] genera a partir de ella un analizador LALR(1). No tiene conflictos, sin `%expect`. Cada producción tiene una acción semántica que construye el nodo correspondiente del AST (traducción dirigida por la sintaxis [1]). El nodo guarda su tipo, línea, columna, hijos (representación hijo izquierdo y hermano derecho) y, después del semántico, un enlace al símbolo.

Ejemplo: `res = res * i ;` se reduce por `sent_id → id = expr ;`; dentro de `expr`, la reducción `multiplicativo → multiplicativo * unario` crea el nodo de la multiplicación:




`-t` imprime el AST y `--ast-json` lo exporta a JSON, que puede revisarse en un visor interactivo local. Ante un error sintáctico el análisis se detiene (sección 8.3 del diseño original). El mensaje se arma con los tokens que el autómata esperaba en ese estado (`%define parse.error custom` con LAC), y una pila con la línea de cada bloque abierto permite reportar, por ejemplo, `el bloque 'L' abierto en la linea 2 nunca se cierra`.

### V-D. Construcción de la tabla de símbolos

La tabla es un árbol de ámbitos: global, uno por función y uno por cada bloque `C`, `A`, `L` y `S`. Insertar agrega el nombre al ámbito actual y rechaza duplicados; buscar recorre el ámbito actual y sus padres hasta el global.

El semántico hace dos pasadas: la primera registra las globales y las firmas de todas las funciones, de modo que el orden en el archivo es libre; la segunda recorre los cuerpos, abre y cierra ámbitos, asigna direcciones y verifica tipos, mutabilidad, cantidad de argumentos e índices, `E` obligatorio y reglas de `R`. Luego busca ciclos en el grafo de llamadas (recursión prohibida) y verifica que queden al menos 256 bytes para la pila. Los errores semánticos se acumulan y se reportan juntos.

Traza con el ejercicio 1 (salida real de `-m`):

```
Scope 0: GLOBAL
    main : void (function) params=[] area=0x0040 size=56 temp_base=0x0048
    factorial : int (function) params=[n] area=0x0000 size=64 temp_base=0x0010
Scope 1: FUNCTION
    i : int width=4 addr=0x000C
    n : int width=4 addr=0x0008
    res : int (return) width=4 addr=0x0004
Scope 2: C
Scope 3: L
Scope 4: FUNCTION
    f : int width=4 addr=0x0044
    x : int width=4 addr=0x0040
```

Cada declaración recibe la dirección del contador y lo avanza 4 bytes: `res`, `n` e `i` quedan en el área de `factorial`, seguidas de sus 48 B de temporales (`0x10` a `0x3F`); el área de `main` empieza en `0x40`, con `x` en 64 y `f` en 68.

### V-E. Tabla de etiquetas

Después del semántico se recorre el AST y se registra una etiqueta por cada punto de salto: entrada y cuerpo de cada función (`.L_func_f`, `.L_body_f`), siguiente rama y fin de cada `C`, inicio y fin de cada `L`, inicio, incremento y fin de cada `S`, fin de cada corto circuito, `fin_main` y `.L_arith_error`. En ese momento la dirección vale −1: se

conoce después de generar y organizar el código (secciones 2.4 y 2.5 del enunciado). `-m` la imprime con su dirección final (columnas recortadas):

```
.L_arith_error   | ARITH_ERROR  | GLOBAL      | Line: 0  | Address: 79
fin_main         | PROGRAM_END  | GLOBAL      | Line: 0  | Address: 78
.L_func_main     | FUNCTION     | FUNCTION    | Line: 14 | Address: 2
.L_while_end_3   | END          | LOOP_L      | Line: 7  | Address: 76
.L_while_start_2 | START        | LOOP_L      | Line: 7  | Address: 47
.L_if_next_0     | NEXT         | CONDITIONAL | Line: 2  | Address: 40
.L_func_factorial| FUNCTION     | FUNCTION    | Line: 1  | Address: 28
```

### *V-F. Generación de ensamblador*

El generador recorre el AST anotado (traducción dirigida por la sintaxis [1]). Las expresiones se evalúan en temporales `t0`–`t11` con un asignador que libera cada registro al consumirlo. Las variables se leen y escriben en memoria en cada uso, con su dirección absoluta.

- **Control de flujo**: `C` evalúa la condición y salta con `beq t, rz` a la siguiente rama; cada rama termina con `jef` al fin. `L` evalúa al inicio y sale con `beq`. `S` guarda `inicio` en su variable y `fin` en una casilla oculta; sale con `bge var, fin` (o `bge fin, var` con paso negativo) y el paso va como inmediato de `saddi`. Así ningún registro queda ocupado durante el cuerpo.
- **Corto circuito**: en `a & b`, si `a` es cero se salta al fin sin evaluar `b`; en `a | b`, si es distinto de cero.
- **Operadores directos**: `+`, `-`, `==`, `!=`, `<` y `<=` usan `add`, `sub`, `seq`, `sne`, `slt` y `sle`. `a > b` es `slt` con los operandos intercambiados; `a >= b` es `slt a, b` seguido de `seq rd, rd, rz`, que lo niega; `!a` es `seq rd, rd, rz`. Una constante que no cabe en 18 bits se arma con `mov`, `sll` y `saddi`.
- **Rutinas aritméticas**: el ISA no tiene multiplicación ni división, así que `*`, `/`, `%` y `^` se expanden en línea, en cada uso, como rutinas con saltos internos:
  - `*`: suma y desplazamiento; por cada bit en 1 del multiplicador se suma el multiplicando desplazado.
  - `/`: 32 iteraciones de desplazamiento y resta sobre los valores absolutos, con el signo corregido al final.
  - `%`: la misma división, y entrega el resto con el signo del dividendo.
  - `^`: exponenciación por cuadrados, con multiplicaciones en línea.

  Usan 11 de los 12 temporales. Un divisor cero o un exponente negativo saltan a `.L_arith_error`, un lazo que detiene el programa. El cálculo de índices de matriz multiplica por una constante (las columnas) y no usa la rutina.
- **Cifrado**: cada sentencia se expande en una plantilla fija con `feis` en el slot 1; los temporales vivos se guardan en `temp_base`.

Cada instrucción se emite primero en una versión conservadora, con un bundle vacío después de cada una, que respeta la separación de la ISA. Luego el organizador compacta el código.

### *V-G. Organizador de bundles*

El organizador (`src/codegen_schedule.c`) planifica por regiones locales:

1. Agrupa instrucciones independientes en los cuatro slots: hasta dos de ALU, una de salto y una de memoria por bundle.
2. Una dependencia lectura después de escritura exige distancia 2 (un bundle intermedio, que puede tener trabajo útil).
3. Las dependencias escritura después de lectura y escritura después de escritura exigen distancia 1, y los accesos a memoria conservan su orden.
4. Un salto cierra la región: puede compartir bundle con operaciones independientes previas, pero no adelanta trabajo del camino siguiente.
5. No reorganiza las plantillas de cifrado.
6. Ante cualquier instrucción que no reconoce, conserva el programa conservador completo; `--no-schedule` lo desactiva.

Un verificador revisa la separación del resultado, y las pruebas comparan cada programa con y sin organizador. En el ejercicio 1 el código baja de 110 a 80 bundles y la ejecución simulada de 337 a 231 ciclos (31 % menos), con el mismo resultado ($5! = 120$) y sin avisos de separación.

### *V-H. Algoritmo de cálculo de saltos*

Es la técnica de *backpatching* [1], en dos pasadas: los saltos se generan con el nombre de la etiqueta y su desplazamiento se llena cuando se conoce la dirección.

1. La Fase 3 crea cada etiqueta sin dirección.
2. El generador emite los saltos con el nombre de la etiqueta.
3. Después de organizar, cada etiqueta recibe el índice del bundle que la sigue. Las direcciones se fijan aquí porque el organizador cambia la posición de los bundles.
4. Pasada 1 del ensamblador: recorre el `.asm`, cuenta bundles y registra cada etiqueta con su índice; rechaza duplicados.
5. Pasada 2: para cada salto calcula $off = dir(etiqueta) - PC$, donde $PC$ es el bundle del salto; verifica el rango (18 bits en `beq`/`bne`/`blt`/`bge`, 22 en `jef`/`jas`) y lo codifica en complemento a dos.

Traza con el ejercicio 1:

| Salto | PC | Etiqueta | off |
|---|---|---|---|
| `jas rl, .L_func_factorial` | 19 | 28 | $28 - 19 = 9$ |
| `jef .L_while_start_2` | 74 | 47 | $47 - 74 = -27$ |

`jas` es formato J: `sel`=0, `funct`=10001, `rd`=`rl`=1, `target`=9, que da `0x44400009`. Para −27, el campo de 22 bits es `0x3FFFE5` y `jef` (10000) da `0x403FFFE5`. Ambas palabras aparecen en el slot 2 de las líneas 20 y 75 del `.hex`:

```
00000000_44400009_00000000_40880004   # bundle 19
0900000C_403FFFE5_00000000_00000000   # bundle 74
```

### V-I. Algoritmo de resolución de referencias

Para cada identificador usado, el semántico busca el nombre desde el ámbito actual hacia los padres y guarda en el nodo un enlace al símbolo encontrado. El símbolo ya tiene su dirección. El generador toma ese enlace y emite el acceso con la dirección como inmediato y `rz` como base. En un arreglo, la dirección del elemento se calcula en la ALU; si el arreglo es un parámetro, se carga primero la dirección guardada en la casilla del parámetro.

Traza de `f = factorial [ x ] ;` (línea 17 del ejercicio 1):

| Nombre | Ámbito | Dirección | Instrucción |
|---|---|---|---|
| `x` | 4 (main) | `0x0040` | `lw t0, 64(rz)` |
| `n` (parámetro) | 1 | `0x0008` | `sw t0, 8(rz)` |
| `res` (retorno) | 1 | `0x0004` | `lw t0, 4(rz)` |
| `f` | 4 (main) | `0x0044` | `sw t0, 68(rz)` |

## VI. Manejo de casos especiales

### VI-A. Saltos anidados

Cada estructura recibe sus propias etiquetas con un número único, así que una estructura interna no puede saltar a las etiquetas de la externa por error. En el ejercicio 3, un `C` está dentro de un `S` (línea 35 del fuente):

| Etiqueta | Bundle | Uso |
|---|---|---|
| `.L_for_start_2` | 324 | condición del `S` |
| `.L_if_next_5` | 496 | rama `A` del `C` |
| `.L_if_end_6` | 589 | fin del `C` |
| `.L_for_update_3` | 589 | incremento del `S` |
| `.L_for_end_4` | 595 | salida del `S` |

El fin del `C` y el incremento del `S` caen en el mismo bundle: al terminar la rama, el control pasa directo al incremento. Dos etiquetas con la misma dirección son válidas. El `bge t0, t1, .L_for_end_4` del bundle 327 salta hacia adelante, sobre todo el cuerpo, y el `jef .L_for_start_2` del bundle 593 salta hacia atrás.

### VI-B. Llamadas a función y retorno de valores

1. El llamador apila los temporales vivos (`sw t, 0(rx)`; `saddi rx, rx, 4`).
2. Evalúa cada argumento escalar, de izquierda a derecha, y lo apila. Un arreglo no se apila: su dirección va a la casilla del parámetro.
3. Desapila los argumentos hacia las casillas de los parámetros.
4. Apila `rl` y ejecuta `jas rl, .L_func_f`, que guarda $PC + 1$ en `rl`.
5. La función pone su retorno en 0 y ejecuta el cuerpo; `E` ejecuta `jefR rz, rl`.
6. El llamador desapila `rl` y los temporales, y lee la casilla de retorno.

La pila hace falta aunque las variables sean estáticas: si `f` llama a `g`, `jas` sobrescribe `rl`, y sin guardarlo `f` no podría volver. La Fig. 2 muestra la secuencia real. `R` no usa este protocolo: escribe los nuevos valores en los parámetros y salta a `.L_body_f`, después del prólogo, para no reiniciar el retorno.

### VI-C. Variables con el mismo nombre en diferentes ámbitos

Cada declaración es un símbolo distinto, en su ámbito, con su propia casilla. La búsqueda desde el ámbito actual hacia los padres encuentra siempre la más cercana, así que no hay conflicto. En el ejercicio 3:

| Nombre | Función | Dirección |
|---|---|---|
| `tam` | `bloques_de` | `0x0018` |
| `tam` | `direccion` | `0x0060` |
| `tam` | `cifrar_archivo` | `0x00E0` |
| `tam` | `descifrar_archivo` | `0x0134` |
| `i` (de un `S`) | `cifrar_archivo` | `0x00E4` |
| `i` (de un `S`) | `descifrar_archivo` | `0x0138` |

Como cada nombre tiene su propia casilla, ni siquiera los bloques hermanos comparten memoria. El `.asm` anotado lista las variables por ámbito con su función dueña.

### VI-D. Asignación de memoria para variables locales y globales

**Globales.** Van desde `0x0000`. Si su inicializador es constante (incluidas expresiones constantes y otras `Vi` constantes), el valor va directo en el `.mem`; si no, la casilla vale 0 en la imagen y el valor se calcula al inicio de `main`. El ejercicio 3 tiene tres globales:

```
@0
00001000
00002000
00003000
```

**Locales.** Están en el área de su función. Su inicialización se emite donde está la declaración, así que se repite cada vez que el control entra al bloque; una local sin inicializador recibe 0 en ese punto. Una local declarada dentro de un `S` vuelve a su valor inicial en cada vuelta.

## VII. Ejemplos de ejecución

Los tres ejercicios están en `ejercicios/`, cada uno con su fuente, `.asm`, `.bin`, `.hex`, `.mem`, la salida de compilación, la traza y el resumen de ejecución. `make ejercicios` los regenera y falla si algún resultado no coincide con el esperado. Se ejecutan en el simulador de ensamblador del proyecto (`tools/sim_asm.py`), que modela el pipeline y la unidad CRYPTO, y avisa si se viola la separación de un bundle. En el simulador, la autenticación usa un modelo de prueba sin Ascon: acepta cualquier token salvo uno reservado para probar el rechazo. La ejecución no se validó en hardware físico. La Tabla VIII resume los resultados.

Tabla VIII
Resultados de los ejercicios

| Ejercicio | Bundles | Ejecut. | Ciclos | Avisos | Resultado |
|---|---|---|---|---|---|
| 1 factorial | 80 | 203 | 231 | 0 | `f = 120` |
| 2 matriz | 162 | 367 | 383 | 0 | `p = [1,1]` |
| 3 cifrado | 925 | 14 770 | 17 083 | 0 | `n1 = n2 = 8` |

**Ejercicio 1, factorial** (sección 7.1 del enunciado). `factorial[5]` con variables locales, un `C` para $n < 0$, un `L` y multiplicación. Resultado: `f = 120`.

```
  { nopi | nopi | nopi | lw t0, 64(rz) }        # 0008  linea 17   carga x
  { nopi | nopi | nopi | sw t0, 0(rx) }         # 0010             apila el argumento
  { saddi rx, rx, 4 | nopi | nopi | nopi }      # 0011
  { saddi rx, rx, -4 | nopi | nopi | nopi }     # 0013             desapila...
  { nopi | nopi | nopi | lw t0, 0(rx) }         # 0015
  { nopi | nopi | nopi | sw t0, 8(rz) }         # 0017             ...hacia n (0x0008)
  { nopi | nopi | nopi | sw rl, 0(rx) }         # 0018             apila rl
  { saddi rx, rx, 4 | nopi | jas rl, .L_func_factorial | nopi }  # 0019   llamada
  { saddi rx, rx, -4 | nopi | nopi | nopi }     # 0021             al volver: desapila rl
  { nopi | nopi | nopi | lw rl, 0(rx) }         # 0023
  { nopi | nopi | nopi | lw t0, 4(rz) }         # 0024             lee el retorno (0x0004)
  { nopi | nopi | jef fin_main | sw t0, 68(rz) } # 0026  linea 18  f = retorno; E de main
.L_func_factorial: # bundle 28
  { nopi | nopi | nopi | sw rz, 4(rz) }         # 0028  linea 1    retorno en 0
  ...
  { nopi | nopi | jefR rz, rl | nopi }          # 0036  linea 3    E: retorno
```

Figura 2. Llamada `f = factorial [ x ]` del ejercicio 1 (extracto de `ejercicio1.asm`; se omiten los bundles vacíos de separación y se agregan comentarios a la derecha).

**Ejercicio 2, búsqueda en matriz** (sección 7.2 del enunciado). Busca 42 en una matriz $3 \times 4$ con dos `S` anidados; la condición de salida es un `E` dentro del `C`. Devuelve la posición en un arreglo pasado por referencia. Resultado: `ok = verdadero`, `p = [1, 1]`.

**Ejercicio 3, cifrado por segmentos** (sección 7.3 del enunciado). Toma 1024 bytes de texto cargados en `0x1000` y los parte en segmentos de 16 bloques de 8 bytes: los pares se cifran con la llave 0 y los impares con la 1, hacia `0x2000`; luego se descifra todo hacia `0x3000`. Usa cinco funciones con parámetros y retorno, varias llamadas, `C` dentro de `S` y `*`, `/` y `%` con variables. Resultado: `0x3000-0x33FF` es igual a `0x1000-0x13FF` y los contadores `n1` y `n2` valen 8.

La traza escribe un evento por línea: escrituras a variables con su valor, saltos condicionales tomados, saltos incondicionales, llamadas, retornos y operaciones de cifrado, cada uno con el bundle y la línea del fuente. Omite los saltos internos de las rutinas aritméticas y las escrituras a la pila.

Los Apéndices D–F contienen, para cada ejercicio, la evidencia de compilación, el código fuente, el ensamblador, el binario en hexadecimal, la traza y el resultado de la ejecución. Todo es salida real del compilador y del simulador, copiada sin editar, y sirve como evidencia de la compilación y de la generación correcta del binario.

## VIII. Pruebas y evidencia

`make check` corre pruebas automatizadas, en su mayoría scripts de Python que comparan contra el simulador de ensamblador: 1746 casos de las rutinas `* / % ^`, 1190 de acceso a arreglos, 184 de cifrado, los vectores de codificación del ISA, el ensamblado de ida y vuelta, programas completos comparados por valor (incluidos los tres ejercicios), la coherencia de las direcciones de etiquetas, 45 casos del simulador y la equivalencia de cada programa con y sin organizador de bundles. Además, `pruebas/correr_pruebas.sh` compila 91 programas: 12 correctos (`p01`–`p12`), que terminan con código 0, y 79 con un error cada uno (`e01`–`e79`), cuya salida se compara contra `pruebas/resultados.txt`. Todo el código se revisó con AddressSanitizer.

### *VIII-A. Uso del compilador*

```
ccom [opciones] <archivo.cls> [-o <salida.bin>]
  -o   archivo binario de salida (por defecto, fuente.bin)
  -v   detalle de cada fase
  -s   genera el .asm anotado
  -t   imprime el AST
  -m   tablas de simbolos y de etiquetas, con direcciones
  -x   ensambla un .asm y genera el binario
  -p   muestra los bundles en ensamblador y en hexadecimal
```

## IX. Limitaciones conocidas

- No se detecta en ejecución el desborde de la pila de llamadas; el semántico solo exige 256 bytes libres, que alcanzan con holgura para los ejercicios.
- Los índices variables de arreglo no se verifican en ejecución.
- Los datos de `ccom` deben caber en `0x0000`–`0x0FFF`.
- La autenticación se validó solo con el modelo de prueba del simulador, sin Ascon. El token de `autenticar` del ejercicio 3 es provisional: el procesador exige el de su cadena de Lamport, calculada con Ascon-Hash256.
- No se validó la ejecución en hardware físico.
- Hay redundancias menores en el código: el paso de argumentos apila y desapila de inmediato.

## X. Conclusiones

`ccom` implementa las seis fases del enunciado sobre la arquitectura VLIW de cuatro slots y compila los tres ejercicios, cuyos resultados son correctos en el simulador de ensamblador. Las decisiones que más simplificaron el trabajo fueron prohibir la recursión ordinaria, que permite la asignación estática, y fijar las direcciones de las etiquetas después de organizar los bundles, de modo que el cálculo de saltos se reduce a una resta en el ensamblador de dos pasadas. Emitir primero una versión conservadora y compactarla después permitió verificar la corrección antes de optimizar.

## Referencias

[1] A. V. Aho, M. S. Lam, R. Sethi y J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2.ª ed., Pearson New International Edition. Harlow, Reino Unido: Pearson, 2014.

[2] J. Leitón Jiménez, “Proyecto: Compilador para código binario,” CE 1108 Compiladores e Intérpretes, Instituto Tecnológico de Costa Rica, sep. 2026.
[3] L. Castro, S. Guzmán, G. Núñez y K. Ruiz, “Calser: definición de sintaxis del lenguaje,” CE 1108, Instituto Tecnológico de Costa Rica, sep. 2026.
[4] Free Software Foundation, *GNU Bison Manual*, versión 3.8.2, 2021.
[5] V. Paxson *et al.*, *Flex: The Fast Lexical Analyzer*, versión 2.6.4.

# Apéndice A
# Conjunto de instrucciones

Tabla IX
Conjunto de instrucciones

| Unidad (slot) | Opcode | Formato | Instrucción | Operación | Ejemplo |
|---|---|---|---|---|---|
| Todas | 00000 | – | `nopi` | sin efecto | `nopi` |
| ALU (0, 1) | 00001 | R | `add rd, rs1, rs2` | $rd = rs1 + rs2$ | `add t0, t0, t1` |
| | 00010 | R | `sub rd, rs1, rs2` | $rd = rs1 - rs2$ | `sub r7, r0, r7` |
| | 00011 | R | `and rd, rs1, rs2` | Y bit a bit | `and r9, r8, r10` |
| | 00100 | R | `or rd, rs1, rs2` | O bit a bit | `or r9, r9, r12` |
| | 00101 | R | `xor rd, rs1, rs2` | O exclusivo bit a bit | `xor r13, r4, r5` |
| | 00110 | R | `sll rd, rs1, rs2` | desplazamiento lógico a la izquierda | `sll r7, r7, r10` |
| | 00111 | R | `srl rd, rs1, rs2` | desplazamiento lógico a la derecha | `srl r8, r8, r10` |
| | 01000 | R | `sra rd, rs1, rs2` | desplazamiento aritmético a la derecha | `sra t0, t0, t1` |
| | 01001 | R | `slt rd, rs1, rs2` | $rd = 1$ si $rs1 < rs2$ (con signo) | `slt t0, t0, t1` |
| | 01010 | R | `sle rd, rs1, rs2` | $rd = 1$ si $rs1 \leq rs2$ (con signo) | `sle t0, t0, t1` |
| | 01011 | R | `seq rd, rs1, rs2` | $rd = 1$ si $rs1 = rs2$ | `seq t0, t0, rz` |
| | 01100 | R | `sne rd, rs1, rs2` | $rd = 1$ si $rs1 \neq rs2$ | `sne t0, t0, t1` |
| | 01101 | R | `ifmov rd, rs1, rs2` | $rd = rs1$ si $rs2 \neq 0$ | `ifmov t0, t1, t2` |
| | 10000 | I | `saddi rd, rs1, imm` | $rd = rs1 + imm$ | `saddi rx, rx, 4` |
| | 10001 | I | `xori rd, rs1, imm` | $rd = rs1 \oplus imm$ | `xori t0, t0, 1` |
| | 10010 | I | `mov rd, imm` | $rd = imm$ | `mov t1, 2` |
| BRU (2) | 00001 | B | `beq rs1, rs2, off` | salta si $rs1 = rs2$ | `beq t0, rz, .L_while_end_3` |
| | 00010 | B | `bne rs1, rs2, off` | salta si $rs1 \neq rs2$ | `bne t0, rz, .L_sc_end_1` |
| | 00011 | B | `blt rs1, rs2, off` | salta si $rs1 < rs2$ (con signo) | `blt r5, r0, .L_arith_error` |
| | 00100 | B | `bge rs1, rs2, off` | salta si $rs1 \geq rs2$ (con signo) | `bge t0, t1, .L_for_end_4` |
| | 00101 | R | `jefR rd, rs1` | $rd = PC + 1$; $PC = rs1$ | `jefR rz, rl` |
| | 10000 | J | `jef target` | $PC = PC + target$ | `jef .L_while_start_2` |
| | 10001 | J | `jas rd, target` | $rd = PC + 1$; $PC = PC + target$ | `jas rl, .L_func_factorial` |
| LSU (3) | 00001 | I | `lw rd, imm(rs1)` | carga una palabra | `lw t0, 64(rz)` |
| | 00010 | I | `sw rs2, imm(rs1)` | guarda una palabra | `sw t0, 68(rz)` |
| | 00011 | I | `lb rd, imm(rs1)` | carga un byte con signo | `lb t0, 0(t1)` |
| | 00100 | I | `sb rs2, imm(rs1)` | guarda un byte | `sb t0, 0(t1)` |
| CRYPTO (1) | 00001 | C | `feis rd, rs1, rs2, k, i` | $rd = rs1 \oplus F(rs2, K_{k,i})$, una ronda Feistel | `feis r5, r3, r4, 0, 2` |
| | 00010 | C | `auth rs1` | valida un token de acceso | `auth t0` |
| | 00011 | C | `logout` | cierra la sesión | `logout` |
| | 00100 | C | `kv_write rs1, k, w` | escribe la subllave $w$ de la llave $k$ | `kv_write t0, 0, 1` |
| | 00101 | C | `kv_lock` | bloquea la escritura de llaves | `kv_lock` |
| | 00110 | C | `sanc rs1` | renueva el token de acceso | `sanc t0` |

Ejemplos de codificación: `add r5, r3, r4` (slot 0) → `054D0000`; `lw r7, 0(r6)` (slot 3) → `05D80000`; `blt r3, r4, -2` (slot 2) → `0D0FFFFE`; `feis r5, r3, r4, 0, 2` (slot 1) → `854D0800`; del ejercicio 1, `jas rl, .L_func_factorial` en el bundle 19 → `44400009` (Sección V-H).

# Apéndice B
## Gramática de Calser

```
programa    → globales funciones
globales    → globales decl | ε
funciones   → funciones funcion | funcion
funcion     → F id [ params ] : retorno ; cuerpo .
retorno     → tipo id | void
params      → lista_param | ε
lista_param → lista_param , param | param
param       → modif_opt tipo id dims_opt
modif       → Vm | Vi
modif_opt   → modif | ε
tipo        → int | bool
dims_opt    → dims | ε
dims        → [ num ] | [ num ] [ num ]

cuerpo      → cuerpo sentencia | ε
sentencia   → decl | sent_id | condicional | ciclo_l
            | ciclo_s | emit | rewind | crypto
decl        → modif id : tipo dims_opt ;
            | modif id : tipo dims_opt = inicial ;
inicial     → expr | lista_init
lista_init  → [ elems ]
elems       → elems , elem | elem
elem        → expr | lista_init
sent_id     → id = expr ;
            | id [ args ] = expr ;
            | id [ args ] [ expr ] = expr ;
            | id [ args ] ;
condicional → C [ expr ] ; cuerpo alts residual .
alts        → alts A [ expr ] ; cuerpo | ε
residual    → A ; cuerpo | ε
ciclo_l     → L [ expr ] ; cuerpo .
ciclo_s     → S [ id , expr , expr , paso ] ; cuerpo .
paso        → num | - num
emit        → E ;
rewind      → R id [ args ] ;
```

```
crypto  → autenticar    [ expr ] ;
        | cerrar_sesion [ ] ;
        | cargar_llave  [ num , num , expr ] ;
        | sellar        [ ] ;
        | cambiar_ancla [ expr ] ;
        | cifrar        [ expr , expr , expr , num ] ;
        | descifrar     [ expr , expr , expr , num ] ;
args      → lista_arg | ε
lista_arg → lista_arg , expr | expr

expr      → disy
disy      → disy '|' conj | conj
conj      → conj & igualdad | igualdad
igualdad → igualdad == relacion
         | igualdad != relacion | relacion
relacion → relacion < aditivo  | relacion > aditivo
         | relacion <= aditivo | relacion >= aditivo
         | aditivo
aditivo  → aditivo + multiplicativo
         | aditivo - multiplicativo | multiplicativo
multiplicativo → multiplicativo * unario
         | multiplicativo / unario
         | multiplicativo % unario | unario
unario   → - unario | ! unario | potencia
potencia → primario ^ unario | primario
primario → ( expr ) | acceso | num
         | verdadero | falso
acceso   → id | id [ args ] | id [ args ] [ expr ]
```

Figura 3. Gramática de Calser. `alts` y `residual` están separados para que la rama `A ;` solo pueda ser la última. `acceso` y `sent_id` unifican llamada e índice; el semántico decide cuál es con la tabla de símbolos.

# Apéndice C
## Cambios al diseño original

Tabla X
Diferencias entre el diseño original [3] y `ccom`. Los números de la primera columna son secciones del diseño original; los de la última, secciones del enunciado

| Tema (diseño) | Diseño de setiembre | Implementación | Motivo |
|---|---|---|---|
| Importación (secciones 4, 7.3, 8.7 y 9.3) | `P "modulo" [f, g] ;`, anotación `@export`, enlace de varios archivos, detección de importación circular | Eliminado. Un programa es un archivo: `programa → globales funciones` | Enunciado, sección 9.1: la primera versión no requiere módulos ni archivos múltiples |
| Fases y banderas (secciones 7.1 y 7.2) | Ensamblador `.s`, objeto `.o`, enlace; banderas `-S`, `-C`, `-I`, `-w`, `-E`; `-s` vuelca la tabla de símbolos | Las seis fases de la sección 2 del enunciado. Banderas `-o -v -s -t -m -x -p`: `-s` genera el `.asm` y `-m` muestra las tablas. Salidas `.bin`, `.hex`, `.mem`. Extras: `--ast-json`, `--no-schedule` | Enunciado, secciones 2, 5.1 y 6 |
| Declaración (sección 5.2) | `decl → modificador tipo id ;` | `decl → modif id : tipo dims_opt ;` y su variante con `= inicial` | Es la forma que usan todos los ejemplos del propio diseño |
| Acceso a arreglos (sección 5.4.2) | `L → L [ E ] \| id [ E ]`, recursiva; la tabla de símbolos limita a dos dimensiones | La segunda dimensión es una producción explícita: tres índices o `int[2][2][2]` son error sintáctico | La gramática impone el límite sin verificación extra |
| Palabras reservadas (sección 2.3) | Se instalan en la tabla de símbolos antes de analizar | Patrones propios de Flex, antes del de identificador | Mismo efecto; Flex resuelve por orden de reglas y prefijo más largo |
| Identificadores (sección 2.3) | `[a-z][a-zA-Z0-9_]*` | `[a-z_][a-zA-Z0-9_]*` | Se permite empezar con `_` |
| Funciones incorporadas (sección 5.6) | El lenguaje no define ninguna | Siete sentencias de cifrado, como palabras reservadas con su propia regla gramatical | El procesador tiene la unidad CRYPTO (Feistel4) y el ejercicio 3 la usa |
| Condición de `L` y `S` (sección 5.8.2) | El texto dice «los paréntesis de la condición son obligatorios» | La condición va entre corchetes `[ ]`, como en todos los ejemplos | Corrección de texto |
| Errores sintácticos (sección 8.5) | Mensajes específicos para cada situación | Implementados: falta `;`, bloque sin cerrar (con su línea de apertura), cierre sobrante, función anidada, `A` sin `C`, `S` sin 4 partes, `)` o `]` faltante. El resto (por ejemplo, `A` residual que no es la última, `R` en una expresión) recibe un mensaje general con el token inesperado y los esperados | Enunciado, sección 2.2: no exige validación compleja; se cubrieron los casos más comunes |
| Errores de ejecución (sección 8.1) | Categoría «condiciones detectables durante la ejecución», sin detalle | División o módulo entre cero y exponente negativo con operandos variables saltan a `.L_arith_error`, un lazo que detiene el programa. Los índices variables no se verifican (como dice la sección 5.4.5) | Enunciado, sección 9.1: no se exige tratamiento de errores en ejecución |
| Marcos de activación (sección 5.7) | Habla de «marcos en pila» y de que `R` sobrescribe el marco vigente | Cada función tiene un área estática fija. Una pila de llamadas en `rx` solo guarda `rl`, los argumentos y los temporales vivos. `R` escribe los parámetros y salta a `.L_body_f` | Sin recursión ordinaria, la asignación estática basta; la pila solo resuelve llamadas anidadas [1] |

# Apéndice D
## Ejercicio 1: factorial

Los Apéndices D–F son salida real del compilador y del simulador, copiada sin editar desde la carpeta `ejercicios/` del repositorio. `make ejercicios` la regenera: compila cada ejercicio con `ccom -v -m -s -p`, lo ejecuta en el simulador de ensamblador (`tools/sim_asm.py`) y falla si algún resultado no coincide con el esperado. Para cada ejercicio se incluye la evidencia de compilación (el comando, su código de salida y el estado de cada fase), el código fuente, el ensamblador, el binario en hexadecimal, la traza y el resumen de la ejecución, con los valores esperados contra los obtenidos.

### *D-A. Evidencia de compilación*

```
$ ./ccom -v -m -s -p ejercicios/ejercicio1_factorial/ejercicio1.cls -o ejercicios/ejercicio1_factorial/ejercicio1.bin
codigo de salida: 0

===== Fases (stderr, -v) =====
ccom: fase 1-2: analisis lexico y sintactico ... ok
ccom: fase 3: analisis semantico ... ok
ccom: fase 3: tabla de etiquetas ... ok
ccom: fase 4: generacion de codigo ... ok
ccom: fase 4: organizacion de bundles ... ok
ccom: ensamblado ... ok
```

### *D-B. Código fuente*

```
F factorial [ int n ] : int res ;
    C [ n < 0 ] ;
        E ;
    .
    res = 1 ;
    Vm i : int = 2 ;
    L [ i <= n ] ;
        res = res * i ;
        i = i + 1 ;
    .
    E ;
.

F main [] : void ;
    Vi x : int = 5 ;
    Vm f : int ;
    f = factorial [ x ] ;
    E ;
.
```

### *D-C. Ensamblador*

```
# ccom: ejercicios/ejercicio1_factorial/ejercicio1.cls
  { mov rx, 120 | nopi | nopi | nopi }                              # 0000
  { nopi | nopi | nopi | nopi }                                     # 0001
.L_func_main:                                                       # bundle 2
.L_body_main:                                                       # bundle 2
  { mov t0, 5 | nopi | nopi | nopi }                                # 0002  linea 15
  { nopi | nopi | nopi | nopi }                                     # 0003
  { nopi | nopi | nopi | sw t0, 64(rz) }                            # 0004
  { mov t0, 0 | nopi | nopi | nopi }                                # 0005  linea 16
  { nopi | nopi | nopi | nopi }                                     # 0006
  { nopi | nopi | nopi | sw t0, 68(rz) }                            # 0007
  { nopi | nopi | nopi | lw t0, 64(rz) }                            # 0008  linea 17
  { nopi | nopi | nopi | nopi }                                     # 0009
  { nopi | nopi | nopi | sw t0, 0(rx) }                             # 0010
  { saddi rx, rx, 4 | nopi | nopi | nopi }                          # 0011
  { nopi | nopi | nopi | nopi }                                     # 0012
  { saddi rx, rx, -4 | nopi | nopi | nopi }                         # 0013
  { nopi | nopi | nopi | nopi }                                     # 0014
  { nopi | nopi | nopi | lw t0, 0(rx) }                             # 0015
  { nopi | nopi | nopi | nopi }                                     # 0016
  { nopi | nopi | nopi | sw t0, 8(rz) }                             # 0017
  { nopi | nopi | nopi | sw rl, 0(rx) }                             # 0018
  { saddi rx, rx, 4 | nopi | jas rl, .L_func_factorial | nopi }     # 0019
  { nopi | nopi | nopi | nopi }                                     # 0020
  { saddi rx, rx, -4 | nopi | nopi | nopi }                         # 0021
  { nopi | nopi | nopi | nopi }                                     # 0022
  { nopi | nopi | nopi | lw rl, 0(rx) }                             # 0023
  { nopi | nopi | nopi | lw t0, 4(rz) }                             # 0024
  { nopi | nopi | nopi | nopi }                                     # 0025
  { nopi | nopi | jef fin_main | sw t0, 68(rz) }                    # 0026  linea 18  slots 0=0,1=0,2=18,3=17
  { nopi | nopi | nopi | nopi }                                     # 0027
.L_func_factorial:                                                  # bundle 28
  { nopi | nopi | nopi | sw rz, 4(rz) }                             # 0028  linea 1
  { nopi | nopi | nopi | nopi }                                     # 0029
.L_body_factorial:                                                  # bundle 30
  { mov t1, 0 | nopi | nopi | lw t0, 8(rz) }                        # 0030  linea 2
  { nopi | nopi | nopi | nopi }                                     # 0031
  { slt t0, t0, t1 | nopi | nopi | nopi }                           # 0032
  { nopi | nopi | nopi | nopi }                                     # 0033
  { nopi | nopi | beq t0, rz, .L_if_next_0 | nopi }                 # 0034
  { nopi | nopi | nopi | nopi }                                     # 0035
  { nopi | nopi | jefR rz, rl | nopi }                              # 0036  linea 3
```

```
  { nopi | nopi | nopi | nopi }                                    # 0037
  { nopi | nopi | jef .L_if_end_1 | nopi }                         # 0038  linea 2
  { nopi | nopi | nopi | nopi }                                    # 0039
.L_if_next_0:                                                      # bundle 40
.L_if_end_1:                                                       # bundle 40
  { mov t0, 1 | nopi | nopi | nopi }                               # 0040  linea 5
  { nopi | nopi | nopi | nopi }                                    # 0041
  { nopi | nopi | nopi | sw t0, 4(rz) }                            # 0042
  { mov t0, 2 | nopi | nopi | nopi }                               # 0043  linea 6
  { nopi | nopi | nopi | nopi }                                    # 0044
  { nopi | nopi | nopi | sw t0, 12(rz) }                           # 0045
  { nopi | nopi | nopi | nopi }                                    # 0046
.L_while_start_2:                                                  # bundle 47
  { nopi | nopi | nopi | lw t0, 12(rz) }                           # 0047  linea 7
  { nopi | nopi | nopi | lw t1, 8(rz) }                            # 0048
  { nopi | nopi | nopi | nopi }                                    # 0049
  { sle t0, t0, t1 | nopi | nopi | nopi }                          # 0050
  { nopi | nopi | nopi | nopi }                                    # 0051
  { nopi | nopi | beq t0, rz, .L_while_end_3 | nopi }               # 0052
  { nopi | nopi | nopi | nopi }                                    # 0053
  { mov r10, 1 | mov r6, 0 | nopi | lw t0, 4(rz) }                 # 0054  linea 8
  { nopi | nopi | nopi | lw t1, 12(rz) }                           # 0055
  { add r7, r4, r0 | nopi | nopi | nopi }                          # 0056
  { add r8, r5, r0 | nopi | nopi | nopi }                          # 0057
  { nopi | nopi | nopi | nopi }                                    # 0058
__ccom_arith_0_mul:                                                # bundle 59
  { nopi | nopi | beq r8, r0, __ccom_arith_2_end | nopi }           # 0059
  { nopi | nopi | nopi | nopi }                                    # 0060
  { and r9, r8, r10 | nopi | nopi | nopi }                         # 0061
  { nopi | nopi | nopi | nopi }                                    # 0062
  { nopi | nopi | beq r9, r0, __ccom_arith_1_skip | nopi }          # 0063
  { nopi | nopi | nopi | nopi }                                    # 0064
  { add r6, r6, r7 | nopi | nopi | nopi }                          # 0065
  { nopi | nopi | nopi | nopi }                                    # 0066
__ccom_arith_1_skip:                                               # bundle 67
  { sll r7, r7, r10 | srl r8, r8, r10 | jef __ccom_arith_0_mul | nopi }  # 0067
  { nopi | nopi | nopi | nopi }                                    # 0068
__ccom_arith_2_end:                                                # bundle 69
  { mov t1, 1 | nopi | nopi | sw t2, 4(rz) }                       # 0069  linea 9  slots 0=9,1=0,2=0,3=8
  { nopi | nopi | nopi | lw t0, 12(rz) }                           # 0070
  { nopi | nopi | nopi | nopi }                                    # 0071
  { add t0, t0, t1 | nopi | nopi | nopi }                          # 0072
  { nopi | nopi | nopi | nopi }                                    # 0073
  { nopi | nopi | jef .L_while_start_2 | sw t0, 12(rz) }           # 0074  linea 7  slots 0=0,1=0,2=7,3=9
  { nopi | nopi | nopi | nopi }                                    # 0075
.L_while_end_3:                                                    # bundle 76
  { nopi | nopi | jefR rz, rl | nopi }                             # 0076  linea 11
  { nopi | nopi | nopi | nopi }                                    # 0077
fin_main:                                                          # bundle 78
  { nopi | nopi | jef fin_main | nopi }                            # 0078  linea -
.L_arith_error:                                                    # bundle 79
  { nopi | nopi | jef .L_arith_error | nopi }                      # 0079

# Tabla de etiquetas (direccion = indice de bundle)
#   .L_func_main                    2
#   .L_body_main                    2
#   .L_func_factorial              28
#   .L_body_factorial              30
#   .L_if_next_0                   40
#   .L_if_end_1                    40
#   .L_while_start_2               47
#   __ccom_arith_0_mul             59  (interna)
#   __ccom_arith_1_skip            67  (interna)
#   __ccom_arith_2_end             69  (interna)
#   .L_while_end_3                 76
#   fin_main                       78
#   .L_arith_error                 79

# Variables (direccion de datos)
#
#   ===== SYMBOL TABLE =====
#   Scope 0: GLOBAL
#       main : void (function) params=[] area=0x0040 size=56 temp_base=0x0048
#       factorial : int (function) params=[n] area=0x0000 size=64 temp_base=0x0010
#
#   Scope 1: FUNCTION factorial
#       i : int width=4 addr=0x000C line=6
#       n : int (parameter) width=4 addr=0x0008 line=1
#       res : int (return) width=4 addr=0x0004 line=1
#
#   Scope 2: C factorial
#
#   Scope 3: L factorial
#
#   Scope 4: FUNCTION main
#       f : int width=4 addr=0x0044 line=16
#       x : int width=4 addr=0x0040 line=15
#
#   ======================
```

## *D-D. Binario en hexadecimal*

Cada entrada es el número de bundle y sus 128 bits, con el slot 3 a la izquierda; se leen por columnas.

```
0000 00000000_00000000_00000000_48800078
0001 00000000_00000000_00000000_00000000
0002 00000000_00000000_00000000_49000005
0003 00000000_00000000_00000000_00000000
0004 09000040_00000000_00000000_00000000
0005 00000000_00000000_00000000_49000000
0006 00000000_00000000_00000000_00000000
0007 09000044_00000000_00000000_00000000
0008 05000040_00000000_00000000_00000000
0009 00000000_00000000_00000000_00000000
0010 09080000_00000000_00000000_00000000
0011 00000000_00000000_00000000_40880004
0012 00000000_00000000_00000000_00000000
0013 00000000_00000000_00000000_408BFFFC
0014 00000000_00000000_00000000_00000000
0015 05080000_00000000_00000000_00000000
0016 00000000_00000000_00000000_00000000
0027 00000000_00000000_00000000_00000000
0028 08000004_00000000_00000000_00000000
0029 00000000_00000000_00000000_00000000
0030 05000008_00000000_00000000_49400000
0031 00000000_00000000_00000000_00000000
0032 00000000_00000000_00000000_25114000
0033 00000000_00000000_00000000_00000000
0034 00000000_04100006_00000000_00000000
0035 00000000_00000000_00000000_00000000
0036 00000000_14040000_00000000_00000000
0037 00000000_00000000_00000000_00000000
0038 00000000_40000002_00000000_00000000
0039 00000000_00000000_00000000_00000000
0040 00000000_00000000_00000000_49000001
0041 00000000_00000000_00000000_00000000
0042 09000004_00000000_00000000_00000000
0043 00000000_00000000_00000000_49000002
0054 05000004_00000000_49800000_4A800001
0055 0540000C_00000000_00000000_00000000
0056 00000000_00000000_00000000_05D00000
0057 00000000_00000000_00000000_06140000
0058 00000000_00000000_00000000_00000000
0059 00000000_0420000A_00000000_00000000
0060 00000000_00000000_00000000_00000000
0061 00000000_00000000_00000000_0E628000
0062 00000000_00000000_00000000_00000000
0063 00000000_04240004_00000000_00000000
0064 00000000_00000000_00000000_00000000
0065 00000000_00000000_00000000_0599C000
0066 00000000_00000000_00000000_00000000
0067 00000000_403FFFF8_1E228000_19DE8000
0068 00000000_00000000_00000000_00000000
0069 09800004_00000000_00000000_49400001
0070 0500000C_00000000_00000000_00000000
```

```
0017 09000008_00000000_00000000_00000000   0044 00000000_00000000_00000000_00000000   0071 00000000_00000000_00000000_00000000
0018 08480000_00000000_00000000_00000000   0045 0900000C_00000000_00000000_00000000   0072 00000000_00000000_00000000_05114000
0019 00000000_44400009_00000000_40880004   0046 00000000_00000000_00000000_00000000   0073 00000000_00000000_00000000_00000000
0020 00000000_00000000_00000000_00000000   0047 0500000C_00000000_00000000_00000000   0074 0900000C_403FFFE5_00000000_00000000
0021 00000000_00000000_00000000_408BFFFC   0048 05400008_00000000_00000000_00000000   0075 00000000_00000000_00000000_00000000
0022 00000000_00000000_00000000_00000000   0049 00000000_00000000_00000000_00000000   0076 00000000_14040000_00000000_00000000
0023 04480000_00000000_00000000_00000000   0050 00000000_00000000_00000000_29114000   0077 00000000_00000000_00000000_00000000
0024 05000004_00000000_00000000_00000000   0051 00000000_00000000_00000000_00000000   0078 00000000_40000000_00000000_00000000
0025 00000000_00000000_00000000_00000000   0052 00000000_04100018_00000000_00000000   0079 00000000_40000000_00000000_00000000
0026 09000044_40000034_00000000_00000000   0053 00000000_00000000_00000000_00000000
```

## *D-E. Traza de ejecución*

```
# Traza de ejecución (enunciado CE1108 §8.4)
# Cada línea: [bundle N · línea L del .cls] evento.
# No se muestran las escrituras fuera de las variables del listado:
# la pila de llamadas (rx), las casillas ocultas del S y los temporales
# que se salvan en las llamadas.
# No se muestran los saltos hacia las rutinas internas
# (__ccom_arith_*, __ccom_array_*, .L_crypto_*); sí el salto a
# .L_arith_error. Los saltos condicionales no tomados tampoco.
[bundle 0004 · línea 15] x = 5
[bundle 0007 · línea 16] f = 0
[bundle 0017 · línea 17] n = 5
[bundle 0019 · línea 17] llamada → .L_func_factorial (bundle 28)
[bundle 0028 · línea 1] res = 0  (prólogo)
[bundle 0034 · línea 2] beq → .L_if_next_0 (bundle 40)
[bundle 0042 · línea 5] res = 1
[bundle 0045 · línea 6] i = 2
[bundle 0069 · línea 8] res = 2
[bundle 0074 · línea 9] i = 3
[bundle 0074 · línea 7] jef → .L_while_start_2 (bundle 47)
[bundle 0069 · línea 8] res = 6
[bundle 0074 · línea 9] i = 4
[bundle 0074 · línea 7] jef → .L_while_start_2 (bundle 47)
[bundle 0069 · línea 8] res = 24
[bundle 0074 · línea 9] i = 5
[bundle 0074 · línea 7] jef → .L_while_start_2 (bundle 47)
[bundle 0069 · línea 8] res = 120
[bundle 0074 · línea 9] i = 6
[bundle 0074 · línea 7] jef → .L_while_start_2 (bundle 47)
[bundle 0052 · línea 7] beq → .L_while_end_3 (bundle 76)
[bundle 0076 · línea 11] retorno → bundle 20
[bundle 0026 · línea 17] f = 120
[bundle 0026 · línea 18] jef → fin_main (bundle 78)
[bundle 0078] jef → fin_main (bundle 78)

Estado final de las variables:
  res = 120
  n = 5
  i = 6
  x = 5
  f = 120
```

## *D-F. Resultado de la ejecución*

```
Ejecucion de ejercicio1 con tools/sim_asm.py
Programa: ejercicios/ejercicio1_factorial/ejercicio1.asm (80 bundles)
Fin: salto a sí mismo; bundles ejecutados: 203; ciclos: 231
Avisos de separación: 0

Valores finales de las variables:
  res = 120
  n = 5
  i = 6
  x = 5
  f = 120

Resultado esperado contra obtenido:
  f: esperado 120, obtenido 120  OK

Veredicto: OK
```

# Apéndice E
# Ejercicio 2: búsqueda en matriz

## *E-A. Evidencia de compilación*

```
$ ./ccom -v -m -s -p ejercicios/ejercicio2_matriz/ejercicio2.cls -o ejercicios/ejercicio2_matriz/ejercicio2.bin
codigo de salida: 0

===== Fases (stderr, -v) =====
ccom: fase 1-2: analisis lexico y sintactico ... ok
ccom: fase 3: analisis semantico ... ok
ccom: fase 3: tabla de etiquetas ... ok
ccom: fase 4: generacion de codigo ... ok
ccom: fase 4: organizacion de bundles ... ok
ccom: ensamblado ... ok
```

## *E-B. Código fuente*

```
Vi m : int [3][4] = [ [  4,  8, 15, 16 ],
                      [ 23, 42,  7,  1 ],
                      [  9,  3,  5,  2 ] ] ;

F buscar [ Vi int mat [3][4], int x, int pos [2] ] : bool hallado ;
    pos[0] = -1 ;
    pos[1] = -1 ;
    S [ i, 0, 3, 1 ] ;
        S [ j, 0, 4, 1 ] ;
            C [ mat[i][j] == x ] ;
                pos[0] = i ;
                pos[1] = j ;
                hallado = verdadero ;
                E ;
            .
        .
    .
    E ;
.

F main [] : void ;
    Vm p : int [2] ;
    Vm ok : bool ;
    ok = buscar [ m, 42, p ] ;
    E ;
.
```

## *E-C. Ensamblador*

```
# ccom: ejercicios/ejercicio2_matriz/ejercicio2.cls
  { mov rx, 192 | nopi | nopi | nopi }                    # 0000
  { nopi | nopi | nopi | nopi }                          # 0001
.L_func_main:                                            # bundle 2
.L_body_main:                                            # bundle 2
  { mov t0, 0 | nopi | nopi | nopi }                     # 0002  linea 22
  { nopi | nopi | nopi | nopi }                          # 0003
  { nopi | nopi | nopi | sw t0, 132(rz) }                # 0004
  { mov t0, 0 | nopi | nopi | nopi }                     # 0005
  { nopi | nopi | nopi | nopi }                          # 0006
  { nopi | nopi | nopi | sw t0, 136(rz) }                # 0007
  { mov t0, 0 | nopi | nopi | nopi }                     # 0008  linea 23
  { nopi | nopi | nopi | nopi }                          # 0009
  { nopi | nopi | nopi | sw t0, 140(rz) }                # 0010
  { mov t0, 42 | nopi | nopi | nopi }                    # 0011  linea 24
  { nopi | nopi | nopi | nopi }                          # 0012
  { nopi | nopi | nopi | sw t0, 0(rx) }                  # 0013
  { saddi rx, rx, 4 | mov r4, 132 | nopi | nopi }        # 0014
  { nopi | nopi | nopi | nopi }                          # 0015
  { saddi rx, rx, -4 | nopi | nopi | sw r4, 64(r0) }     # 0016
  { nopi | nopi | nopi | nopi }                          # 0017
  { nopi | nopi | nopi | lw t0, 0(rx) }                  # 0018
  { nopi | nopi | nopi | nopi }                          # 0019
  { nopi | nopi | nopi | sw t0, 60(rz) }                 # 0020
  { mov r4, 0 | nopi | nopi | nopi }                     # 0021
  { nopi | nopi | nopi | nopi }                          # 0022
  { nopi | nopi | nopi | sw r4, 56(r0) }                 # 0023
  { nopi | nopi | nopi | sw rl, 0(rx) }                  # 0024
  { saddi rx, rx, 4 | nopi | jas rl, .L_func_buscar | nopi }  # 0025
  { nopi | nopi | nopi | nopi }                          # 0026
  { saddi rx, rx, -4 | nopi | nopi | nopi }              # 0027
  { nopi | nopi | nopi | nopi }                          # 0028
  { nopi | nopi | nopi | lw rl, 0(rx) }                  # 0029
  { nopi | nopi | nopi | lw t0, 52(rz) }                 # 0030
  { nopi | nopi | nopi | nopi }                          # 0031
  { nopi | nopi | jef fin_main | sw t0, 140(rz) }        # 0032  linea 25  slots 0=0,1=0,2=25,3=24
  { nopi | nopi | nopi | nopi }                          # 0033
.L_func_buscar:                                          # bundle 34
  { nopi | nopi | nopi | sw rz, 52(rz) }                 # 0034  linea 5
  { nopi | nopi | nopi | nopi }                          # 0035
.L_body_buscar:                                          # bundle 36
  { mov t0, 1 | mov t1, 0 | nopi | lw r8, 64(r0) }       # 0036  linea 6
  { mov r7, 2 | nopi | nopi | nopi }                     # 0037
  { sub t0, rz, t0 | add r6, r5, r0 | nopi | nopi }      # 0038
  { nopi | nopi | nopi | nopi }                          # 0039
```

```
  { sll r6, r6, r7 | nopi | nopi | nopi }                    # 0040
  { mov r7, 2 | nopi | nopi | nopi }                         # 0041
  { add r6, r6, r8 | nopi | nopi | nopi }                    # 0042
  { add r6, r5, r0 | nopi | nopi | lw r8, 64(r0) }           # 0043
  { mov t1, 1 | nopi | nopi | nopi }                         # 0044  linea 7
  { sll r6, r6, r7 | nopi | nopi | nopi }                    # 0045  linea 6
  { mov r7, 2 | nopi | nopi | nopi }                         # 0046  linea 7
  { add r6, r6, r8 | nopi | nopi | nopi }                    # 0047  linea 6
  { nopi | nopi | nopi | nopi }                              # 0048
  { nopi | nopi | nopi | sw r4, 0(r6) }                      # 0049
  { mov t0, 1 | add r6, r5, r0 | nopi | lw r8, 64(r0) }      # 0050  linea 7
  { nopi | nopi | nopi | nopi }                              # 0051
  { sub t0, rz, t0 | sll r6, r6, r7 | nopi | nopi }          # 0052
  { mov r7, 2 | nopi | nopi | nopi }                         # 0053
  { add r6, r6, r8 | nopi | nopi | nopi }                    # 0054
  { add r6, r5, r0 | nopi | nopi | lw r8, 64(r0) }           # 0055
  { mov t1, 3 | nopi | nopi | nopi }                         # 0056  linea 8
  { sll r6, r6, r7 | nopi | nopi | nopi }                    # 0057  linea 7
  { nopi | nopi | nopi | nopi }                              # 0058
  { add r6, r6, r8 | nopi | nopi | nopi }                    # 0059
  { nopi | nopi | nopi | nopi }                              # 0060
  { nopi | nopi | nopi | sw r4, 0(r6) }                      # 0061
  { mov t0, 0 | nopi | nopi | nopi }                         # 0062  linea 8
  { nopi | nopi | nopi | nopi }                              # 0063
  { nopi | nopi | nopi | sw t0, 68(rz) }                     # 0064
  { nopi | nopi | nopi | sw t1, 72(rz) }                     # 0065
  { nopi | nopi | nopi | nopi }                              # 0066
.L_for_start_0:                                             # bundle 67
  { nopi | nopi | nopi | lw t0, 68(rz) }                     # 0067
  { nopi | nopi | nopi | lw t1, 72(rz) }                     # 0068
  { nopi | nopi | nopi | nopi }                              # 0069
  { nopi | nopi | bge t0, t1, .L_for_end_2 | nopi }          # 0070
  { nopi | nopi | nopi | nopi }                              # 0071
  { mov t0, 0 | mov t1, 4 | nopi | nopi }                    # 0072  linea 9
  { nopi | nopi | nopi | nopi }                              # 0073
  { nopi | nopi | nopi | sw t0, 76(rz) }                     # 0074
  { nopi | nopi | nopi | sw t1, 80(rz) }                     # 0075
  { nopi | nopi | nopi | nopi }                              # 0076
.L_for_start_3:                                             # bundle 77
  { nopi | nopi | nopi | lw t0, 76(rz) }                     # 0077
  { nopi | nopi | nopi | lw t1, 80(rz) }                     # 0078
  { nopi | nopi | nopi | nopi }                              # 0079
  { nopi | nopi | bge t0, t1, .L_for_end_5 | nopi }          # 0080
  { nopi | nopi | nopi | nopi }                              # 0081
  { mov r7, 1 | nopi | nopi | lw t0, 68(rz) }                # 0082  linea 10
  { nopi | nopi | nopi | lw t1, 76(rz) }                     # 0083
  { add r6, r4, r0 | nopi | nopi | lw r8, 56(r0) }           # 0084
  { nopi | nopi | nopi | nopi }                              # 0085
  { sll r6, r6, r7 | nopi | nopi | nopi }                    # 0086
  { nopi | nopi | nopi | nopi }                              # 0087
  { sll r6, r6, r7 | nopi | nopi | nopi }                    # 0088
  { mov r7, 2 | nopi | nopi | nopi }                         # 0089
  { add r6, r6, r5 | nopi | nopi | nopi }                    # 0090
  { nopi | nopi | nopi | nopi }                              # 0091
  { sll r6, r6, r7 | nopi | nopi | nopi }                    # 0092
  { mov r7, 1 | nopi | nopi | nopi }                         # 0093
  { add r6, r6, r8 | nopi | nopi | nopi }                    # 0094
  { add r6, r4, r0 | nopi | nopi | lw r8, 56(r0) }           # 0095
  { nopi | nopi | nopi | nopi }                              # 0096
  { sll r6, r6, r7 | nopi | nopi | nopi }                    # 0097
  { nopi | nopi | nopi | nopi }                              # 0098
  { sll r6, r6, r7 | nopi | nopi | nopi }                    # 0099
  { mov r7, 2 | nopi | nopi | nopi }                         # 0100
  { add r6, r6, r5 | nopi | nopi | nopi }                    # 0101
  { nopi | nopi | nopi | nopi }                              # 0102
  { sll r6, r6, r7 | nopi | nopi | nopi }                    # 0103
  { nopi | nopi | nopi | nopi }                              # 0104
  { add r6, r6, r8 | nopi | nopi | nopi }                    # 0105
  { nopi | nopi | nopi | nopi }                              # 0106
  { nopi | nopi | nopi | lw r9, 0(r6) }                      # 0107
  { nopi | nopi | nopi | lw t0, 60(rz) }                     # 0108
  { nopi | nopi | nopi | nopi }                              # 0109
  { seq t5, t5, t0 | nopi | nopi | nopi }                    # 0110
  { nopi | nopi | nopi | nopi }                              # 0111
  { nopi | nopi | beq t5, rz, .L_if_next_6 | nopi }          # 0112
  { nopi | nopi | nopi | nopi }                              # 0113
  { mov t1, 0 | mov r7, 2 | nopi | lw t0, 68(rz) }           # 0114  linea 11
  { nopi | nopi | nopi | lw r8, 64(r0) }                     # 0115
  { add r6, r5, r0 | nopi | nopi | nopi }                    # 0116
  { nopi | nopi | nopi | nopi }                              # 0117
  { sll r6, r6, r7 | nopi | nopi | nopi }                    # 0118
  { mov r7, 2 | nopi | nopi | nopi }                         # 0119
  { add r6, r6, r8 | nopi | nopi | nopi }                    # 0120
  { add r6, r5, r0 | nopi | nopi | lw r8, 64(r0) }           # 0121
  { mov t1, 1 | nopi | nopi | nopi }                         # 0122  linea 12
  { sll r6, r6, r7 | nopi | nopi | nopi }                    # 0123  linea 11
  { mov r7, 2 | nopi | nopi | nopi }                         # 0124  linea 12
  { add r6, r6, r8 | nopi | nopi | nopi }                    # 0125  linea 11
  { nopi | nopi | nopi | nopi }                              # 0126
  { nopi | nopi | nopi | sw r4, 0(r6) }                      # 0127
  { add r6, r5, r0 | nopi | nopi | lw t0, 76(rz) }           # 0128  linea 12
  { nopi | nopi | nopi | lw r8, 64(r0) }                     # 0129
  { sll r6, r6, r7 | nopi | nopi | nopi }                    # 0130
  { mov r7, 2 | nopi | nopi | nopi }                         # 0131
  { add r6, r6, r8 | nopi | nopi | nopi }                    # 0132
  { add r6, r5, r0 | nopi | nopi | lw r8, 64(r0) }           # 0133
  { nopi | nopi | nopi | nopi }                              # 0134
  { sll r6, r6, r7 | nopi | nopi | nopi }                    # 0135
  { nopi | nopi | nopi | nopi }                              # 0136
  { add r6, r6, r8 | nopi | nopi | nopi }                    # 0137
  { nopi | nopi | nopi | nopi }                              # 0138
  { nopi | nopi | nopi | sw r4, 0(r6) }                      # 0139
  { mov t0, 1 | nopi | nopi | nopi }                         # 0140  linea 13
  { nopi | nopi | nopi | nopi }                              # 0141
  { nopi | nopi | jefR rz, rl | sw t0, 52(rz) }              # 0142  linea 14  slots 0=0,1=0,2=14,3=13
  { nopi | nopi | nopi | nopi }                              # 0143
  { nopi | nopi | jef .L_if_end_7 | nopi }                   # 0144  linea 10
  { nopi | nopi | nopi | nopi }                              # 0145
.L_if_next_6:                                               # bundle 146
.L_if_end_7:                                                # bundle 146
.L_for_update_4:                                            # bundle 146
  { nopi | nopi | nopi | lw t0, 76(rz) }                     # 0146  linea 9
  { nopi | nopi | nopi | nopi }                              # 0147
  { saddi t0, t0, 1 | nopi | nopi | nopi }                   # 0148
  { nopi | nopi | nopi | nopi }                              # 0149
```

```
  { nopi | nopi | jef .L_for_start_3 | sw t0, 76(rz) }        # 0150
  { nopi | nopi | nopi | nopi }                               # 0151
.L_for_end_5:                                                 # bundle 152
.L_for_update_1:                                              # bundle 152
  { nopi | nopi | nopi | lw t0, 68(rz) }                      # 0152  linea 8
  { nopi | nopi | nopi | nopi }                               # 0153
  { saddi t0, t0, 1 | nopi | nopi | nopi }                    # 0154
  { nopi | nopi | nopi | nopi }                               # 0155
  { nopi | nopi | jef .L_for_start_0 | sw t0, 68(rz) }        # 0156
  { nopi | nopi | nopi | nopi }                               # 0157
.L_for_end_2:                                                 # bundle 158
  { nopi | nopi | jefR rz, rl | nopi }                        # 0158  linea 18
  { nopi | nopi | nopi | nopi }                               # 0159
fin_main:                                                     # bundle 160
  { nopi | nopi | jef fin_main | nopi }                       # 0160  linea -
.L_arith_error:                                               # bundle 161
  { nopi | nopi | jef .L_arith_error | nopi }                 # 0161

# Tabla de etiquetas (direccion = indice de bundle)
#   .L_func_main                    2
#   .L_body_main                    2
#   .L_func_buscar                 34
#   .L_body_buscar                 36
#   .L_for_start_0                 67
#   .L_for_start_3                 77
#   .L_if_next_6                  146
#   .L_if_end_7                   146
#   .L_for_update_4               146
#   .L_for_end_5                  152
#   .L_for_update_1               152
#   .L_for_end_2                  158
#   fin_main                      160
#   .L_arith_error                161

# Variables (direccion de datos)
#
#   ===== SYMBOL TABLE =====
#   Scope 0: GLOBAL
#       main : void (function) params=[] area=0x0084 size=60 temp_base=0x0090
#       buscar : bool (function) params=[mat, x, pos] area=0x0030 size=84 temp_base=0x0054
#       m : int[3][4] width=48 addr=0x0000 line=1
#
#   Scope 1: FUNCTION buscar
#       pos : int[2] (parameter) width=4 addr=0x0040 line=5
#       x : int (parameter) width=4 addr=0x003C line=5
#       mat : int[3][4] (parameter) width=4 addr=0x0038 line=5
#       hallado : bool (return) width=4 addr=0x0034 line=5
#
#   Scope 2: S buscar
#       i : int width=4 addr=0x0044 line=8
#
#   Scope 3: S buscar
#       j : int width=4 addr=0x004C line=9
#
#   Scope 4: C buscar
#
#   Scope 5: FUNCTION main
#       ok : bool width=4 addr=0x008C line=23
#       p : int[2] width=8 addr=0x0084 line=22
#
#   ======================
```

## *E-D. Binario en hexadecimal*

Cada entrada es el número de bundle y sus 128 bits, con el slot 3 a la izquierda; se leen por columnas.

```
0000 00000000_00000000_00000000_488000C0
0001 00000000_00000000_00000000_00000000
0002 00000000_00000000_00000000_49000000
0003 00000000_00000000_00000000_00000000
0004 09000084_00000000_00000000_00000000
0005 00000000_00000000_00000000_49000000
0006 00000000_00000000_00000000_00000000
0007 09000088_00000000_00000000_00000000
0008 00000000_00000000_00000000_49000000
0009 00000000_00000000_00000000_00000000
0010 0900008C_00000000_00000000_00000000
0011 00000000_00000000_00000000_4900002A
0012 00000000_00000000_00000000_00000000
0013 09080000_00000000_00000000_00000000
0014 00000000_00000000_49000084_40880004
0015 00000000_00000000_00000000_00000000
0016 09000040_00000000_00000000_408BFFFC
0017 00000000_00000000_00000000_00000000
0018 05080000_00000000_00000000_00000000
0019 00000000_00000000_00000000_00000000
0020 0900003C_00000000_00000000_00000000
0021 00000000_00000000_00000000_49000000
0022 00000000_00000000_00000000_00000000
0023 09000038_00000000_00000000_00000000
0024 08480000_00000000_00000000_00000000
0025 00000000_44400009_00000000_40880004
0026 00000000_00000000_00000000_00000000
0027 00000000_00000000_00000000_408BFFFC
0028 00000000_00000000_00000000_00000000
0029 04480000_00000000_00000000_00000000
0030 05000034_00000000_00000000_00000000
0031 00000000_00000000_00000000_00000000
0032 0900008C_40000080_00000000_00000000
0033 00000000_00000000_00000000_00000000
0034 08000034_00000000_00000000_00000000
0035 00000000_00000000_00000000_00000000
0036 06000040_00000000_49400000_49000001
0037 00000000_00000000_00000000_49C00002
0038 00000000_00000000_05940000_09010000
0039 00000000_00000000_00000000_00000000
0040 00000000_00000000_00000000_1999C000
0041 00000000_00000000_00000000_49C00002
0042 00000000_00000000_00000000_059A0000
0043 06000040_00000000_00000000_05940000
0044 00000000_00000000_00000000_49400001
0054 00000000_00000000_00000000_059A0000
0055 06000040_00000000_00000000_05940000
0056 00000000_00000000_00000000_49400003
0057 00000000_00000000_00000000_1999C000
0058 00000000_00000000_00000000_00000000
0059 00000000_00000000_00000000_059A0000
0060 00000000_00000000_00000000_00000000
0061 09180000_00000000_00000000_00000000
0062 00000000_00000000_00000000_49000000
0063 00000000_00000000_00000000_00000000
0064 09000044_00000000_00000000_00000000
0065 09400048_00000000_00000000_00000000
0066 00000000_00000000_00000000_00000000
0067 05000044_00000000_00000000_00000000
0068 05400048_00000000_00000000_00000000
0069 00000000_00000000_00000000_00000000
0070 00000000_11500058_00000000_00000000
0071 00000000_00000000_00000000_00000000
0072 00000000_00000000_49400004_49000000
0073 00000000_00000000_00000000_00000000
0074 0900004C_00000000_00000000_00000000
0075 09400050_00000000_00000000_00000000
0076 00000000_00000000_00000000_00000000
0077 0500004C_00000000_00000000_00000000
0078 05400050_00000000_00000000_00000000
0079 00000000_00000000_00000000_00000000
0080 00000000_11500048_00000000_00000000
0081 00000000_00000000_00000000_00000000
0082 05000044_00000000_00000000_49C00001
0083 0540004C_00000000_00000000_00000000
0084 06000038_00000000_00000000_05900000
0085 00000000_00000000_00000000_00000000
0086 00000000_00000000_00000000_1999C000
0087 00000000_00000000_00000000_00000000
0088 00000000_00000000_00000000_1999C000
0089 00000000_00000000_00000000_49C00002
0090 00000000_00000000_00000000_05994000
0091 00000000_00000000_00000000_00000000
0092 00000000_00000000_00000000_1999C000
0093 00000000_00000000_00000000_49C00001
0094 00000000_00000000_00000000_059A0000
0095 06000038_00000000_00000000_05900000
0096 00000000_00000000_00000000_00000000
0097 00000000_00000000_00000000_1999C000
0098 00000000_00000000_00000000_00000000
0108 0500003C_00000000_00000000_00000000
0109 00000000_00000000_00000000_00000000
0110 00000000_00000000_00000000_2E650000
0111 00000000_00000000_00000000_00000000
0112 00000000_04240022_00000000_00000000
0113 00000000_00000000_00000000_00000000
0114 05000044_00000000_49C00002_49400000
0115 06000040_00000000_00000000_00000000
0116 00000000_00000000_00000000_05940000
0117 00000000_00000000_00000000_00000000
0118 00000000_00000000_00000000_1999C000
0119 00000000_00000000_00000000_49C00002
0120 00000000_00000000_00000000_059A0000
0121 06000040_00000000_00000000_05940000
0122 00000000_00000000_00000000_49400001
0123 00000000_00000000_00000000_1999C000
0124 00000000_00000000_00000000_49C00002
0125 00000000_00000000_00000000_059A0000
0126 00000000_00000000_00000000_00000000
0127 09180000_00000000_00000000_00000000
0128 0500004C_00000000_00000000_05940000
0129 06000040_00000000_00000000_00000000
0130 00000000_00000000_00000000_1999C000
0131 00000000_00000000_00000000_49C00002
0132 00000000_00000000_00000000_059A0000
0133 06000040_00000000_00000000_05940000
0134 00000000_00000000_00000000_00000000
0135 00000000_00000000_00000000_1999C000
0136 00000000_00000000_00000000_00000000
0137 00000000_00000000_00000000_059A0000
0138 00000000_00000000_00000000_00000000
0139 09180000_00000000_00000000_00000000
0140 00000000_00000000_00000000_49000001
0141 00000000_00000000_00000000_00000000
0142 09000034_14040000_00000000_00000000
0143 00000000_00000000_00000000_00000000
0144 00000000_40000002_00000000_00000000
0145 00000000_00000000_00000000_00000000
0146 0500004C_00000000_00000000_00000000
0147 00000000_00000000_00000000_00000000
0148 00000000_00000000_00000000_41100001
0149 00000000_00000000_00000000_00000000
0150 0900004C_403FFFB7_00000000_00000000
0151 00000000_00000000_00000000_00000000
0152 05000044_00000000_00000000_00000000
```

```
0045 00000000_00000000_00000000_1999C000   0099 00000000_00000000_00000000_1999C000   0153 00000000_00000000_00000000_00000000
0046 00000000_00000000_00000000_49C00002   0100 00000000_00000000_00000000_49C00002   0154 00000000_00000000_00000000_41100001
0047 00000000_00000000_00000000_059A0000   0101 00000000_00000000_00000000_05994000   0155 00000000_00000000_00000000_00000000
0048 00000000_00000000_00000000_00000000   0102 00000000_00000000_00000000_00000000   0156 09000044_403FFFA7_00000000_00000000
0049 09180000_00000000_00000000_00000000   0103 00000000_00000000_00000000_1999C000   0157 00000000_00000000_00000000_00000000
0050 06000040_00000000_05940000_49000001   0104 00000000_00000000_00000000_00000000   0158 00000000_14040000_00000000_00000000
0051 00000000_00000000_00000000_00000000   0105 00000000_00000000_00000000_059A0000   0159 00000000_00000000_00000000_00000000
0052 00000000_00000000_1999C000_09010000   0106 00000000_00000000_00000000_00000000   0160 00000000_40000000_00000000_00000000
0053 00000000_00000000_00000000_49C00002   0107 06580000_00000000_00000000_00000000   0161 00000000_40000000_00000000_00000000
```

## *E-E. Traza de ejecución*

```
# Traza de ejecución (enunciado CE1108 §8.4)
# Cada línea: [bundle N · línea L del .cls] evento.
# No se muestran las escrituras fuera de las variables del listado:
# la pila de llamadas (rx), las casillas ocultas del S y los temporales
# que se salvan en las llamadas.
# No se muestran los saltos hacia las rutinas internas
# (__ccom_arith_*, __ccom_array_*, .L_crypto_*); sí el salto a
# .L_arith_error. Los saltos condicionales no tomados tampoco.
[bundle 0004 · línea 22] p[0] = 0
[bundle 0007 · línea 22] p[1] = 0
[bundle 0010 · línea 23] ok = falso
[bundle 0016 · línea 24] pos = @0x0084
[bundle 0020 · línea 24] x = 42
[bundle 0023 · línea 24] mat = @0x0000
[bundle 0025 · línea 24] llamada → .L_func_buscar (bundle 34)
[bundle 0034 · línea 5] hallado = falso  (prólogo)
[bundle 0049 · línea 6] p[0] = -1
[bundle 0061 · línea 7] p[1] = -1
[bundle 0064 · línea 8] i = 0
[bundle 0074 · línea 9] j = 0
[bundle 0112 · línea 10] beq → .L_if_next_6 (bundle 146)
[bundle 0150 · línea 9] j = 1
[bundle 0150 · línea 9] jef → .L_for_start_3 (bundle 77)
[bundle 0112 · línea 10] beq → .L_if_next_6 (bundle 146)
[bundle 0150 · línea 9] j = 2
[bundle 0150 · línea 9] jef → .L_for_start_3 (bundle 77)
[bundle 0112 · línea 10] beq → .L_if_next_6 (bundle 146)
[bundle 0150 · línea 9] j = 3
[bundle 0150 · línea 9] jef → .L_for_start_3 (bundle 77)
[bundle 0112 · línea 10] beq → .L_if_next_6 (bundle 146)
[bundle 0150 · línea 9] j = 4
[bundle 0150 · línea 9] jef → .L_for_start_3 (bundle 77)
[bundle 0080 · línea 9] bge → .L_for_end_5 (bundle 152)
[bundle 0156 · línea 8] i = 1
[bundle 0156 · línea 8] jef → .L_for_start_0 (bundle 67)
[bundle 0074 · línea 9] j = 0
[bundle 0112 · línea 10] beq → .L_if_next_6 (bundle 146)
[bundle 0150 · línea 9] j = 1
[bundle 0150 · línea 9] jef → .L_for_start_3 (bundle 77)
[bundle 0127 · línea 11] p[0] = 1
[bundle 0139 · línea 12] p[1] = 1
[bundle 0142 · línea 13] hallado = verdadero
[bundle 0142 · línea 14] retorno → bundle 26
[bundle 0032 · línea 24] ok = verdadero
[bundle 0032 · línea 25] jef → fin_main (bundle 160)
[bundle 0160] jef → fin_main (bundle 160)

Estado final de las variables:
  m = [4, 8, 15, 16, 23, 42, 7, 1, 9, 3, 5, 2]
  hallado = verdadero
  mat = @0x0000
  x = 42
  pos = @0x0084
  i = 1
  j = 1
  p = [1, 1]
  ok = verdadero
```

## *E-F. Resultado de la ejecución*

```
Ejecucion de ejercicio2 con tools/sim_asm.py
Programa: ejercicios/ejercicio2_matriz/ejercicio2.asm (162 bundles)
Fin: salto a sí mismo; bundles ejecutados: 367; ciclos: 383
Avisos de separación: 0

Valores finales de las variables:
  m = [4, 8, 15, 16, 23, 42, 7, 1, 9, 3, 5, 2]
  hallado = verdadero
  mat = @0x0000
  x = 42
  pos = @0x0084
  i = 1
  j = 1
  p = [1, 1]
  ok = verdadero

Resultado esperado contra obtenido:
  ok: esperado verdadero, obtenido verdadero  OK
  p: esperado [1, 1], obtenido [1, 1]  OK

Veredicto: OK
```

# Apéndice F
# Ejercicio 3: cifrado por segmentos

## *F-A. Evidencia de compilación*

```
$ ./ccom -v -m -s -p ejercicios/ejercicio3_cifrado/ejercicio3.cls -o ejercicios/ejercicio3_cifrado/ejercicio3.bin
codigo de salida: 0

===== Fases (stderr, -v) =====
ccom: fase 1-2: analisis lexico y sintactico ... ok
ccom: fase 3: analisis semantico ... ok
ccom: fase 3: tabla de etiquetas ... ok
ccom: fase 4: generacion de codigo ... ok
ccom: fase 4: organizacion de bundles ... ok
ccom: ensamblado ... ok
```

## *F-B. Código fuente*

```
Vi origen : int = 4096 ;
Vi cifrado : int = 8192 ;
Vi vuelta : int = 12288 ;

F bloques_de [ int total , int tam , int i ] : int b ;
    Vm resto : int = total - i * tam ;
    C [ resto < tam ] ;
        b = resto ;
    A ;
        b = tam ;
    .
    E ;
.

F direccion [ int base , int tam , int i ] : int d ;
    d = base + i * tam * 8 ;
    E ;
.

F preparar_boveda [] : void ;
    cargar_llave [ 0 , 0 , 19114957 ] ;
    cargar_llave [ 0 , 1 , -1985229329 ] ;
    cargar_llave [ 0 , 2 , 324508639 ] ;
    cargar_llave [ 0 , 3 , 610839776 ] ;
    cargar_llave [ 1 , 0 , 286331153 ] ;
    cargar_llave [ 1 , 1 , 572662306 ] ;
    cargar_llave [ 1 , 2 , 858993459 ] ;
    cargar_llave [ 1 , 3 , 1145324612 ] ;
    sellar [ ] ;
    E ;
.

F cifrar_archivo [ int desde , int hacia , int total , int tam ] : int n ;
    n = ( total + tam - 1 ) / tam ;
    S [ i , 0 , n , 1 ] ;
        Vm b : int = bloques_de [ total , tam , i ] ;
        C [ i % 2 == 0 ] ;
            cifrar [ direccion [ desde , tam , i ] , direccion [ hacia , tam , i ] , b , 0 ] ;
        A ;
            cifrar [ direccion [ desde , tam , i ] , direccion [ hacia , tam , i ] , b , 1 ] ;
        .
    .
    E ;
.

F descifrar_archivo [ int desde , int hacia , int total , int tam ] : int n ;
    n = ( total + tam - 1 ) / tam ;
    S [ i , 0 , n , 1 ] ;
        Vm b : int = bloques_de [ total , tam , i ] ;
        C [ i % 2 == 0 ] ;
            descifrar [ direccion [ desde , tam , i ] , direccion [ hacia , tam , i ] , b , 0 ] ;
        A ;
            descifrar [ direccion [ desde , tam , i ] , direccion [ hacia , tam , i ] , b , 1 ] ;
        .
    .
    E ;
.

F main [] : void ;
    Vm n1 : int ;
    Vm n2 : int ;
    autenticar [ 1234567 ] ;
    preparar_boveda [ ] ;
    n1 = cifrar_archivo [ origen , cifrado , 128 , 16 ] ;
    n2 = descifrar_archivo [ cifrado , vuelta , 128 , 16 ] ;
```

```
    cerrar_sesion [ ] ;
    E ;
.
```

## *F-C. Ensamblador*

```
# ccom: ejercicios/ejercicio3_cifrado/ejercicio3.cls
  { mov rx, 428 | nopi | nopi | nopi }                                  # 0000
  { nopi | nopi | nopi | nopi }                                        # 0001
.L_func_main:                                                          # bundle 2
.L_body_main:                                                          # bundle 2
  { mov t0, 0 | nopi | nopi | nopi }                                   # 0002  linea 60
  { nopi | nopi | nopi | nopi }                                        # 0003
  { nopi | nopi | nopi | sw t0, 372(rz) }                              # 0004
  { mov t0, 0 | nopi | nopi | nopi }                                   # 0005  linea 61
  { nopi | nopi | nopi | nopi }                                        # 0006
  { nopi | nopi | nopi | sw t0, 376(rz) }                              # 0007
  { nopi | nopi | nopi | nopi }                                        # 0008
  { nopi | nopi | nopi | nopi }                                        # 0009
  { mov t0, 37 | mov t1, 15 | nopi | nopi }                            # 0010  linea 62
  { nopi | nopi | nopi | nopi }                                        # 0011
  { sll t0, t0, t1 | nopi | nopi | nopi }                              # 0012
  { nopi | nopi | nopi | nopi }                                        # 0013
  { saddi t0, t0, 22151 | nopi | nopi | nopi }                         # 0014
  { nopi | nopi | nopi | nopi }                                        # 0015
  { saddi t6, t0, 0 | nopi | nopi | nopi }                             # 0016
  { nopi | nopi | nopi | nopi }                                        # 0017
  { nopi | nopi | nopi | nopi }                                        # 0018
  { nopi | auth t6 | nopi | nopi }                                     # 0019
  { nopi | nopi | nopi | sw rl, 0(rx) }                                # 0020  linea 63
  { saddi rx, rx, 4 | nopi | jas rl, .L_func_preparar_boveda | nopi }  # 0021
  { nopi | nopi | nopi | nopi }                                        # 0022
  { saddi rx, rx, -4 | nopi | nopi | nopi }                            # 0023
  { nopi | nopi | nopi | nopi }                                        # 0024
  { nopi | nopi | nopi | lw rl, 0(rx) }                                # 0025
  { nopi | nopi | nopi | lw t0, 0(rz) }                                # 0026  linea 64
  { nopi | nopi | nopi | nopi }                                        # 0027
  { nopi | nopi | nopi | sw t0, 0(rx) }                                # 0028
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 4(rz) }                     # 0029
  { nopi | nopi | nopi | nopi }                                        # 0030
  { nopi | nopi | nopi | sw t0, 0(rx) }                                # 0031
  { saddi rx, rx, 4 | mov t0, 128 | nopi | nopi }                      # 0032
  { nopi | nopi | nopi | nopi }                                        # 0033
  { nopi | nopi | nopi | sw t0, 0(rx) }                                # 0034
  { saddi rx, rx, 4 | mov t0, 16 | nopi | nopi }                       # 0035
  { nopi | nopi | nopi | nopi }                                        # 0036
  { nopi | nopi | nopi | sw t0, 0(rx) }                                # 0037
  { saddi rx, rx, 4 | nopi | nopi | nopi }                             # 0038
  { nopi | nopi | nopi | nopi }                                        # 0039
  { saddi rx, rx, -4 | nopi | nopi | nopi }                            # 0040
  { nopi | nopi | nopi | nopi }                                        # 0041
  { nopi | nopi | nopi | lw t0, 0(rx) }                                # 0042
  { saddi rx, rx, -4 | nopi | nopi | nopi }                            # 0043
  { nopi | nopi | nopi | sw t0, 224(rz) }                              # 0044
  { nopi | nopi | nopi | lw t0, 0(rx) }                                # 0045
  { saddi rx, rx, -4 | nopi | nopi | nopi }                            # 0046
  { nopi | nopi | nopi | sw t0, 220(rz) }                              # 0047
  { nopi | nopi | nopi | lw t0, 0(rx) }                                # 0048
  { saddi rx, rx, -4 | nopi | nopi | nopi }                            # 0049
  { nopi | nopi | nopi | sw t0, 216(rz) }                              # 0050
  { nopi | nopi | nopi | lw t0, 0(rx) }                                # 0051
  { nopi | nopi | nopi | nopi }                                        # 0052
  { nopi | nopi | nopi | sw t0, 212(rz) }                              # 0053
  { nopi | nopi | nopi | sw rl, 0(rx) }                                # 0054
  { saddi rx, rx, 4 | nopi | jas rl, .L_func_cifrar_archivo | nopi }   # 0055
  { nopi | nopi | nopi | nopi }                                        # 0056
  { saddi rx, rx, -4 | nopi | nopi | nopi }                            # 0057
  { nopi | nopi | nopi | nopi }                                        # 0058
  { nopi | nopi | nopi | lw rl, 0(rx) }                                # 0059
  { nopi | nopi | nopi | lw t0, 208(rz) }                              # 0060
  { nopi | nopi | nopi | nopi }                                        # 0061
  { nopi | nopi | nopi | sw t0, 372(rz) }                              # 0062
  { nopi | nopi | nopi | lw t0, 4(rz) }                                # 0063  linea 65
  { nopi | nopi | nopi | nopi }                                        # 0064
  { nopi | nopi | nopi | sw t0, 0(rx) }                                # 0065
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 8(rz) }                     # 0066
  { nopi | nopi | nopi | nopi }                                        # 0067
  { nopi | nopi | nopi | sw t0, 0(rx) }                                # 0068
  { saddi rx, rx, 4 | mov t0, 128 | nopi | nopi }                      # 0069
  { nopi | nopi | nopi | nopi }                                        # 0070
  { nopi | nopi | nopi | sw t0, 0(rx) }                                # 0071
  { saddi rx, rx, 4 | mov t0, 16 | nopi | nopi }                       # 0072
  { nopi | nopi | nopi | nopi }                                        # 0073
  { nopi | nopi | nopi | sw t0, 0(rx) }                                # 0074
  { saddi rx, rx, 4 | nopi | nopi | nopi }                             # 0075
  { nopi | nopi | nopi | nopi }                                        # 0076
  { saddi rx, rx, -4 | nopi | nopi | nopi }                            # 0077
  { nopi | nopi | nopi | nopi }                                        # 0078
  { nopi | nopi | nopi | lw t0, 0(rx) }                                # 0079
  { saddi rx, rx, -4 | nopi | nopi | nopi }                            # 0080
  { nopi | nopi | nopi | sw t0, 308(rz) }                              # 0081
  { nopi | nopi | nopi | lw t0, 0(rx) }                                # 0082
  { saddi rx, rx, -4 | nopi | nopi | nopi }                            # 0083
  { nopi | nopi | nopi | sw t0, 304(rz) }                              # 0084
  { nopi | nopi | nopi | lw t0, 0(rx) }                                # 0085
  { saddi rx, rx, -4 | nopi | nopi | nopi }                            # 0086
  { nopi | nopi | nopi | sw t0, 300(rz) }                              # 0087
  { nopi | nopi | nopi | lw t0, 0(rx) }                                # 0088
  { nopi | nopi | nopi | nopi }                                        # 0089
  { nopi | nopi | nopi | sw t0, 296(rz) }                              # 0090
  { nopi | nopi | nopi | sw rl, 0(rx) }                                # 0091
  { saddi rx, rx, 4 | nopi | jas rl, .L_func_descifrar_archivo | nopi } # 0092
  { nopi | nopi | nopi | nopi }                                        # 0093
  { saddi rx, rx, -4 | nopi | nopi | nopi }                            # 0094
  { nopi | nopi | nopi | nopi }                                        # 0095
  { nopi | nopi | nopi | lw rl, 0(rx) }                                # 0096
  { nopi | nopi | nopi | lw t0, 292(rz) }                              # 0097
  { nopi | nopi | nopi | nopi }                                        # 0098
  { nopi | nopi | nopi | sw t0, 376(rz) }                              # 0099
  { nopi | nopi | nopi | nopi }                                        # 0100
```

```
  { nopi | logout | nopi | nopi }                                  # 0101  linea 66
  { nopi | nopi | jef fin_main | nopi }                            # 0102  linea 67
  { nopi | nopi | nopi | nopi }                                    # 0103
.L_func_bloques_de:                                                # bundle 104
  { nopi | nopi | nopi | sw rz, 16(rz) }                           # 0104  linea 5
  { nopi | nopi | nopi | nopi }                                    # 0105
.L_body_bloques_de:                                                # bundle 106
  { mov r11, 1 | mov r7, 0 | nopi | lw t0, 20(rz) }                # 0106  linea 6
  { nopi | nopi | nopi | lw t1, 28(rz) }                           # 0107
  { nopi | nopi | nopi | lw t2, 24(rz) }                           # 0108
  { add r8, r5, r0 | nopi | nopi | nopi }                          # 0109
  { add r9, r6, r0 | nopi | nopi | nopi }                          # 0110
  { nopi | nopi | nopi | nopi }                                    # 0111
__ccom_arith_0_mul:                                                # bundle 112
  { nopi | nopi | beq r9, r0, __ccom_arith_2_end | nopi }           # 0112
  { nopi | nopi | nopi | nopi }                                    # 0113
  { and r10, r9, r11 | nopi | nopi | nopi }                        # 0114
  { nopi | nopi | nopi | nopi }                                    # 0115
  { nopi | nopi | beq r10, r0, __ccom_arith_1_skip | nopi }         # 0116
  { nopi | nopi | nopi | nopi }                                    # 0117
  { add r7, r7, r8 | nopi | nopi | nopi }                          # 0118
  { nopi | nopi | nopi | nopi }                                    # 0119
__ccom_arith_1_skip:                                               # bundle 120
  { sll r8, r8, r11 | srl r9, r9, r11 | jef __ccom_arith_0_mul | nopi }  # 0120
  { nopi | nopi | nopi | nopi }                                    # 0121
__ccom_arith_2_end:                                                # bundle 122
  { sub t0, t0, t3 | nopi | nopi | nopi }                          # 0122
  { nopi | nopi | nopi | nopi }                                    # 0123
  { nopi | nopi | nopi | sw t0, 32(rz) }                           # 0124
  { nopi | nopi | nopi | lw t0, 32(rz) }                           # 0125  linea 7
  { nopi | nopi | nopi | lw t1, 24(rz) }                           # 0126
  { nopi | nopi | nopi | nopi }                                    # 0127
  { slt t0, t0, t1 | nopi | nopi | nopi }                          # 0128
  { nopi | nopi | nopi | nopi }                                    # 0129
  { nopi | nopi | beq t0, rz, .L_if_next_0 | nopi }                # 0130
  { nopi | nopi | nopi | nopi }                                    # 0131
  { nopi | nopi | nopi | lw t0, 32(rz) }                           # 0132  linea 8
  { nopi | nopi | nopi | nopi }                                    # 0133
  { nopi | nopi | jef .L_if_end_1 | sw t0, 16(rz) }                # 0134  linea 7  slots 0=0,1=0,2=7,3=8
  { nopi | nopi | nopi | nopi }                                    # 0135
.L_if_next_0:                                                      # bundle 136
  { nopi | nopi | nopi | lw t0, 24(rz) }                           # 0136  linea 10
  { nopi | nopi | nopi | nopi }                                    # 0137
  { nopi | nopi | nopi | sw t0, 16(rz) }                           # 0138
  { nopi | nopi | nopi | nopi }                                    # 0139
.L_if_end_1:                                                       # bundle 140
  { nopi | nopi | jefR rz, rl | nopi }                             # 0140  linea 12
  { nopi | nopi | nopi | nopi }                                    # 0141
.L_func_direccion:                                                 # bundle 142
  { nopi | nopi | nopi | sw rz, 88(rz) }                           # 0142  linea 15
  { nopi | nopi | nopi | nopi }                                    # 0143
.L_body_direccion:                                                 # bundle 144
  { mov r11, 1 | mov r7, 0 | nopi | lw t0, 92(rz) }                # 0144  linea 16
  { nopi | nopi | nopi | lw t1, 100(rz) }                          # 0145
  { nopi | nopi | nopi | lw t2, 96(rz) }                           # 0146
  { add r8, r5, r0 | nopi | nopi | nopi }                          # 0147
  { add r9, r6, r0 | nopi | nopi | nopi }                          # 0148
  { nopi | nopi | nopi | nopi }                                    # 0149
__ccom_arith_3_mul:                                                # bundle 150
  { nopi | nopi | beq r9, r0, __ccom_arith_5_end | nopi }           # 0150
  { nopi | nopi | nopi | nopi }                                    # 0151
  { and r10, r9, r11 | nopi | nopi | nopi }                        # 0152
  { nopi | nopi | nopi | nopi }                                    # 0153
  { nopi | nopi | beq r10, r0, __ccom_arith_4_skip | nopi }         # 0154
  { nopi | nopi | nopi | nopi }                                    # 0155
  { add r7, r7, r8 | nopi | nopi | nopi }                          # 0156
  { nopi | nopi | nopi | nopi }                                    # 0157
__ccom_arith_4_skip:                                               # bundle 158
  { sll r8, r8, r11 | srl r9, r9, r11 | jef __ccom_arith_3_mul | nopi }  # 0158
  { nopi | nopi | nopi | nopi }                                    # 0159
__ccom_arith_5_end:                                                # bundle 160
  { mov t1, 8 | mov r11, 1 | nopi | nopi }                         # 0160
  { add r8, r7, r0 | mov r6, 0 | nopi | nopi }                     # 0161
  { add r9, r5, r0 | nopi | nopi | nopi }                          # 0162
  { nopi | nopi | nopi | nopi }                                    # 0163
__ccom_arith_6_mul:                                                # bundle 164
  { nopi | nopi | beq r9, r0, __ccom_arith_8_end | nopi }           # 0164
  { nopi | nopi | nopi | nopi }                                    # 0165
  { and r10, r9, r11 | nopi | nopi | nopi }                        # 0166
  { nopi | nopi | nopi | nopi }                                    # 0167
  { nopi | nopi | beq r10, r0, __ccom_arith_7_skip | nopi }         # 0168
  { nopi | nopi | nopi | nopi }                                    # 0169
  { add r6, r6, r8 | nopi | nopi | nopi }                          # 0170
  { nopi | nopi | nopi | nopi }                                    # 0171
__ccom_arith_7_skip:                                               # bundle 172
  { sll r8, r8, r11 | srl r9, r9, r11 | jef __ccom_arith_6_mul | nopi }  # 0172
  { nopi | nopi | nopi | nopi }                                    # 0173
__ccom_arith_8_end:                                                # bundle 174
  { add t0, t0, t2 | nopi | nopi | nopi }                          # 0174
  { nopi | nopi | nopi | nopi }                                    # 0175
  { nopi | nopi | jefR rz, rl | sw t0, 88(rz) }                    # 0176  linea 17  slots 0=0,1=0,2=17,3=16
  { nopi | nopi | nopi | nopi }                                    # 0177
.L_func_preparar_boveda:                                           # bundle 178
.L_body_preparar_boveda:                                           # bundle 178
  { nopi | nopi | nopi | nopi }                                    # 0178
  { mov t0, 583 | mov t1, 15 | nopi | nopi }                       # 0179  linea 21
  { nopi | nopi | nopi | nopi }                                    # 0180
  { sll t0, t0, t1 | nopi | nopi | nopi }                          # 0181
  { nopi | nopi | nopi | nopi }                                    # 0182
  { saddi t0, t0, 11213 | nopi | nopi | nopi }                     # 0183
  { nopi | nopi | nopi | nopi }                                    # 0184
  { saddi t6, t0, 0 | nopi | nopi | nopi }                         # 0185
  { nopi | nopi | nopi | nopi }                                    # 0186
  { nopi | nopi | nopi | nopi }                                    # 0187
  { nopi | kv_write t6, 0, 0 | nopi | nopi }                       # 0188
  { nopi | nopi | nopi | nopi }                                    # 0189
  { mov t0, 60584 | mov t1, 15 | nopi | nopi }                     # 0190  linea 22
  { nopi | nopi | nopi | nopi }                                    # 0191
  { sll t0, t0, t1 | nopi | nopi | nopi }                          # 0192
  { nopi | nopi | nopi | nopi }                                    # 0193
  { saddi t0, t0, 12817 | nopi | nopi | nopi }                     # 0194
  { nopi | nopi | nopi | nopi }                                    # 0195
  { sub t0, rz, t0 | nopi | nopi | nopi }                          # 0196
  { nopi | nopi | nopi | nopi }                                    # 0197
  { saddi t6, t0, 0 | nopi | nopi | nopi }                         # 0198
```

```
  { nopi | nopi | nopi | nopi }                                  # 0199
  { nopi | nopi | nopi | nopi }                                  # 0200
  { nopi | kv_write t6, 0, 1 | nopi | nopi }                     # 0201
  { nopi | nopi | nopi | nopi }                                  # 0202
  { mov t0, 9903 | mov t1, 15 | nopi | nopi }                    # 0203  linea 23
  { nopi | nopi | nopi | nopi }                                  # 0204
  { sll t0, t0, t1 | nopi | nopi | nopi }                        # 0205
  { nopi | nopi | nopi | nopi }                                  # 0206
  { saddi t0, t0, 7135 | nopi | nopi | nopi }                    # 0207
  { nopi | nopi | nopi | nopi }                                  # 0208
  { saddi t6, t0, 0 | nopi | nopi | nopi }                       # 0209
  { nopi | nopi | nopi | nopi }                                  # 0210
  { nopi | nopi | nopi | nopi }                                  # 0211
  { nopi | kv_write t6, 0, 2 | nopi | nopi }                     # 0212
  { nopi | nopi | nopi | nopi }                                  # 0213
  { mov t0, 18641 | mov t1, 15 | nopi | nopi }                   # 0214  linea 24
  { nopi | nopi | nopi | nopi }                                  # 0215
  { sll t0, t0, t1 | nopi | nopi | nopi }                        # 0216
  { nopi | nopi | nopi | nopi }                                  # 0217
  { saddi t0, t0, 11488 | nopi | nopi | nopi }                   # 0218
  { nopi | nopi | nopi | nopi }                                  # 0219
  { saddi t6, t0, 0 | nopi | nopi | nopi }                       # 0220
  { nopi | nopi | nopi | nopi }                                  # 0221
  { nopi | nopi | nopi | nopi }                                  # 0222
  { nopi | kv_write t6, 0, 3 | nopi | nopi }                     # 0223
  { nopi | nopi | nopi | nopi }                                  # 0224
  { mov t0, 8738 | mov t1, 15 | nopi | nopi }                    # 0225  linea 25
  { nopi | nopi | nopi | nopi }                                  # 0226
  { sll t0, t0, t1 | nopi | nopi | nopi }                        # 0227
  { nopi | nopi | nopi | nopi }                                  # 0228
  { saddi t0, t0, 4369 | nopi | nopi | nopi }                    # 0229
  { nopi | nopi | nopi | nopi }                                  # 0230
  { saddi t6, t0, 0 | nopi | nopi | nopi }                       # 0231
  { nopi | nopi | nopi | nopi }                                  # 0232
  { nopi | nopi | nopi | nopi }                                  # 0233
  { nopi | kv_write t6, 1, 0 | nopi | nopi }                     # 0234
  { nopi | nopi | nopi | nopi }                                  # 0235
  { mov t0, 17476 | mov t1, 15 | nopi | nopi }                   # 0236  linea 26
  { nopi | nopi | nopi | nopi }                                  # 0237
  { sll t0, t0, t1 | nopi | nopi | nopi }                        # 0238
  { nopi | nopi | nopi | nopi }                                  # 0239
  { saddi t0, t0, 8738 | nopi | nopi | nopi }                    # 0240
  { nopi | nopi | nopi | nopi }                                  # 0241
  { saddi t6, t0, 0 | nopi | nopi | nopi }                       # 0242
  { nopi | nopi | nopi | nopi }                                  # 0243
  { nopi | nopi | nopi | nopi }                                  # 0244
  { nopi | kv_write t6, 1, 1 | nopi | nopi }                     # 0245
  { nopi | nopi | nopi | nopi }                                  # 0246
  { mov t0, 26214 | mov t1, 15 | nopi | nopi }                   # 0247  linea 27
  { nopi | nopi | nopi | nopi }                                  # 0248
  { sll t0, t0, t1 | nopi | nopi | nopi }                        # 0249
  { nopi | nopi | nopi | nopi }                                  # 0250
  { saddi t0, t0, 13107 | nopi | nopi | nopi }                   # 0251
  { nopi | nopi | nopi | nopi }                                  # 0252
  { saddi t6, t0, 0 | nopi | nopi | nopi }                       # 0253
  { nopi | nopi | nopi | nopi }                                  # 0254
  { nopi | nopi | nopi | nopi }                                  # 0255
  { nopi | kv_write t6, 1, 2 | nopi | nopi }                     # 0256
  { nopi | nopi | nopi | nopi }                                  # 0257
  { mov t0, 34952 | mov t1, 15 | nopi | nopi }                   # 0258  linea 28
  { nopi | nopi | nopi | nopi }                                  # 0259
  { sll t0, t0, t1 | nopi | nopi | nopi }                        # 0260
  { nopi | nopi | nopi | nopi }                                  # 0261
  { saddi t0, t0, 17476 | nopi | nopi | nopi }                   # 0262
  { nopi | nopi | nopi | nopi }                                  # 0263
  { saddi t6, t0, 0 | nopi | nopi | nopi }                       # 0264
  { nopi | nopi | nopi | nopi }                                  # 0265
  { nopi | nopi | nopi | nopi }                                  # 0266
  { nopi | kv_write t6, 1, 3 | nopi | nopi }                     # 0267
  { nopi | kv_lock | nopi | nopi }                               # 0268  linea 29
  { nopi | nopi | jefR rz, rl | nopi }                           # 0269  linea 30
  { nopi | nopi | nopi | nopi }                                  # 0270
.L_func_cifrar_archivo:                                          # bundle 271
  { nopi | nopi | nopi | sw rz, 208(rz) }                        # 0271  linea 33
  { nopi | nopi | nopi | nopi }                                  # 0272
.L_body_cifrar_archivo:                                          # bundle 273
  { nopi | nopi | nopi | lw t0, 220(rz) }                        # 0273  linea 34
  { nopi | nopi | nopi | lw t1, 224(rz) }                        # 0274
  { nopi | nopi | nopi | nopi }                                  # 0275
  { add t0, t0, t1 | nopi | nopi | nopi }                        # 0276
  { mov t1, 1 | nopi | nopi | nopi }                             # 0277
  { nopi | nopi | nopi | nopi }                                  # 0278
  { sub t0, t0, t1 | nopi | nopi | nopi }                        # 0279
  { nopi | nopi | nopi | lw t1, 224(rz) }                        # 0280
  { nopi | nopi | nopi | nopi }                                  # 0281
  { nopi | nopi | beq r5, r0, .L_arith_error | nopi }             # 0282
  { nopi | nopi | nopi | nopi }                                  # 0283
  { add r7, r4, r0 | add r8, r5, r0 | nopi | nopi }              # 0284
  { nopi | nopi | nopi | nopi }                                  # 0285
  { nopi | nopi | bge r7, r0, __ccom_arith_9_apos | nopi }        # 0286
  { nopi | nopi | nopi | nopi }                                  # 0287
  { sub r7, r0, r7 | nopi | nopi | nopi }                        # 0288
  { nopi | nopi | nopi | nopi }                                  # 0289
__ccom_arith_9_apos:                                             # bundle 290
  { nopi | nopi | bge r8, r0, __ccom_arith_10_bpos | nopi }      # 0290
  { nopi | nopi | nopi | nopi }                                  # 0291
  { sub r8, r0, r8 | nopi | nopi | nopi }                        # 0292
  { nopi | nopi | nopi | nopi }                                  # 0293
__ccom_arith_10_bpos:                                            # bundle 294
  { mov r11, 1 | mov r10, 31 | nopi | nopi }                     # 0294
  { mov r9, 0 | mov r6, 0 | nopi | nopi }                        # 0295
  { sll r14, r11, r10 | nopi | nopi | nopi }                     # 0296
  { mov r10, 32 | nopi | nopi | nopi }                           # 0297
  { nopi | nopi | nopi | nopi }                                  # 0298
__ccom_arith_11_div:                                             # bundle 299
  { slt r12, r7, r0 | sll r9, r9, r11 | nopi | nopi }            # 0299
  { sll r7, r7, r11 | sll r6, r6, r11 | nopi | nopi }            # 0300
  { or r9, r9, r12 | nopi | nopi | nopi }                        # 0301
  { xor r12, r8, r14 | nopi | nopi | nopi }                      # 0302
  { xor r13, r9, r14 | nopi | nopi | nopi }                      # 0303
  { nopi | nopi | nopi | nopi }                                  # 0304
  { nopi | nopi | blt r13, r12, __ccom_arith_12_skip | nopi }    # 0305
  { nopi | nopi | nopi | nopi }                                  # 0306
  { sub r9, r9, r8 | saddi r6, r6, 1 | nopi | nopi }             # 0307
  { nopi | nopi | nopi | nopi }                                  # 0308
```

```
__ccom_arith_12_skip:                                   # bundle 309
  { saddi r10, r10, -1 | nopi | nopi | nopi }           # 0309
  { nopi | nopi | nopi | nopi }                         # 0310
  { nopi | nopi | bne r10, r0, __ccom_arith_11_div | nopi }  # 0311
  { nopi | nopi | nopi | nopi }                         # 0312
  { xor r13, r4, r5 | nopi | nopi | nopi }              # 0313
  { nopi | nopi | nopi | nopi }                         # 0314
  { nopi | nopi | bge r13, r0, __ccom_arith_13_end | nopi }  # 0315
  { nopi | nopi | nopi | nopi }                         # 0316
  { sub r6, r0, r6 | nopi | nopi | nopi }               # 0317
  { nopi | nopi | nopi | nopi }                         # 0318
__ccom_arith_13_end:                                    # bundle 319
  { mov t0, 0 | nopi | nopi | sw t2, 208(rz) }          # 0319  linea 35  slots 0=35,1=0,2=0,3=34
  { nopi | nopi | nopi | lw t1, 208(rz) }               # 0320
  { nopi | nopi | nopi | sw t0, 228(rz) }               # 0321
  { nopi | nopi | nopi | sw t1, 232(rz) }               # 0322
  { nopi | nopi | nopi | nopi }                         # 0323
.L_for_start_2:                                         # bundle 324
  { nopi | nopi | nopi | lw t0, 228(rz) }               # 0324
  { nopi | nopi | nopi | lw t1, 232(rz) }               # 0325
  { nopi | nopi | nopi | nopi }                         # 0326
  { nopi | nopi | bge t0, t1, .L_for_end_4 | nopi }     # 0327
  { nopi | nopi | nopi | nopi }                         # 0328
  { nopi | nopi | nopi | lw t0, 220(rz) }               # 0329  linea 36
  { nopi | nopi | nopi | nopi }                         # 0330
  { nopi | nopi | nopi | sw t0, 0(rx) }                 # 0331
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 224(rz) }    # 0332
  { nopi | nopi | nopi | nopi }                         # 0333
  { nopi | nopi | nopi | sw t0, 0(rx) }                 # 0334
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 228(rz) }    # 0335
  { nopi | nopi | nopi | nopi }                         # 0336
  { nopi | nopi | nopi | sw t0, 0(rx) }                 # 0337
  { saddi rx, rx, 4 | nopi | nopi | nopi }              # 0338
  { nopi | nopi | nopi | nopi }                         # 0339
  { saddi rx, rx, -4 | nopi | nopi | nopi }             # 0340
  { nopi | nopi | nopi | nopi }                         # 0341
  { nopi | nopi | nopi | lw t0, 0(rx) }                 # 0342
  { saddi rx, rx, -4 | nopi | nopi | nopi }             # 0343
  { nopi | nopi | nopi | sw t0, 28(rz) }                # 0344
  { nopi | nopi | nopi | lw t0, 0(rx) }                 # 0345
  { saddi rx, rx, -4 | nopi | nopi | nopi }             # 0346
  { nopi | nopi | nopi | sw t0, 24(rz) }                # 0347
  { nopi | nopi | nopi | lw t0, 0(rx) }                 # 0348
  { nopi | nopi | nopi | nopi }                         # 0349
  { nopi | nopi | nopi | sw t0, 20(rz) }                # 0350
  { nopi | nopi | nopi | sw rl, 0(rx) }                 # 0351
  { saddi rx, rx, 4 | nopi | jas rl, .L_func_bloques_de | nopi }  # 0352
  { nopi | nopi | nopi | nopi }                         # 0353
  { saddi rx, rx, -4 | mov t1, 2 | nopi | nopi }        # 0354  slots 0=36,1=37,2=0,3=0
  { nopi | nopi | nopi | nopi }                         # 0355
  { nopi | nopi | nopi | lw rl, 0(rx) }                 # 0356
  { nopi | nopi | nopi | lw t0, 16(rz) }                # 0357
  { nopi | nopi | nopi | nopi }                         # 0358
  { nopi | nopi | nopi | sw t0, 236(rz) }               # 0359
  { nopi | nopi | beq r5, r0, .L_arith_error | lw t0, 228(rz) }  # 0360  linea 37
  { nopi | nopi | nopi | nopi }                         # 0361
  { add r7, r4, r0 | add r8, r5, r0 | nopi | nopi }     # 0362
  { nopi | nopi | nopi | nopi }                         # 0363
  { nopi | nopi | bge r7, r0, __ccom_arith_14_apos | nopi }  # 0364
  { nopi | nopi | nopi | nopi }                         # 0365
  { sub r7, r0, r7 | nopi | nopi | nopi }               # 0366
  { nopi | nopi | nopi | nopi }                         # 0367
__ccom_arith_14_apos:                                   # bundle 368
  { nopi | nopi | bge r8, r0, __ccom_arith_15_bpos | nopi }  # 0368
  { nopi | nopi | nopi | nopi }                         # 0369
  { sub r8, r0, r8 | nopi | nopi | nopi }               # 0370
  { nopi | nopi | nopi | nopi }                         # 0371
__ccom_arith_15_bpos:                                   # bundle 372
  { mov r11, 1 | mov r10, 31 | nopi | nopi }            # 0372
  { mov r9, 0 | mov r6, 0 | nopi | nopi }               # 0373
  { sll r14, r11, r10 | nopi | nopi | nopi }            # 0374
  { mov r10, 32 | nopi | nopi | nopi }                  # 0375
  { nopi | nopi | nopi | nopi }                         # 0376
__ccom_arith_16_div:                                    # bundle 377
  { slt r12, r7, r0 | sll r9, r9, r11 | nopi | nopi }   # 0377
  { sll r7, r7, r11 | sll r6, r6, r11 | nopi | nopi }   # 0378
  { or r9, r9, r12 | nopi | nopi | nopi }               # 0379
  { xor r12, r8, r14 | nopi | nopi | nopi }             # 0380
  { xor r13, r9, r14 | nopi | nopi | nopi }             # 0381
  { nopi | nopi | nopi | nopi }                         # 0382
  { nopi | nopi | blt r13, r12, __ccom_arith_17_skip | nopi }  # 0383
  { nopi | nopi | nopi | nopi }                         # 0384
  { sub r9, r9, r8 | saddi r6, r6, 1 | nopi | nopi }    # 0385
  { nopi | nopi | nopi | nopi }                         # 0386
__ccom_arith_17_skip:                                   # bundle 387
  { saddi r10, r10, -1 | nopi | nopi | nopi }           # 0387
  { nopi | nopi | nopi | nopi }                         # 0388
  { nopi | nopi | bne r10, r0, __ccom_arith_16_div | nopi }  # 0389
  { nopi | nopi | nopi | nopi }                         # 0390
  { add r6, r9, r0 | nopi | bge r4, r0, __ccom_arith_18_end | nopi }  # 0391
  { nopi | nopi | nopi | nopi }                         # 0392
  { sub r6, r0, r6 | nopi | nopi | nopi }               # 0393
  { nopi | nopi | nopi | nopi }                         # 0394
__ccom_arith_18_end:                                    # bundle 395
  { mov t0, 0 | nopi | nopi | nopi }                    # 0395
  { nopi | nopi | nopi | nopi }                         # 0396
  { seq t2, t2, t0 | nopi | nopi | nopi }               # 0397
  { nopi | nopi | nopi | nopi }                         # 0398
  { nopi | nopi | beq t2, rz, .L_if_next_5 | nopi }     # 0399
  { nopi | nopi | nopi | nopi }                         # 0400
  { nopi | nopi | nopi | nopi }                         # 0401
  { nopi | nopi | nopi | lw t0, 212(rz) }               # 0402  linea 38
  { nopi | nopi | nopi | nopi }                         # 0403
  { nopi | nopi | nopi | sw t0, 0(rx) }                 # 0404
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 224(rz) }    # 0405
  { nopi | nopi | nopi | nopi }                         # 0406
  { nopi | nopi | nopi | sw t0, 0(rx) }                 # 0407
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 228(rz) }    # 0408
  { nopi | nopi | nopi | nopi }                         # 0409
  { nopi | nopi | nopi | sw t0, 0(rx) }                 # 0410
  { saddi rx, rx, 4 | nopi | nopi | nopi }              # 0411
  { nopi | nopi | nopi | nopi }                         # 0412
  { saddi rx, rx, -4 | nopi | nopi | nopi }             # 0413
  { nopi | nopi | nopi | nopi }                         # 0414
  { nopi | nopi | nopi | lw t0, 0(rx) }                 # 0415
```

```
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0416
  { nopi | nopi | nopi | sw t0, 100(rz) }                                    # 0417
  { nopi | nopi | nopi | lw t0, 0(rx) }                                      # 0418
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0419
  { nopi | nopi | nopi | sw t0, 96(rz) }                                     # 0420
  { nopi | nopi | nopi | lw t0, 0(rx) }                                      # 0421
  { nopi | nopi | nopi | nopi }                                              # 0422
  { nopi | nopi | nopi | sw t0, 92(rz) }                                     # 0423
  { nopi | nopi | nopi | sw rl, 0(rx) }                                      # 0424
  { saddi rx, rx, 4 | nopi | jas rl, .L_func_direccion | nopi }              # 0425
  { nopi | nopi | nopi | nopi }                                              # 0426
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0427
  { nopi | nopi | nopi | nopi }                                              # 0428
  { nopi | nopi | nopi | lw rl, 0(rx) }                                      # 0429
  { nopi | nopi | nopi | lw t0, 88(rz) }                                     # 0430
  { nopi | nopi | nopi | nopi }                                              # 0431
  { saddi t2, t0, 0 | nopi | nopi | nopi }                                   # 0432
  { nopi | nopi | nopi | nopi }                                              # 0433
  { nopi | nopi | nopi | nopi }                                              # 0434
  { nopi | nopi | nopi | nopi }                                              # 0435
  { nopi | nopi | nopi | sw t2, 240(rz) }                                    # 0436
  { nopi | nopi | nopi | lw t0, 216(rz) }                                    # 0437
  { nopi | nopi | nopi | nopi }                                              # 0438
  { nopi | nopi | nopi | sw t0, 0(rx) }                                      # 0439
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 224(rz) }                         # 0440
  { nopi | nopi | nopi | nopi }                                              # 0441
  { nopi | nopi | nopi | sw t0, 0(rx) }                                      # 0442
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 228(rz) }                         # 0443
  { nopi | nopi | nopi | nopi }                                              # 0444
  { nopi | nopi | nopi | sw t0, 0(rx) }                                      # 0445
  { saddi rx, rx, 4 | nopi | nopi | nopi }                                   # 0446
  { nopi | nopi | nopi | nopi }                                              # 0447
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0448
  { nopi | nopi | nopi | nopi }                                              # 0449
  { nopi | nopi | nopi | lw t0, 0(rx) }                                      # 0450
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0451
  { nopi | nopi | nopi | sw t0, 100(rz) }                                    # 0452
  { nopi | nopi | nopi | lw t0, 0(rx) }                                      # 0453
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0454
  { nopi | nopi | nopi | sw t0, 96(rz) }                                     # 0455
  { nopi | nopi | nopi | lw t0, 0(rx) }                                      # 0456
  { nopi | nopi | nopi | nopi }                                              # 0457
  { nopi | nopi | nopi | sw t0, 92(rz) }                                     # 0458
  { nopi | nopi | nopi | sw rl, 0(rx) }                                      # 0459
  { saddi rx, rx, 4 | nopi | jas rl, .L_func_direccion | nopi }              # 0460
  { nopi | nopi | nopi | nopi }                                              # 0461
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0462
  { nopi | nopi | nopi | nopi }                                              # 0463
  { nopi | nopi | nopi | lw rl, 0(rx) }                                      # 0464
  { nopi | nopi | nopi | lw t0, 88(rz) }                                     # 0465
  { nopi | nopi | nopi | lw t2, 240(rz) }                                    # 0466
  { saddi t3, t0, 0 | nopi | nopi | nopi }                                   # 0467
  { nopi | nopi | nopi | nopi }                                              # 0468
  { nopi | nopi | nopi | nopi }                                              # 0469
  { nopi | nopi | nopi | nopi }                                              # 0470
  { nopi | nopi | nopi | sw t2, 240(rz) }                                    # 0471
  { nopi | nopi | nopi | sw t3, 244(rz) }                                    # 0472
  { nopi | nopi | nopi | lw t0, 236(rz) }                                    # 0473
  { nopi | nopi | nopi | lw t2, 240(rz) }                                    # 0474
  { saddi t5, t0, 0 | nopi | nopi | lw t3, 244(rz) }                         # 0475
  { nopi | nopi | nopi | nopi }                                              # 0476
  { nopi | nopi | nopi | nopi }                                              # 0477
  { mov t4, 0 | nopi | nopi | nopi }                                         # 0478
  { nopi | nopi | nopi | nopi }                                              # 0479
  { nopi | nopi | bge rz, t5, .L_crypto_fin_cifra_1 | nopi }                 # 0480
.L_crypto_cifra_1:                                                           # bundle 481
  { nopi | nopi | nopi | lw t0, 0(t2) }                                      # 0481
  { nopi | nopi | nopi | lw t1, 4(t2) }                                      # 0482
  { saddi t2, t2, 8 | nopi | nopi | nopi }                                   # 0483
  { nopi | feis t0, t0, t1, 0, 0 | nopi | nopi }                             # 0484
  { saddi t4, t4, 1 | nopi | nopi | nopi }                                   # 0485
  { nopi | feis t1, t1, t0, 0, 1 | nopi | nopi }                             # 0486
  { nopi | nopi | nopi | nopi }                                              # 0487
  { nopi | feis t0, t0, t1, 0, 2 | nopi | nopi }                             # 0488
  { nopi | nopi | nopi | nopi }                                              # 0489
  { nopi | feis t1, t1, t0, 0, 3 | nopi | nopi }                             # 0490
  { nopi | nopi | nopi | sw t0, 0(t3) }                                      # 0491
  { saddi t3, t3, 8 | nopi | blt t4, t5, .L_crypto_cifra_1 | sw t1, 4(t3) }  # 0492
.L_crypto_fin_cifra_1:                                                       # bundle 493
  { nopi | nopi | nopi | nopi }                                              # 0493
  { nopi | nopi | jef .L_if_end_6 | nopi }                                   # 0494  linea 37
  { nopi | nopi | nopi | nopi }                                              # 0495
.L_if_next_5:                                                                # bundle 496
  { nopi | nopi | nopi | nopi }                                              # 0496
  { nopi | nopi | nopi | lw t0, 212(rz) }                                    # 0497  linea 40
  { nopi | nopi | nopi | nopi }                                              # 0498
  { nopi | nopi | nopi | sw t0, 0(rx) }                                      # 0499
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 224(rz) }                         # 0500
  { nopi | nopi | nopi | nopi }                                              # 0501
  { nopi | nopi | nopi | sw t0, 0(rx) }                                      # 0502
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 228(rz) }                         # 0503
  { nopi | nopi | nopi | nopi }                                              # 0504
  { nopi | nopi | nopi | sw t0, 0(rx) }                                      # 0505
  { saddi rx, rx, 4 | nopi | nopi | nopi }                                   # 0506
  { nopi | nopi | nopi | nopi }                                              # 0507
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0508
  { nopi | nopi | nopi | nopi }                                              # 0509
  { nopi | nopi | nopi | lw t0, 0(rx) }                                      # 0510
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0511
  { nopi | nopi | nopi | sw t0, 100(rz) }                                    # 0512
  { nopi | nopi | nopi | lw t0, 0(rx) }                                      # 0513
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0514
  { nopi | nopi | nopi | sw t0, 96(rz) }                                     # 0515
  { nopi | nopi | nopi | lw t0, 0(rx) }                                      # 0516
  { nopi | nopi | nopi | nopi }                                              # 0517
  { nopi | nopi | nopi | sw t0, 92(rz) }                                     # 0518
  { nopi | nopi | nopi | sw rl, 0(rx) }                                      # 0519
  { saddi rx, rx, 4 | nopi | jas rl, .L_func_direccion | nopi }              # 0520
  { nopi | nopi | nopi | nopi }                                              # 0521
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0522
  { nopi | nopi | nopi | nopi }                                              # 0523
  { nopi | nopi | nopi | lw rl, 0(rx) }                                      # 0524
  { nopi | nopi | nopi | lw t0, 88(rz) }                                     # 0525
  { nopi | nopi | nopi | nopi }                                              # 0526
  { saddi t2, t0, 0 | nopi | nopi | nopi }                                   # 0527
```

```
  { nopi | nopi | nopi | nopi }                                          # 0528
  { nopi | nopi | nopi | nopi }                                          # 0529
  { nopi | nopi | nopi | nopi }                                          # 0530
  { nopi | nopi | nopi | sw t2, 240(rz) }                                # 0531
  { nopi | nopi | nopi | lw t0, 216(rz) }                                # 0532
  { nopi | nopi | nopi | nopi }                                          # 0533
  { nopi | nopi | nopi | sw t0, 0(rx) }                                  # 0534
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 224(rz) }                     # 0535
  { nopi | nopi | nopi | nopi }                                          # 0536
  { nopi | nopi | nopi | sw t0, 0(rx) }                                  # 0537
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 228(rz) }                     # 0538
  { nopi | nopi | nopi | nopi }                                          # 0539
  { nopi | nopi | nopi | sw t0, 0(rx) }                                  # 0540
  { saddi rx, rx, 4 | nopi | nopi | nopi }                               # 0541
  { nopi | nopi | nopi | nopi }                                          # 0542
  { saddi rx, rx, -4 | nopi | nopi | nopi }                              # 0543
  { nopi | nopi | nopi | nopi }                                          # 0544
  { nopi | nopi | nopi | lw t0, 0(rx) }                                  # 0545
  { saddi rx, rx, -4 | nopi | nopi | nopi }                              # 0546
  { nopi | nopi | nopi | sw t0, 100(rz) }                                # 0547
  { nopi | nopi | nopi | lw t0, 0(rx) }                                  # 0548
  { saddi rx, rx, -4 | nopi | nopi | nopi }                              # 0549
  { nopi | nopi | nopi | sw t0, 96(rz) }                                 # 0550
  { nopi | nopi | nopi | lw t0, 0(rx) }                                  # 0551
  { nopi | nopi | nopi | nopi }                                          # 0552
  { nopi | nopi | nopi | sw t0, 92(rz) }                                 # 0553
  { nopi | nopi | nopi | sw rl, 0(rx) }                                  # 0554
  { saddi rx, rx, 4 | nopi | jas rl, .L_func_direccion | nopi }          # 0555
  { nopi | nopi | nopi | nopi }                                          # 0556
  { saddi rx, rx, -4 | nopi | nopi | nopi }                              # 0557
  { nopi | nopi | nopi | nopi }                                          # 0558
  { nopi | nopi | nopi | lw rl, 0(rx) }                                  # 0559
  { nopi | nopi | nopi | lw t0, 88(rz) }                                 # 0560
  { nopi | nopi | nopi | lw t2, 240(rz) }                                # 0561
  { saddi t3, t0, 0 | nopi | nopi | nopi }                               # 0562
  { nopi | nopi | nopi | nopi }                                          # 0563
  { nopi | nopi | nopi | nopi }                                          # 0564
  { nopi | nopi | nopi | nopi }                                          # 0565
  { nopi | nopi | nopi | sw t2, 240(rz) }                                # 0566
  { nopi | nopi | nopi | sw t3, 244(rz) }                                # 0567
  { nopi | nopi | nopi | lw t0, 236(rz) }                                # 0568
  { nopi | nopi | nopi | lw t2, 240(rz) }                                # 0569
  { saddi t5, t0, 0 | nopi | nopi | lw t3, 244(rz) }                     # 0570
  { nopi | nopi | nopi | nopi }                                          # 0571
  { nopi | nopi | nopi | nopi }                                          # 0572
  { mov t4, 0 | nopi | nopi | nopi }                                     # 0573
  { nopi | nopi | nopi | nopi }                                          # 0574
  { nopi | nopi | bge rz, t5, .L_crypto_fin_cifra_2 | nopi }             # 0575
.L_crypto_cifra_2:                                                       # bundle 576
  { nopi | nopi | nopi | lw t0, 0(t2) }                                  # 0576
  { nopi | nopi | nopi | lw t1, 4(t2) }                                  # 0577
  { saddi t2, t2, 8 | nopi | nopi | nopi }                               # 0578
  { nopi | feis t0, t0, t1, 1, 0 | nopi | nopi }                         # 0579
  { saddi t4, t4, 1 | nopi | nopi | nopi }                               # 0580
  { nopi | feis t1, t1, t0, 1, 1 | nopi | nopi }                         # 0581
  { nopi | nopi | nopi | nopi }                                          # 0582
  { nopi | feis t0, t0, t1, 1, 2 | nopi | nopi }                         # 0583
  { nopi | nopi | nopi | nopi }                                          # 0584
  { nopi | feis t1, t1, t0, 1, 3 | nopi | nopi }                         # 0585
  { nopi | nopi | nopi | sw t0, 0(t3) }                                  # 0586
  { saddi t3, t3, 8 | nopi | blt t4, t5, .L_crypto_cifra_2 | sw t1, 4(t3) }  # 0587
.L_crypto_fin_cifra_2:                                                   # bundle 588
  { nopi | nopi | nopi | nopi }                                          # 0588
.L_if_end_6:                                                             # bundle 589
.L_for_update_3:                                                         # bundle 589
  { nopi | nopi | nopi | lw t0, 228(rz) }                                # 0589  linea 35
  { nopi | nopi | nopi | nopi }                                          # 0590
  { saddi t0, t0, 1 | nopi | nopi | nopi }                               # 0591
  { nopi | nopi | nopi | nopi }                                          # 0592
  { nopi | nopi | jef .L_for_start_2 | sw t0, 228(rz) }                  # 0593
  { nopi | nopi | nopi | nopi }                                          # 0594
.L_for_end_4:                                                            # bundle 595
  { nopi | nopi | jefR rz, rl | nopi }                                   # 0595  linea 43
  { nopi | nopi | nopi | nopi }                                          # 0596
.L_func_descifrar_archivo:                                               # bundle 597
  { nopi | nopi | nopi | sw rz, 292(rz) }                                # 0597  linea 46
  { nopi | nopi | nopi | nopi }                                          # 0598
.L_body_descifrar_archivo:                                               # bundle 599
  { nopi | nopi | nopi | lw t0, 304(rz) }                                # 0599  linea 47
  { nopi | nopi | nopi | lw t1, 308(rz) }                                # 0600
  { nopi | nopi | nopi | nopi }                                          # 0601
  { add t0, t0, t1 | nopi | nopi | nopi }                                # 0602
  { mov t1, 1 | nopi | nopi | nopi }                                     # 0603
  { nopi | nopi | nopi | nopi }                                          # 0604
  { sub t0, t0, t1 | nopi | nopi | nopi }                                # 0605
  { nopi | nopi | nopi | lw t1, 308(rz) }                                # 0606
  { nopi | nopi | nopi | nopi }                                          # 0607
  { nopi | nopi | beq r5, r0, .L_arith_error | nopi }                    # 0608
  { nopi | nopi | nopi | nopi }                                          # 0609
  { add r7, r4, r0 | add r8, r5, r0 | nopi | nopi }                      # 0610
  { nopi | nopi | nopi | nopi }                                          # 0611
  { nopi | nopi | bge r7, r0, __ccom_arith_19_apos | nopi }               # 0612
  { nopi | nopi | nopi | nopi }                                          # 0613
  { sub r7, r0, r7 | nopi | nopi | nopi }                                # 0614
  { nopi | nopi | nopi | nopi }                                          # 0615
__ccom_arith_19_apos:                                                    # bundle 616
  { nopi | nopi | bge r8, r0, __ccom_arith_20_bpos | nopi }               # 0616
  { nopi | nopi | nopi | nopi }                                          # 0617
  { sub r8, r0, r8 | nopi | nopi | nopi }                                # 0618
  { nopi | nopi | nopi | nopi }                                          # 0619
__ccom_arith_20_bpos:                                                    # bundle 620
  { mov r11, 1 | mov r10, 31 | nopi | nopi }                             # 0620
  { mov r9, 0 | mov r6, 0 | nopi | nopi }                                # 0621
  { sll r14, r11, r10 | nopi | nopi | nopi }                             # 0622
  { mov r10, 32 | nopi | nopi | nopi }                                   # 0623
  { nopi | nopi | nopi | nopi }                                          # 0624
__ccom_arith_21_div:                                                     # bundle 625
  { slt r12, r7, r0 | sll r9, r9, r11 | nopi | nopi }                    # 0625
  { sll r7, r7, r11 | sll r6, r6, r11 | nopi | nopi }                    # 0626
  { or r9, r9, r12 | nopi | nopi | nopi }                                # 0627
  { xor r12, r8, r14 | nopi | nopi | nopi }                              # 0628
  { xor r13, r9, r14 | nopi | nopi | nopi }                              # 0629
  { nopi | nopi | nopi | nopi }                                          # 0630
  { nopi | nopi | blt r13, r12, __ccom_arith_22_skip | nopi }            # 0631
  { nopi | nopi | nopi | nopi }                                          # 0632
```

```
  { sub r9, r9, r8 | saddi r6, r6, 1 | nopi | nopi }                    # 0633
  { nopi | nopi | nopi | nopi }                                         # 0634
__ccom_arith_22_skip:                                                   # bundle 635
  { saddi r10, r10, -1 | nopi | nopi | nopi }                           # 0635
  { nopi | nopi | nopi | nopi }                                         # 0636
  { nopi | nopi | bne r10, r0, __ccom_arith_21_div | nopi }              # 0637
  { nopi | nopi | nopi | nopi }                                         # 0638
  { xor r13, r4, r5 | nopi | nopi | nopi }                              # 0639
  { nopi | nopi | nopi | nopi }                                         # 0640
  { nopi | nopi | bge r13, r0, __ccom_arith_23_end | nopi }              # 0641
  { nopi | nopi | nopi | nopi }                                         # 0642
  { sub r6, r0, r6 | nopi | nopi | nopi }                               # 0643
  { nopi | nopi | nopi | nopi }                                         # 0644
__ccom_arith_23_end:                                                    # bundle 645
  { mov t0, 0 | nopi | nopi | sw t2, 292(rz) }                          # 0645  linea 48  slots 0=48,1=0,2=0,3=47
  { nopi | nopi | nopi | lw t1, 292(rz) }                               # 0646
  { nopi | nopi | nopi | sw t0, 312(rz) }                               # 0647
  { nopi | nopi | nopi | sw t1, 316(rz) }                               # 0648
  { nopi | nopi | nopi | nopi }                                         # 0649
.L_for_start_7:                                                         # bundle 650
  { nopi | nopi | nopi | lw t0, 312(rz) }                               # 0650
  { nopi | nopi | nopi | lw t1, 316(rz) }                               # 0651
  { nopi | nopi | nopi | nopi }                                         # 0652
  { nopi | nopi | bge t0, t1, .L_for_end_9 | nopi }                     # 0653
  { nopi | nopi | nopi | nopi }                                         # 0654
  { nopi | nopi | nopi | lw t0, 304(rz) }                               # 0655  linea 49
  { nopi | nopi | nopi | nopi }                                         # 0656
  { nopi | nopi | nopi | sw t0, 0(rx) }                                 # 0657
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 308(rz) }                    # 0658
  { nopi | nopi | nopi | nopi }                                         # 0659
  { nopi | nopi | nopi | sw t0, 0(rx) }                                 # 0660
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 312(rz) }                    # 0661
  { nopi | nopi | nopi | nopi }                                         # 0662
  { nopi | nopi | nopi | sw t0, 0(rx) }                                 # 0663
  { saddi rx, rx, 4 | nopi | nopi | nopi }                              # 0664
  { nopi | nopi | nopi | nopi }                                         # 0665
  { saddi rx, rx, -4 | nopi | nopi | nopi }                             # 0666
  { nopi | nopi | nopi | nopi }                                         # 0667
  { nopi | nopi | nopi | lw t0, 0(rx) }                                 # 0668
  { saddi rx, rx, -4 | nopi | nopi | nopi }                             # 0669
  { nopi | nopi | nopi | sw t0, 28(rz) }                                # 0670
  { nopi | nopi | nopi | lw t0, 0(rx) }                                 # 0671
  { saddi rx, rx, -4 | nopi | nopi | nopi }                             # 0672
  { nopi | nopi | nopi | sw t0, 24(rz) }                                # 0673
  { nopi | nopi | nopi | lw t0, 0(rx) }                                 # 0674
  { nopi | nopi | nopi | nopi }                                         # 0675
  { nopi | nopi | nopi | sw t0, 20(rz) }                                # 0676
  { nopi | nopi | nopi | sw rl, 0(rx) }                                 # 0677
  { saddi rx, rx, 4 | nopi | jas rl, .L_func_bloques_de | nopi }        # 0678
  { nopi | nopi | nopi | nopi }                                         # 0679
  { saddi rx, rx, -4 | mov t1, 2 | nopi | nopi }                        # 0680  slots 0=49,1=50,2=0,3=0
  { nopi | nopi | nopi | nopi }                                         # 0681
  { nopi | nopi | nopi | lw rl, 0(rx) }                                 # 0682
  { nopi | nopi | nopi | lw t0, 16(rz) }                                # 0683
  { nopi | nopi | nopi | nopi }                                         # 0684
  { nopi | nopi | nopi | sw t0, 320(rz) }                               # 0685
  { nopi | nopi | beq r5, r0, .L_arith_error | lw t0, 312(rz) }         # 0686  linea 50
  { nopi | nopi | nopi | nopi }                                         # 0687
  { add r7, r4, r0 | add r8, r5, r0 | nopi | nopi }                     # 0688
  { nopi | nopi | nopi | nopi }                                         # 0689
  { nopi | nopi | bge r7, r0, __ccom_arith_24_apos | nopi }              # 0690
  { nopi | nopi | nopi | nopi }                                         # 0691
  { sub r7, r0, r7 | nopi | nopi | nopi }                               # 0692
  { nopi | nopi | nopi | nopi }                                         # 0693
__ccom_arith_24_apos:                                                   # bundle 694
  { nopi | nopi | bge r8, r0, __ccom_arith_25_bpos | nopi }              # 0694
  { nopi | nopi | nopi | nopi }                                         # 0695
  { sub r8, r0, r8 | nopi | nopi | nopi }                               # 0696
  { nopi | nopi | nopi | nopi }                                         # 0697
__ccom_arith_25_bpos:                                                   # bundle 698
  { mov r11, 1 | mov r10, 31 | nopi | nopi }                            # 0698
  { mov r9, 0 | mov r6, 0 | nopi | nopi }                               # 0699
  { sll r14, r11, r10 | nopi | nopi | nopi }                            # 0700
  { mov r10, 32 | nopi | nopi | nopi }                                  # 0701
  { nopi | nopi | nopi | nopi }                                         # 0702
__ccom_arith_26_div:                                                    # bundle 703
  { slt r12, r7, r0 | sll r9, r9, r11 | nopi | nopi }                   # 0703
  { sll r7, r7, r11 | sll r6, r6, r11 | nopi | nopi }                   # 0704
  { or r9, r9, r12 | nopi | nopi | nopi }                               # 0705
  { xor r12, r8, r14 | nopi | nopi | nopi }                             # 0706
  { xor r13, r9, r14 | nopi | nopi | nopi }                             # 0707
  { nopi | nopi | nopi | nopi }                                         # 0708
  { nopi | nopi | blt r13, r12, __ccom_arith_27_skip | nopi }           # 0709
  { nopi | nopi | nopi | nopi }                                         # 0710
  { sub r9, r9, r8 | saddi r6, r6, 1 | nopi | nopi }                    # 0711
  { nopi | nopi | nopi | nopi }                                         # 0712
__ccom_arith_27_skip:                                                   # bundle 713
  { saddi r10, r10, -1 | nopi | nopi | nopi }                           # 0713
  { nopi | nopi | nopi | nopi }                                         # 0714
  { nopi | nopi | bne r10, r0, __ccom_arith_26_div | nopi }              # 0715
  { nopi | nopi | nopi | nopi }                                         # 0716
  { add r6, r9, r0 | nopi | bge r4, r0, __ccom_arith_28_end | nopi }    # 0717
  { nopi | nopi | nopi | nopi }                                         # 0718
  { sub r6, r0, r6 | nopi | nopi | nopi }                               # 0719
  { nopi | nopi | nopi | nopi }                                         # 0720
__ccom_arith_28_end:                                                    # bundle 721
  { mov t0, 0 | nopi | nopi | nopi }                                    # 0721
  { nopi | nopi | nopi | nopi }                                         # 0722
  { seq t2, t2, t0 | nopi | nopi | nopi }                               # 0723
  { nopi | nopi | nopi | nopi }                                         # 0724
  { nopi | nopi | beq t2, rz, .L_if_next_10 | nopi }                    # 0725
  { nopi | nopi | nopi | nopi }                                         # 0726
  { nopi | nopi | nopi | nopi }                                         # 0727
  { nopi | nopi | nopi | lw t0, 296(rz) }                               # 0728  linea 51
  { nopi | nopi | nopi | nopi }                                         # 0729
  { nopi | nopi | nopi | sw t0, 0(rx) }                                 # 0730
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 308(rz) }                    # 0731
  { nopi | nopi | nopi | nopi }                                         # 0732
  { nopi | nopi | nopi | sw t0, 0(rx) }                                 # 0733
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 312(rz) }                    # 0734
  { nopi | nopi | nopi | nopi }                                         # 0735
  { nopi | nopi | nopi | sw t0, 0(rx) }                                 # 0736
  { saddi rx, rx, 4 | nopi | nopi | nopi }                              # 0737
  { nopi | nopi | nopi | nopi }                                         # 0738
  { saddi rx, rx, -4 | nopi | nopi | nopi }                             # 0739
```

```
  { nopi | nopi | nopi | nopi }                                              # 0740
  { nopi | nopi | nopi | lw t0, 0(rx) }                                      # 0741
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0742
  { nopi | nopi | nopi | sw t0, 100(rz) }                                    # 0743
  { nopi | nopi | nopi | lw t0, 0(rx) }                                      # 0744
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0745
  { nopi | nopi | nopi | sw t0, 96(rz) }                                     # 0746
  { nopi | nopi | nopi | lw t0, 0(rx) }                                      # 0747
  { nopi | nopi | nopi | nopi }                                              # 0748
  { nopi | nopi | nopi | sw t0, 92(rz) }                                     # 0749
  { nopi | nopi | nopi | sw rl, 0(rx) }                                      # 0750
  { saddi rx, rx, 4 | nopi | jas rl, .L_func_direccion | nopi }              # 0751
  { nopi | nopi | nopi | nopi }                                              # 0752
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0753
  { nopi | nopi | nopi | nopi }                                              # 0754
  { nopi | nopi | nopi | lw rl, 0(rx) }                                      # 0755
  { nopi | nopi | nopi | lw t0, 88(rz) }                                     # 0756
  { nopi | nopi | nopi | nopi }                                              # 0757
  { saddi t2, t0, 0 | nopi | nopi | nopi }                                   # 0758
  { nopi | nopi | nopi | nopi }                                              # 0759
  { nopi | nopi | nopi | nopi }                                              # 0760
  { nopi | nopi | nopi | nopi }                                              # 0761
  { nopi | nopi | nopi | sw t2, 324(rz) }                                    # 0762
  { nopi | nopi | nopi | lw t0, 300(rz) }                                    # 0763
  { nopi | nopi | nopi | nopi }                                              # 0764
  { nopi | nopi | nopi | sw t0, 0(rx) }                                      # 0765
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 308(rz) }                         # 0766
  { nopi | nopi | nopi | nopi }                                              # 0767
  { nopi | nopi | nopi | sw t0, 0(rx) }                                      # 0768
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 312(rz) }                         # 0769
  { nopi | nopi | nopi | nopi }                                              # 0770
  { nopi | nopi | nopi | sw t0, 0(rx) }                                      # 0771
  { saddi rx, rx, 4 | nopi | nopi | nopi }                                   # 0772
  { nopi | nopi | nopi | nopi }                                              # 0773
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0774
  { nopi | nopi | nopi | nopi }                                              # 0775
  { nopi | nopi | nopi | lw t0, 0(rx) }                                      # 0776
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0777
  { nopi | nopi | nopi | sw t0, 100(rz) }                                    # 0778
  { nopi | nopi | nopi | lw t0, 0(rx) }                                      # 0779
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0780
  { nopi | nopi | nopi | sw t0, 96(rz) }                                     # 0781
  { nopi | nopi | nopi | lw t0, 0(rx) }                                      # 0782
  { nopi | nopi | nopi | nopi }                                              # 0783
  { nopi | nopi | nopi | sw t0, 92(rz) }                                     # 0784
  { nopi | nopi | nopi | sw rl, 0(rx) }                                      # 0785
  { saddi rx, rx, 4 | nopi | jas rl, .L_func_direccion | nopi }              # 0786
  { nopi | nopi | nopi | nopi }                                              # 0787
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0788
  { nopi | nopi | nopi | nopi }                                              # 0789
  { nopi | nopi | nopi | lw rl, 0(rx) }                                      # 0790
  { nopi | nopi | nopi | lw t0, 88(rz) }                                     # 0791
  { nopi | nopi | nopi | lw t2, 324(rz) }                                    # 0792
  { saddi t3, t0, 0 | nopi | nopi | nopi }                                   # 0793
  { nopi | nopi | nopi | nopi }                                              # 0794
  { nopi | nopi | nopi | nopi }                                              # 0795
  { nopi | nopi | nopi | nopi }                                              # 0796
  { nopi | nopi | nopi | sw t2, 324(rz) }                                    # 0797
  { nopi | nopi | nopi | sw t3, 328(rz) }                                    # 0798
  { nopi | nopi | nopi | lw t0, 320(rz) }                                    # 0799
  { nopi | nopi | nopi | lw t2, 324(rz) }                                    # 0800
  { saddi t5, t0, 0 | nopi | nopi | lw t3, 328(rz) }                         # 0801
  { nopi | nopi | nopi | nopi }                                              # 0802
  { nopi | nopi | nopi | nopi }                                              # 0803
  { mov t4, 0 | nopi | nopi | nopi }                                         # 0804
  { nopi | nopi | nopi | nopi }                                              # 0805
  { nopi | nopi | bge rz, t5, .L_crypto_fin_descifra_3 | nopi }              # 0806
.L_crypto_descifra_3:                                                        # bundle 807
  { nopi | nopi | nopi | lw t0, 4(t2) }                                      # 0807
  { nopi | nopi | nopi | lw t1, 0(t2) }                                      # 0808
  { saddi t2, t2, 8 | nopi | nopi | nopi }                                   # 0809
  { nopi | feis t0, t0, t1, 0, 3 | nopi | nopi }                             # 0810
  { saddi t4, t4, 1 | nopi | nopi | nopi }                                   # 0811
  { nopi | feis t1, t1, t0, 0, 2 | nopi | nopi }                             # 0812
  { nopi | nopi | nopi | nopi }                                              # 0813
  { nopi | feis t0, t0, t1, 0, 1 | nopi | nopi }                             # 0814
  { nopi | nopi | nopi | nopi }                                              # 0815
  { nopi | feis t1, t1, t0, 0, 0 | nopi | nopi }                             # 0816
  { nopi | nopi | nopi | sw t0, 4(t3) }                                      # 0817
  { saddi t3, t3, 8 | nopi | blt t4, t5, .L_crypto_descifra_3 | sw t1, 0(t3) } # 0818
.L_crypto_fin_descifra_3:                                                    # bundle 819
  { nopi | nopi | nopi | nopi }                                              # 0819
  { nopi | nopi | jef .L_if_end_11 | nopi }                                  # 0820  linea 50
  { nopi | nopi | nopi | nopi }                                              # 0821
.L_if_next_10:                                                               # bundle 822
  { nopi | nopi | nopi | nopi }                                              # 0822
  { nopi | nopi | nopi | lw t0, 296(rz) }                                    # 0823  linea 53
  { nopi | nopi | nopi | nopi }                                              # 0824
  { nopi | nopi | nopi | sw t0, 0(rx) }                                      # 0825
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 308(rz) }                         # 0826
  { nopi | nopi | nopi | nopi }                                              # 0827
  { nopi | nopi | nopi | sw t0, 0(rx) }                                      # 0828
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 312(rz) }                         # 0829
  { nopi | nopi | nopi | nopi }                                              # 0830
  { nopi | nopi | nopi | sw t0, 0(rx) }                                      # 0831
  { saddi rx, rx, 4 | nopi | nopi | nopi }                                   # 0832
  { nopi | nopi | nopi | nopi }                                              # 0833
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0834
  { nopi | nopi | nopi | nopi }                                              # 0835
  { nopi | nopi | nopi | lw t0, 0(rx) }                                      # 0836
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0837
  { nopi | nopi | nopi | sw t0, 100(rz) }                                    # 0838
  { nopi | nopi | nopi | lw t0, 0(rx) }                                      # 0839
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0840
  { nopi | nopi | nopi | sw t0, 96(rz) }                                     # 0841
  { nopi | nopi | nopi | lw t0, 0(rx) }                                      # 0842
  { nopi | nopi | nopi | nopi }                                              # 0843
  { nopi | nopi | nopi | sw t0, 92(rz) }                                     # 0844
  { nopi | nopi | nopi | sw rl, 0(rx) }                                      # 0845
  { saddi rx, rx, 4 | nopi | jas rl, .L_func_direccion | nopi }              # 0846
  { nopi | nopi | nopi | nopi }                                              # 0847
  { saddi rx, rx, -4 | nopi | nopi | nopi }                                  # 0848
  { nopi | nopi | nopi | nopi }                                              # 0849
  { nopi | nopi | nopi | lw rl, 0(rx) }                                      # 0850
  { nopi | nopi | nopi | lw t0, 88(rz) }                                     # 0851
```

```
  { nopi | nopi | nopi | nopi }                                          # 0852
  { saddi t2, t0, 0 | nopi | nopi | nopi }                               # 0853
  { nopi | nopi | nopi | nopi }                                          # 0854
  { nopi | nopi | nopi | nopi }                                          # 0855
  { nopi | nopi | nopi | nopi }                                          # 0856
  { nopi | nopi | nopi | sw t2, 324(rz) }                                # 0857
  { nopi | nopi | nopi | lw t0, 300(rz) }                                # 0858
  { nopi | nopi | nopi | nopi }                                          # 0859
  { nopi | nopi | nopi | sw t0, 0(rx) }                                  # 0860
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 308(rz) }                     # 0861
  { nopi | nopi | nopi | nopi }                                          # 0862
  { nopi | nopi | nopi | sw t0, 0(rx) }                                  # 0863
  { saddi rx, rx, 4 | nopi | nopi | lw t0, 312(rz) }                     # 0864
  { nopi | nopi | nopi | nopi }                                          # 0865
  { nopi | nopi | nopi | sw t0, 0(rx) }                                  # 0866
  { saddi rx, rx, 4 | nopi | nopi | nopi }                               # 0867
  { nopi | nopi | nopi | nopi }                                          # 0868
  { saddi rx, rx, -4 | nopi | nopi | nopi }                              # 0869
  { nopi | nopi | nopi | nopi }                                          # 0870
  { nopi | nopi | nopi | lw t0, 0(rx) }                                  # 0871
  { saddi rx, rx, -4 | nopi | nopi | nopi }                              # 0872
  { nopi | nopi | nopi | sw t0, 100(rz) }                                # 0873
  { nopi | nopi | nopi | lw t0, 0(rx) }                                  # 0874
  { saddi rx, rx, -4 | nopi | nopi | nopi }                              # 0875
  { nopi | nopi | nopi | sw t0, 96(rz) }                                 # 0876
  { nopi | nopi | nopi | lw t0, 0(rx) }                                  # 0877
  { nopi | nopi | nopi | nopi }                                          # 0878
  { nopi | nopi | nopi | sw t0, 92(rz) }                                 # 0879
  { nopi | nopi | nopi | sw rl, 0(rx) }                                  # 0880
  { saddi rx, rx, 4 | nopi | jas rl, .L_func_direccion | nopi }          # 0881
  { nopi | nopi | nopi | nopi }                                          # 0882
  { saddi rx, rx, -4 | nopi | nopi | nopi }                              # 0883
  { nopi | nopi | nopi | nopi }                                          # 0884
  { nopi | nopi | nopi | lw rl, 0(rx) }                                  # 0885
  { nopi | nopi | nopi | lw t0, 88(rz) }                                 # 0886
  { nopi | nopi | nopi | lw t2, 324(rz) }                                # 0887
  { saddi t3, t0, 0 | nopi | nopi | nopi }                               # 0888
  { nopi | nopi | nopi | nopi }                                          # 0889
  { nopi | nopi | nopi | nopi }                                          # 0890
  { nopi | nopi | nopi | nopi }                                          # 0891
  { nopi | nopi | nopi | sw t2, 324(rz) }                                # 0892
  { nopi | nopi | nopi | sw t3, 328(rz) }                                # 0893
  { nopi | nopi | nopi | lw t0, 320(rz) }                                # 0894
  { nopi | nopi | nopi | lw t2, 324(rz) }                                # 0895
  { saddi t5, t0, 0 | nopi | nopi | lw t3, 328(rz) }                     # 0896
  { nopi | nopi | nopi | nopi }                                          # 0897
  { nopi | nopi | nopi | nopi }                                          # 0898
  { mov t4, 0 | nopi | nopi | nopi }                                     # 0899
  { nopi | nopi | nopi | nopi }                                          # 0900
  { nopi | nopi | bge rz, t5, .L_crypto_fin_descifra_4 | nopi }           # 0901
.L_crypto_descifra_4:                                                    # bundle 902
  { nopi | nopi | nopi | lw t0, 4(t2) }                                  # 0902
  { nopi | nopi | nopi | lw t1, 0(t2) }                                  # 0903
  { saddi t2, t2, 8 | nopi | nopi | nopi }                               # 0904
  { nopi | feis t0, t0, t1, 1, 3 | nopi | nopi }                         # 0905
  { saddi t4, t4, 1 | nopi | nopi | nopi }                               # 0906
  { nopi | feis t1, t1, t0, 1, 2 | nopi | nopi }                         # 0907
  { nopi | nopi | nopi | nopi }                                          # 0908
  { nopi | feis t0, t0, t1, 1, 1 | nopi | nopi }                         # 0909
  { nopi | nopi | nopi | nopi }                                          # 0910
  { nopi | feis t1, t1, t0, 1, 0 | nopi | nopi }                         # 0911
  { nopi | nopi | nopi | sw t0, 4(t3) }                                  # 0912
  { saddi t3, t3, 8 | nopi | blt t4, t5, .L_crypto_descifra_4 | sw t1, 0(t3) }  # 0913
.L_crypto_fin_descifra_4:                                                # bundle 914
  { nopi | nopi | nopi | nopi }                                          # 0914
.L_if_end_11:                                                            # bundle 915
.L_for_update_8:                                                         # bundle 915
  { nopi | nopi | nopi | lw t0, 312(rz) }                                # 0915  linea 48
  { nopi | nopi | nopi | nopi }                                          # 0916
  { saddi t0, t0, 1 | nopi | nopi | nopi }                               # 0917
  { nopi | nopi | nopi | nopi }                                          # 0918
  { nopi | nopi | jef .L_for_start_7 | sw t0, 312(rz) }                  # 0919
  { nopi | nopi | nopi | nopi }                                          # 0920
.L_for_end_9:                                                            # bundle 921
  { nopi | nopi | jefR rz, rl | nopi }                                   # 0921  linea 56
  { nopi | nopi | nopi | nopi }                                          # 0922
fin_main:                                                                # bundle 923
  { nopi | nopi | jef fin_main | nopi }                                  # 0923  linea -
.L_arith_error:                                                          # bundle 924
  { nopi | nopi | jef .L_arith_error | nopi }                            # 0924

# Tabla de etiquetas (direccion = indice de bundle)
#   .L_func_main                    2
#   .L_body_main                    2
#   .L_func_bloques_de            104
#   .L_body_bloques_de            106
#   __ccom_arith_0_mul            112  (interna)
#   __ccom_arith_1_skip           120  (interna)
#   __ccom_arith_2_end            122  (interna)
#   .L_if_next_0                  136
#   .L_if_end_1                   140
#   .L_func_direccion             142
#   .L_body_direccion             144
#   __ccom_arith_3_mul            150  (interna)
#   __ccom_arith_4_skip           158  (interna)
#   __ccom_arith_5_end            160  (interna)
#   __ccom_arith_6_mul            164  (interna)
#   __ccom_arith_7_skip           172  (interna)
#   __ccom_arith_8_end            174  (interna)
#   .L_func_preparar_boveda       178
#   .L_body_preparar_boveda       178
#   .L_func_cifrar_archivo        271
#   .L_body_cifrar_archivo        273
#   __ccom_arith_9_apos           290  (interna)
#   __ccom_arith_10_bpos          294  (interna)
#   __ccom_arith_11_div           299  (interna)
#   __ccom_arith_12_skip          309  (interna)
#   __ccom_arith_13_end           319  (interna)
#   .L_for_start_2                324
#   __ccom_arith_14_apos          368  (interna)
#   __ccom_arith_15_bpos          372  (interna)
#   __ccom_arith_16_div           377  (interna)
#   __ccom_arith_17_skip          387  (interna)
#   __ccom_arith_18_end           395  (interna)
#   .L_crypto_cifra_1             481  (interna)
```

```
#   .L_crypto_fin_cifra_1          493  (interna)
#   .L_if_next_5                   496
#   .L_crypto_cifra_2              576  (interna)
#   .L_crypto_fin_cifra_2          588  (interna)
#   .L_if_end_6                    589
#   .L_for_update_3                589
#   .L_for_end_4                   595
#   .L_func_descifrar_archivo      597
#   .L_body_descifrar_archivo      599
#   __ccom_arith_19_apos           616  (interna)
#   __ccom_arith_20_bpos           620  (interna)
#   __ccom_arith_21_div            625  (interna)
#   __ccom_arith_22_skip           635  (interna)
#   __ccom_arith_23_end            645  (interna)
#   .L_for_start_7                 650
#   __ccom_arith_24_apos           694  (interna)
#   __ccom_arith_25_bpos           698  (interna)
#   __ccom_arith_26_div            703  (interna)
#   __ccom_arith_27_skip           713  (interna)
#   __ccom_arith_28_end            721  (interna)
#   .L_crypto_descifra_3           807  (interna)
#   .L_crypto_fin_descifra_3       819  (interna)
#   .L_if_next_10                  822
#   .L_crypto_descifra_4           902  (interna)
#   .L_crypto_fin_descifra_4       914  (interna)
#   .L_if_end_11                   915
#   .L_for_update_8                915
#   .L_for_end_9                   921
#   fin_main                       923
#   .L_arith_error                 924

# Variables (direccion de datos)
#
#   ===== SYMBOL TABLE =====
#   Scope 0: GLOBAL
#       main : void (function) params=[] area=0x0174 size=56 temp_base=0x017C
#       descifrar_archivo : int (function) params=[desde, hacia, total, tam] area=0x0120 size=84 temp_base=0x0144
#       cifrar_archivo : int (function) params=[desde, hacia, total, tam] area=0x00CC size=84 temp_base=0x00F0
#       preparar_boveda : void (function) params=[] area=0x0098 size=52 temp_base=0x009C
#       direccion : int (function) params=[base, tam, i] area=0x0054 size=68 temp_base=0x0068
#       bloques_de : int (function) params=[total, tam, i] area=0x000C size=72 temp_base=0x0024
#       vuelta : int width=4 addr=0x0008 line=3
#       cifrado : int width=4 addr=0x0004 line=2
#       origen : int width=4 addr=0x0000 line=1
#
#   Scope 1: FUNCTION bloques_de
#       resto : int width=4 addr=0x0020 line=6
#       i : int (parameter) width=4 addr=0x001C line=5
#       tam : int (parameter) width=4 addr=0x0018 line=5
#       total : int (parameter) width=4 addr=0x0014 line=5
#       b : int (return) width=4 addr=0x0010 line=5
#
#   Scope 2: C bloques_de
#
#   Scope 3: RESIDUAL bloques_de
#
#   Scope 4: FUNCTION direccion
#       i : int (parameter) width=4 addr=0x0064 line=15
#       tam : int (parameter) width=4 addr=0x0060 line=15
#       base : int (parameter) width=4 addr=0x005C line=15
#       d : int (return) width=4 addr=0x0058 line=15
#
#   Scope 5: FUNCTION preparar_boveda
#
#   Scope 6: FUNCTION cifrar_archivo
#       tam : int (parameter) width=4 addr=0x00E0 line=33
#       total : int (parameter) width=4 addr=0x00DC line=33
#       hacia : int (parameter) width=4 addr=0x00D8 line=33
#       desde : int (parameter) width=4 addr=0x00D4 line=33
#       n : int (return) width=4 addr=0x00D0 line=33
#
#   Scope 7: S cifrar_archivo
#       b : int width=4 addr=0x00EC line=36
#       i : int width=4 addr=0x00E4 line=35
#
#   Scope 8: C cifrar_archivo
#
#   Scope 9: RESIDUAL cifrar_archivo
#
#   Scope 10: FUNCTION descifrar_archivo
#       tam : int (parameter) width=4 addr=0x0134 line=46
#       total : int (parameter) width=4 addr=0x0130 line=46
#       hacia : int (parameter) width=4 addr=0x012C line=46
#       desde : int (parameter) width=4 addr=0x0128 line=46
#       n : int (return) width=4 addr=0x0124 line=46
#
#   Scope 11: S descifrar_archivo
#       b : int width=4 addr=0x0140 line=49
#       i : int width=4 addr=0x0138 line=48
#
#   Scope 12: C descifrar_archivo
#
#   Scope 13: RESIDUAL descifrar_archivo
#
#   Scope 14: FUNCTION main
#       n2 : int width=4 addr=0x0178 line=61
#       n1 : int width=4 addr=0x0174 line=60
#
#   ======================
```

### *F-D. Binario en hexadecimal*

Cada entrada es el número de bundle y sus 128 bits, con el slot 3 a la izquierda; se leen por columnas.

```
0000 00000000_00000000_00000000_488001AC   0309 00000000_00000000_00000000_42ABFFFF   0618 00000000_00000000_00000000_0A020000
0001 00000000_00000000_00000000_00000000   0310 00000000_00000000_00000000_00000000   0619 00000000_00000000_00000000_00000000
0002 00000000_00000000_00000000_49000000   0311 00000000_082BFFF4_00000000_00000000   0620 00000000_00000000_4A80001F_4AC00001
0003 00000000_00000000_00000000_00000000   0312 00000000_00000000_00000000_00000000   0621 00000000_00000000_49800000_4A400000
0004 09000174_00000000_00000000_00000000   0313 00000000_00000000_00000000_17514000   0622 00000000_00000000_00000000_1BAE8000
0005 00000000_00000000_00000000_49000000   0314 00000000_00000000_00000000_00000000   0623 00000000_00000000_00000000_4A800020
0006 00000000_00000000_00000000_00000000   0315 00000000_10340004_00000000_00000000   0624 00000000_00000000_00000000_00000000
```

```
0007 09000178_00000000_00000000_00000000
0008 00000000_00000000_00000000_00000000
0009 00000000_00000000_00000000_00000000
0010 00000000_00000000_4940000F_49000025
0011 00000000_00000000_00000000_00000000
0012 00000000_00000000_00000000_19114000
0013 00000000_00000000_00000000_00000000
0014 00000000_00000000_00000000_41105687
0015 00000000_00000000_00000000_00000000
0016 00000000_00000000_00000000_42900000
0017 00000000_00000000_00000000_00000000
0018 00000000_00000000_00000000_00000000
0019 00000000_00000000_88280000_00000000
0020 08480000_00000000_00000000_00000000
0021 00000000_4440009B_00000000_40880004
0022 00000000_00000000_00000000_00000000
0023 00000000_00000000_00000000_408BFFFC
0024 00000000_00000000_00000000_00000000
0025 04480000_00000000_00000000_00000000
0026 05000000_00000000_00000000_00000000
0027 00000000_00000000_00000000_00000000
0028 09080000_00000000_00000000_00000000
0029 05000004_00000000_00000000_40880004
0030 00000000_00000000_00000000_00000000
0031 09080000_00000000_00000000_00000000
0032 00000000_00000000_49000080_40880004
0033 00000000_00000000_00000000_00000000
0034 09080000_00000000_00000000_00000000
0035 00000000_00000000_49000010_40880004
0036 00000000_00000000_00000000_00000000
0037 09080000_00000000_00000000_00000000
0038 00000000_00000000_00000000_40880004
0039 00000000_00000000_00000000_00000000
0040 00000000_00000000_00000000_408BFFFC
0041 00000000_00000000_00000000_00000000
0042 05080000_00000000_00000000_00000000
0043 00000000_00000000_00000000_408BFFFC
0044 090000E0_00000000_00000000_00000000
0045 05080000_00000000_00000000_00000000
0046 00000000_00000000_00000000_408BFFFC
0047 090000DC_00000000_00000000_00000000
0048 05080000_00000000_00000000_00000000
0049 00000000_00000000_00000000_408BFFFC
0050 090000D8_00000000_00000000_00000000
0051 05080000_00000000_00000000_00000000
0052 00000000_00000000_00000000_00000000
0053 090000D4_00000000_00000000_00000000
0054 08480000_00000000_00000000_00000000
0055 00000000_444000D8_00000000_40880004
0056 00000000_00000000_00000000_00000000
0057 00000000_00000000_00000000_408BFFFC
0058 00000000_00000000_00000000_00000000
0059 04480000_00000000_00000000_00000000
0060 050000D0_00000000_00000000_00000000
0061 00000000_00000000_00000000_00000000
0062 09000174_00000000_00000000_00000000
0063 05000004_00000000_00000000_00000000
0064 00000000_00000000_00000000_00000000
0065 09080000_00000000_00000000_00000000
0066 05000008_00000000_00000000_40880004
0067 00000000_00000000_00000000_00000000
0068 09080000_00000000_00000000_00000000
0069 00000000_00000000_49000080_40880004
0070 00000000_00000000_00000000_00000000
0071 09080000_00000000_00000000_00000000
0072 00000000_00000000_49000010_40880004
0073 00000000_00000000_00000000_00000000
0074 09080000_00000000_00000000_00000000
0075 00000000_00000000_00000000_40880004
0076 00000000_00000000_00000000_00000000
0077 00000000_00000000_00000000_408BFFFC
0078 00000000_00000000_00000000_00000000
0079 05080000_00000000_00000000_00000000
0080 00000000_00000000_00000000_408BFFFC
0081 09000134_00000000_00000000_00000000
0082 05080000_00000000_00000000_00000000
0083 00000000_00000000_00000000_408BFFFC
0084 09000130_00000000_00000000_00000000
0085 05080000_00000000_00000000_00000000
0086 00000000_00000000_00000000_408BFFFC
0087 0900012C_00000000_00000000_00000000
0088 05080000_00000000_00000000_00000000
0089 00000000_00000000_00000000_00000000
0090 09000128_00000000_00000000_00000000
0091 08480000_00000000_00000000_00000000
0092 00000000_444001F9_00000000_40880004
0093 00000000_00000000_00000000_00000000
0094 00000000_00000000_00000000_408BFFFC
0095 00000000_00000000_00000000_00000000
0096 04480000_00000000_00000000_00000000
0097 05000124_00000000_00000000_00000000
0098 00000000_00000000_00000000_00000000
0099 09000178_00000000_00000000_00000000
0100 00000000_00000000_00000000_00000000
0101 00000000_00000000_8C000000_00000000
0102 00000000_40000335_00000000_00000000
0103 00000000_00000000_00000000_00000000
0104 08000010_00000000_00000000_00000000
0105 00000000_00000000_00000000_00000000
0106 05000014_00000000_49C00000_4AC00001
0107 0540001C_00000000_00000000_00000000
0108 05800018_00000000_00000000_00000000
0109 00000000_00000000_00000000_06140000
0110 00000000_00000000_00000000_06580000
0111 00000000_00000000_00000000_00000000
0112 00000000_0424000A_00000000_00000000
0113 00000000_00000000_00000000_00000000
0114 00000000_00000000_00000000_0EA6C000
0115 00000000_00000000_00000000_00000000
0116 00000000_04280004_00000000_00000000
0117 00000000_00000000_00000000_00000000
0118 00000000_00000000_00000000_05DE0000
0119 00000000_00000000_00000000_00000000
0120 00000000_403FFFF8_1E66C000_1A22C000
0121 00000000_00000000_00000000_00000000
0316 00000000_00000000_00000000_00000000
0317 00000000_00000000_00000000_09818000
0318 00000000_00000000_00000000_00000000
0319 098000D0_00000000_00000000_49000000
0320 054000D0_00000000_00000000_00000000
0321 090000E4_00000000_00000000_00000000
0322 094000E8_00000000_00000000_00000000
0323 00000000_00000000_00000000_00000000
0324 050000E4_00000000_00000000_00000000
0325 054000E8_00000000_00000000_00000000
0326 00000000_00000000_00000000_00000000
0327 00000000_1150010C_00000000_00000000
0328 00000000_00000000_00000000_00000000
0329 050000DC_00000000_00000000_00000000
0330 00000000_00000000_00000000_00000000
0331 09080000_00000000_00000000_00000000
0332 050000E0_00000000_00000000_40880004
0333 00000000_00000000_00000000_00000000
0334 09080000_00000000_00000000_00000000
0335 050000E4_00000000_00000000_40880004
0336 00000000_00000000_00000000_00000000
0337 09080000_00000000_00000000_00000000
0338 00000000_00000000_00000000_40880004
0339 00000000_00000000_00000000_00000000
0340 00000000_00000000_00000000_408BFFFC
0341 00000000_00000000_00000000_00000000
0342 05080000_00000000_00000000_00000000
0343 00000000_00000000_00000000_408BFFFC
0344 0900001C_00000000_00000000_00000000
0345 05080000_00000000_00000000_00000000
0346 00000000_00000000_00000000_408BFFFC
0347 09000018_00000000_00000000_00000000
0348 05080000_00000000_00000000_00000000
0349 00000000_00000000_00000000_00000000
0350 09000014_00000000_00000000_00000000
0351 08480000_00000000_00000000_00000000
0352 00000000_447FFF08_00000000_40880004
0353 00000000_00000000_00000000_00000000
0354 00000000_00000000_49400002_408BFFFC
0355 00000000_00000000_00000000_00000000
0356 04480000_00000000_00000000_00000000
0357 05000010_00000000_00000000_00000000
0358 00000000_00000000_00000000_00000000
0359 090000EC_00000000_00000000_00000000
0360 050000E4_04140234_00000000_00000000
0361 00000000_00000000_00000000_00000000
0362 00000000_00000000_06140000_05D00000
0363 00000000_00000000_00000000_00000000
0364 00000000_101C0004_00000000_00000000
0365 00000000_00000000_00000000_00000000
0366 00000000_00000000_00000000_09C1C000
0367 00000000_00000000_00000000_00000000
0368 00000000_10200004_00000000_00000000
0369 00000000_00000000_00000000_00000000
0370 00000000_00000000_00000000_0A020000
0371 00000000_00000000_00000000_00000000
0372 00000000_00000000_4A80001F_4AC00001
0373 00000000_00000000_49800000_4A400000
0374 00000000_00000000_00000000_1BAE8000
0375 00000000_00000000_00000000_4A800020
0376 00000000_00000000_00000000_00000000
0377 00000000_00000000_1A66C000_271C0000
0378 00000000_00000000_199AC000_19DEC000
0379 00000000_00000000_00000000_12670000
0380 00000000_00000000_00000000_17238000
0381 00000000_00000000_00000000_17678000
0382 00000000_00000000_00000000_00000000
0383 00000000_0F340004_00000000_00000000
0384 00000000_00000000_00000000_00000000
0385 00000000_00000000_41980001_0A660000
0386 00000000_00000000_00000000_00000000
0387 00000000_00000000_00000000_42ABFFFF
0388 00000000_00000000_00000000_00000000
0389 00000000_082BFFF4_00000000_00000000
0390 00000000_00000000_00000000_00000000
0391 00000000_10100004_00000000_05A40000
0392 00000000_00000000_00000000_00000000
0393 00000000_00000000_00000000_09818000
0394 00000000_00000000_00000000_00000000
0395 00000000_00000000_00000000_49000000
0396 00000000_00000000_00000000_00000000
0397 00000000_00000000_00000000_2D990000
0398 00000000_00000000_00000000_00000000
0399 00000000_04180061_00000000_00000000
0400 00000000_00000000_00000000_00000000
0401 00000000_00000000_00000000_00000000
0402 050000D4_00000000_00000000_00000000
0403 00000000_00000000_00000000_00000000
0404 09080000_00000000_00000000_00000000
0405 050000E0_00000000_00000000_40880004
0406 00000000_00000000_00000000_00000000
0407 09080000_00000000_00000000_00000000
0408 050000E4_00000000_00000000_40880004
0409 00000000_00000000_00000000_00000000
0410 09080000_00000000_00000000_00000000
0411 00000000_00000000_00000000_40880004
0412 00000000_00000000_00000000_00000000
0413 00000000_00000000_00000000_408BFFFC
0414 00000000_00000000_00000000_00000000
0415 05080000_00000000_00000000_00000000
0416 00000000_00000000_00000000_408BFFFC
0417 09000064_00000000_00000000_00000000
0418 05080000_00000000_00000000_00000000
0419 00000000_00000000_00000000_408BFFFC
0420 09000060_00000000_00000000_00000000
0421 05080000_00000000_00000000_00000000
0422 00000000_00000000_00000000_00000000
0423 0900005C_00000000_00000000_00000000
0424 08480000_00000000_00000000_00000000
0425 00000000_447FFEE5_00000000_40880004
0426 00000000_00000000_00000000_00000000
0427 00000000_00000000_00000000_408BFFFC
0428 00000000_00000000_00000000_00000000
0429 04480000_00000000_00000000_00000000
0430 05000058_00000000_00000000_00000000
0625 00000000_00000000_1A66C000_271C0000
0626 00000000_00000000_199AC000_19DEC000
0627 00000000_00000000_00000000_12670000
0628 00000000_00000000_00000000_17238000
0629 00000000_00000000_00000000_17678000
0630 00000000_00000000_00000000_00000000
0631 00000000_0F340004_00000000_00000000
0632 00000000_00000000_00000000_00000000
0633 00000000_00000000_41980001_0A660000
0634 00000000_00000000_00000000_00000000
0635 00000000_00000000_00000000_42ABFFFF
0636 00000000_00000000_00000000_00000000
0637 00000000_082BFFF4_00000000_00000000
0638 00000000_00000000_00000000_00000000
0639 00000000_00000000_00000000_17514000
0640 00000000_00000000_00000000_00000000
0641 00000000_10340004_00000000_00000000
0642 00000000_00000000_00000000_00000000
0643 00000000_00000000_00000000_09818000
0644 00000000_00000000_00000000_00000000
0645 09800124_00000000_00000000_49000000
0646 05400124_00000000_00000000_00000000
0647 09000138_00000000_00000000_00000000
0648 0940013C_00000000_00000000_00000000
0649 00000000_00000000_00000000_00000000
0650 05000138_00000000_00000000_00000000
0651 0540013C_00000000_00000000_00000000
0652 00000000_00000000_00000000_00000000
0653 00000000_1150010C_00000000_00000000
0654 00000000_00000000_00000000_00000000
0655 05000130_00000000_00000000_00000000
0656 00000000_00000000_00000000_00000000
0657 09080000_00000000_00000000_00000000
0658 05000134_00000000_00000000_40880004
0659 00000000_00000000_00000000_00000000
0660 09080000_00000000_00000000_00000000
0661 05000138_00000000_00000000_40880004
0662 00000000_00000000_00000000_00000000
0663 09080000_00000000_00000000_00000000
0664 00000000_00000000_00000000_40880004
0665 00000000_00000000_00000000_00000000
0666 00000000_00000000_00000000_408BFFFC
0667 00000000_00000000_00000000_00000000
0668 05080000_00000000_00000000_00000000
0669 00000000_00000000_00000000_408BFFFC
0670 0900001C_00000000_00000000_00000000
0671 05080000_00000000_00000000_00000000
0672 00000000_00000000_00000000_408BFFFC
0673 09000018_00000000_00000000_00000000
0674 05080000_00000000_00000000_00000000
0675 00000000_00000000_00000000_00000000
0676 09000014_00000000_00000000_00000000
0677 08480000_00000000_00000000_00000000
0678 00000000_447FFDC2_00000000_40880004
0679 00000000_00000000_00000000_00000000
0680 00000000_00000000_49400002_408BFFFC
0681 00000000_00000000_00000000_00000000
0682 04480000_00000000_00000000_00000000
0683 05000010_00000000_00000000_00000000
0684 00000000_00000000_00000000_00000000
0685 09000140_00000000_00000000_00000000
0686 05000138_041400EE_00000000_00000000
0687 00000000_00000000_00000000_00000000
0688 00000000_00000000_06140000_05D00000
0689 00000000_00000000_00000000_00000000
0690 00000000_101C0004_00000000_00000000
0691 00000000_00000000_00000000_00000000
0692 00000000_00000000_00000000_09C1C000
0693 00000000_00000000_00000000_00000000
0694 00000000_10200004_00000000_00000000
0695 00000000_00000000_00000000_00000000
0696 00000000_00000000_00000000_0A020000
0697 00000000_00000000_00000000_00000000
0698 00000000_00000000_4A80001F_4AC00001
0699 00000000_00000000_49800000_4A400000
0700 00000000_00000000_00000000_1BAE8000
0701 00000000_00000000_00000000_4A800020
0702 00000000_00000000_00000000_00000000
0703 00000000_00000000_1A66C000_271C0000
0704 00000000_00000000_199AC000_19DEC000
0705 00000000_00000000_00000000_12670000
0706 00000000_00000000_00000000_17238000
0707 00000000_00000000_00000000_17678000
0708 00000000_00000000_00000000_00000000
0709 00000000_0F340004_00000000_00000000
0710 00000000_00000000_00000000_00000000
0711 00000000_00000000_41980001_0A660000
0712 00000000_00000000_00000000_00000000
0713 00000000_00000000_00000000_42ABFFFF
0714 00000000_00000000_00000000_00000000
0715 00000000_082BFFF4_00000000_00000000
0716 00000000_00000000_00000000_00000000
0717 00000000_10100004_00000000_05A40000
0718 00000000_00000000_00000000_00000000
0719 00000000_00000000_00000000_09818000
0720 00000000_00000000_00000000_00000000
0721 00000000_00000000_00000000_49000000
0722 00000000_00000000_00000000_00000000
0723 00000000_00000000_00000000_2D990000
0724 00000000_00000000_00000000_00000000
0725 00000000_04180061_00000000_00000000
0726 00000000_00000000_00000000_00000000
0727 00000000_00000000_00000000_00000000
0728 05000128_00000000_00000000_00000000
0729 00000000_00000000_00000000_00000000
0730 09080000_00000000_00000000_00000000
0731 05000134_00000000_00000000_40880004
0732 00000000_00000000_00000000_00000000
0733 09080000_00000000_00000000_00000000
0734 05000138_00000000_00000000_40880004
0735 00000000_00000000_00000000_00000000
0736 09080000_00000000_00000000_00000000
0737 00000000_00000000_00000000_40880004
0738 00000000_00000000_00000000_00000000
0739 00000000_00000000_00000000_408BFFFC
```

```
0122 00000000_00000000_00000000_0911C000
0123 00000000_00000000_00000000_00000000
0124 09000020_00000000_00000000_00000000
0125 05000020_00000000_00000000_00000000
0126 05400018_00000000_00000000_00000000
0127 00000000_00000000_00000000_00000000
0128 00000000_00000000_00000000_25114000
0129 00000000_00000000_00000000_00000000
0130 00000000_04100006_00000000_00000000
0131 00000000_00000000_00000000_00000000
0132 05000020_00000000_00000000_00000000
0133 00000000_00000000_00000000_00000000
0134 09000010_40000006_00000000_00000000
0135 00000000_00000000_00000000_00000000
0136 05000018_00000000_00000000_00000000
0137 00000000_00000000_00000000_00000000
0138 09000010_00000000_00000000_00000000
0139 00000000_00000000_00000000_00000000
0140 00000000_14040000_00000000_00000000
0141 00000000_00000000_00000000_00000000
0142 08000058_00000000_00000000_00000000
0143 00000000_00000000_00000000_00000000
0144 0500005C_00000000_49C00000_4AC00001
0145 05400064_00000000_00000000_00000000
0146 05800060_00000000_00000000_00000000
0147 00000000_00000000_00000000_06140000
0148 00000000_00000000_00000000_06580000
0149 00000000_00000000_00000000_00000000
0150 00000000_0424000A_00000000_00000000
0151 00000000_00000000_00000000_00000000
0152 00000000_00000000_00000000_0EA6C000
0153 00000000_00000000_00000000_00000000
0154 00000000_04280004_00000000_00000000
0155 00000000_00000000_00000000_00000000
0156 00000000_00000000_00000000_05DE0000
0157 00000000_00000000_00000000_00000000
0158 00000000_403FFFF8_1E66C000_1A22C000
0159 00000000_00000000_00000000_00000000
0160 00000000_00000000_4AC00001_49400008
0161 00000000_00000000_49800000_061C0000
0162 00000000_00000000_00000000_06540000
0163 00000000_00000000_00000000_00000000
0164 00000000_0424000A_00000000_00000000
0165 00000000_00000000_00000000_00000000
0166 00000000_00000000_00000000_0EA6C000
0167 00000000_00000000_00000000_00000000
0168 00000000_04280004_00000000_00000000
0169 00000000_00000000_00000000_00000000
0170 00000000_00000000_00000000_059A0000
0171 00000000_00000000_00000000_00000000
0172 00000000_403FFFF8_1E66C000_1A22C000
0173 00000000_00000000_00000000_00000000
0174 00000000_00000000_00000000_05118000
0175 00000000_00000000_00000000_00000000
0176 09000058_14040000_00000000_00000000
0177 00000000_00000000_00000000_00000000
0178 00000000_00000000_00000000_00000000
0179 00000000_00000000_4940000F_49000247
0180 00000000_00000000_00000000_00000000
0181 00000000_00000000_00000000_19114000
0182 00000000_00000000_00000000_00000000
0183 00000000_00000000_00000000_41102BCD
0184 00000000_00000000_00000000_00000000
0185 00000000_00000000_00000000_42900000
0186 00000000_00000000_00000000_00000000
0187 00000000_00000000_00000000_00000000
0188 00000000_00000000_90280000_00000000
0189 00000000_00000000_00000000_00000000
0190 00000000_00000000_4940000F_4900ECA8
0191 00000000_00000000_00000000_00000000
0192 00000000_00000000_00000000_19114000
0193 00000000_00000000_00000000_00000000
0194 00000000_00000000_00000000_41103211
0195 00000000_00000000_00000000_00000000
0196 00000000_00000000_00000000_09010000
0197 00000000_00000000_00000000_00000000
0198 00000000_00000000_00000000_42900000
0199 00000000_00000000_00000000_00000000
0200 00000000_00000000_00000000_00000000
0201 00000000_00000000_90280400_00000000
0202 00000000_00000000_00000000_00000000
0203 00000000_00000000_4940000F_490026AF
0204 00000000_00000000_00000000_00000000
0205 00000000_00000000_00000000_19114000
0206 00000000_00000000_00000000_00000000
0207 00000000_00000000_00000000_41101BDF
0208 00000000_00000000_00000000_00000000
0209 00000000_00000000_00000000_42900000
0210 00000000_00000000_00000000_00000000
0211 00000000_00000000_00000000_00000000
0212 00000000_00000000_90280800_00000000
0213 00000000_00000000_00000000_00000000
0214 00000000_00000000_4940000F_490048D1
0215 00000000_00000000_00000000_00000000
0216 00000000_00000000_00000000_19114000
0217 00000000_00000000_00000000_00000000
0218 00000000_00000000_00000000_41102CE0
0219 00000000_00000000_00000000_00000000
0220 00000000_00000000_00000000_42900000
0221 00000000_00000000_00000000_00000000
0222 00000000_00000000_00000000_00000000
0223 00000000_00000000_90280C00_00000000
0224 00000000_00000000_00000000_00000000
0225 00000000_00000000_4940000F_49002222
0226 00000000_00000000_00000000_00000000
0227 00000000_00000000_00000000_19114000
0228 00000000_00000000_00000000_00000000
0229 00000000_00000000_00000000_41101111
0230 00000000_00000000_00000000_00000000
0231 00000000_00000000_00000000_42900000
0232 00000000_00000000_00000000_00000000
0233 00000000_00000000_00000000_00000000
0234 00000000_00000000_90281000_00000000
0235 00000000_00000000_00000000_00000000
0236 00000000_00000000_4940000F_49004444
0431 00000000_00000000_00000000_00000000
0432 00000000_00000000_00000000_41900000
0433 00000000_00000000_00000000_00000000
0434 00000000_00000000_00000000_00000000
0435 00000000_00000000_00000000_00000000
0436 098000F0_00000000_00000000_00000000
0437 050000D8_00000000_00000000_00000000
0438 00000000_00000000_00000000_00000000
0439 09080000_00000000_00000000_00000000
0440 050000E0_00000000_00000000_40880004
0441 00000000_00000000_00000000_00000000
0442 09080000_00000000_00000000_00000000
0443 050000E4_00000000_00000000_40880004
0444 00000000_00000000_00000000_00000000
0445 09080000_00000000_00000000_00000000
0446 00000000_00000000_00000000_40880004
0447 00000000_00000000_00000000_00000000
0448 00000000_00000000_00000000_408BFFFC
0449 00000000_00000000_00000000_00000000
0450 05080000_00000000_00000000_00000000
0451 00000000_00000000_00000000_408BFFFC
0452 09000064_00000000_00000000_00000000
0453 05080000_00000000_00000000_00000000
0454 00000000_00000000_00000000_408BFFFC
0455 09000060_00000000_00000000_00000000
0456 05080000_00000000_00000000_00000000
0457 00000000_00000000_00000000_00000000
0458 0900005C_00000000_00000000_00000000
0459 08480000_00000000_00000000_00000000
0460 00000000_447FFEC2_00000000_40880004
0461 00000000_00000000_00000000_00000000
0462 00000000_00000000_00000000_408BFFFC
0463 00000000_00000000_00000000_00000000
0464 04480000_00000000_00000000_00000000
0465 05000058_00000000_00000000_00000000
0466 058000F0_00000000_00000000_00000000
0467 00000000_00000000_00000000_41D00000
0468 00000000_00000000_00000000_00000000
0469 00000000_00000000_00000000_00000000
0470 00000000_00000000_00000000_00000000
0471 098000F0_00000000_00000000_00000000
0472 09C000F4_00000000_00000000_00000000
0473 050000EC_00000000_00000000_00000000
0474 058000F0_00000000_00000000_00000000
0475 05C000F4_00000000_00000000_42500000
0476 00000000_00000000_00000000_00000000
0477 00000000_00000000_00000000_00000000
0478 00000000_00000000_00000000_4A000000
0479 00000000_00000000_00000000_00000000
0480 00000000_1240000D_00000000_00000000
0481 05180000_00000000_00000000_00000000
0482 05580004_00000000_00000000_00000000
0483 00000000_00000000_00000000_41980008
0484 00000000_00000000_85114000_00000000
0485 00000000_00000000_00000000_42200001
0486 00000000_00000000_85550400_00000000
0487 00000000_00000000_00000000_00000000
0488 00000000_00000000_85114800_00000000
0489 00000000_00000000_00000000_00000000
0490 00000000_00000000_85550C00_00000000
0491 091C0000_00000000_00000000_00000000
0492 095C0004_0E63FFF5_00000000_41DC0008
0493 00000000_00000000_00000000_00000000
0494 00000000_4000005F_00000000_00000000
0495 00000000_00000000_00000000_00000000
0496 00000000_00000000_00000000_00000000
0497 050000D4_00000000_00000000_00000000
0498 00000000_00000000_00000000_00000000
0499 09080000_00000000_00000000_00000000
0500 050000E0_00000000_00000000_40880004
0501 00000000_00000000_00000000_00000000
0502 09080000_00000000_00000000_00000000
0503 050000E4_00000000_00000000_40880004
0504 00000000_00000000_00000000_00000000
0505 09080000_00000000_00000000_00000000
0506 00000000_00000000_00000000_40880004
0507 00000000_00000000_00000000_00000000
0508 00000000_00000000_00000000_408BFFFC
0509 00000000_00000000_00000000_00000000
0510 05080000_00000000_00000000_00000000
0511 00000000_00000000_00000000_408BFFFC
0512 09000064_00000000_00000000_00000000
0513 05080000_00000000_00000000_00000000
0514 00000000_00000000_00000000_408BFFFC
0515 09000060_00000000_00000000_00000000
0516 05080000_00000000_00000000_00000000
0517 00000000_00000000_00000000_00000000
0518 0900005C_00000000_00000000_00000000
0519 08480000_00000000_00000000_00000000
0520 00000000_447FFE86_00000000_40880004
0521 00000000_00000000_00000000_00000000
0522 00000000_00000000_00000000_408BFFFC
0523 00000000_00000000_00000000_00000000
0524 04480000_00000000_00000000_00000000
0525 05000058_00000000_00000000_00000000
0526 00000000_00000000_00000000_00000000
0527 00000000_00000000_00000000_41900000
0528 00000000_00000000_00000000_00000000
0529 00000000_00000000_00000000_00000000
0530 00000000_00000000_00000000_00000000
0531 098000F0_00000000_00000000_00000000
0532 050000D8_00000000_00000000_00000000
0533 00000000_00000000_00000000_00000000
0534 09080000_00000000_00000000_00000000
0535 050000E0_00000000_00000000_40880004
0536 00000000_00000000_00000000_00000000
0537 09080000_00000000_00000000_00000000
0538 050000E4_00000000_00000000_40880004
0539 00000000_00000000_00000000_00000000
0540 09080000_00000000_00000000_00000000
0541 00000000_00000000_00000000_40880004
0542 00000000_00000000_00000000_00000000
0543 00000000_00000000_00000000_408BFFFC
0544 00000000_00000000_00000000_00000000
0545 05080000_00000000_00000000_00000000
0740 00000000_00000000_00000000_00000000
0741 05080000_00000000_00000000_00000000
0742 00000000_00000000_00000000_408BFFFC
0743 09000064_00000000_00000000_00000000
0744 05080000_00000000_00000000_00000000
0745 00000000_00000000_00000000_408BFFFC
0746 09000060_00000000_00000000_00000000
0747 05080000_00000000_00000000_00000000
0748 00000000_00000000_00000000_00000000
0749 0900005C_00000000_00000000_00000000
0750 08480000_00000000_00000000_00000000
0751 00000000_447FFD9F_00000000_40880004
0752 00000000_00000000_00000000_00000000
0753 00000000_00000000_00000000_408BFFFC
0754 00000000_00000000_00000000_00000000
0755 04480000_00000000_00000000_00000000
0756 05000058_00000000_00000000_00000000
0757 00000000_00000000_00000000_00000000
0758 00000000_00000000_00000000_41900000
0759 00000000_00000000_00000000_00000000
0760 00000000_00000000_00000000_00000000
0761 00000000_00000000_00000000_00000000
0762 09800144_00000000_00000000_00000000
0763 0500012C_00000000_00000000_00000000
0764 00000000_00000000_00000000_00000000
0765 09080000_00000000_00000000_00000000
0766 05000134_00000000_00000000_40880004
0767 00000000_00000000_00000000_00000000
0768 09080000_00000000_00000000_00000000
0769 05000138_00000000_00000000_40880004
0770 00000000_00000000_00000000_00000000
0771 09080000_00000000_00000000_00000000
0772 00000000_00000000_00000000_40880004
0773 00000000_00000000_00000000_00000000
0774 00000000_00000000_00000000_408BFFFC
0775 00000000_00000000_00000000_00000000
0776 05080000_00000000_00000000_00000000
0777 00000000_00000000_00000000_408BFFFC
0778 09000064_00000000_00000000_00000000
0779 05080000_00000000_00000000_00000000
0780 00000000_00000000_00000000_408BFFFC
0781 09000060_00000000_00000000_00000000
0782 05080000_00000000_00000000_00000000
0783 00000000_00000000_00000000_00000000
0784 0900005C_00000000_00000000_00000000
0785 08480000_00000000_00000000_00000000
0786 00000000_447FFD7C_00000000_40880004
0787 00000000_00000000_00000000_00000000
0788 00000000_00000000_00000000_408BFFFC
0789 00000000_00000000_00000000_00000000
0790 04480000_00000000_00000000_00000000
0791 05000058_00000000_00000000_00000000
0792 05800144_00000000_00000000_00000000
0793 00000000_00000000_00000000_41D00000
0794 00000000_00000000_00000000_00000000
0795 00000000_00000000_00000000_00000000
0796 00000000_00000000_00000000_00000000
0797 09800144_00000000_00000000_00000000
0798 09C00148_00000000_00000000_00000000
0799 05000140_00000000_00000000_00000000
0800 05800144_00000000_00000000_00000000
0801 05C00148_00000000_00000000_42500000
0802 00000000_00000000_00000000_00000000
0803 00000000_00000000_00000000_00000000
0804 00000000_00000000_00000000_4A000000
0805 00000000_00000000_00000000_00000000
0806 00000000_1240000D_00000000_00000000
0807 05180004_00000000_00000000_00000000
0808 05580000_00000000_00000000_00000000
0809 00000000_00000000_00000000_41980008
0810 00000000_00000000_85114C00_00000000
0811 00000000_00000000_00000000_42200001
0812 00000000_00000000_85550800_00000000
0813 00000000_00000000_00000000_00000000
0814 00000000_00000000_85114400_00000000
0815 00000000_00000000_00000000_00000000
0816 00000000_00000000_85550000_00000000
0817 091C0004_00000000_00000000_00000000
0818 095C0000_0E63FFF5_00000000_41DC0008
0819 00000000_00000000_00000000_00000000
0820 00000000_4000005F_00000000_00000000
0821 00000000_00000000_00000000_00000000
0822 00000000_00000000_00000000_00000000
0823 05000128_00000000_00000000_00000000
0824 00000000_00000000_00000000_00000000
0825 09080000_00000000_00000000_00000000
0826 05000134_00000000_00000000_40880004
0827 00000000_00000000_00000000_00000000
0828 09080000_00000000_00000000_00000000
0829 05000138_00000000_00000000_40880004
0830 00000000_00000000_00000000_00000000
0831 09080000_00000000_00000000_00000000
0832 00000000_00000000_00000000_40880004
0833 00000000_00000000_00000000_00000000
0834 00000000_00000000_00000000_408BFFFC
0835 00000000_00000000_00000000_00000000
0836 05080000_00000000_00000000_00000000
0837 00000000_00000000_00000000_408BFFFC
0838 09000064_00000000_00000000_00000000
0839 05080000_00000000_00000000_00000000
0840 00000000_00000000_00000000_408BFFFC
0841 09000060_00000000_00000000_00000000
0842 05080000_00000000_00000000_00000000
0843 00000000_00000000_00000000_00000000
0844 0900005C_00000000_00000000_00000000
0845 08480000_00000000_00000000_00000000
0846 00000000_447FFD40_00000000_40880004
0847 00000000_00000000_00000000_00000000
0848 00000000_00000000_00000000_408BFFFC
0849 00000000_00000000_00000000_00000000
0850 04480000_00000000_00000000_00000000
0851 05000058_00000000_00000000_00000000
0852 00000000_00000000_00000000_00000000
0853 00000000_00000000_00000000_41900000
0854 00000000_00000000_00000000_00000000
```

```
0237 00000000_00000000_00000000_00000000
0238 00000000_00000000_00000000_19114000
0239 00000000_00000000_00000000_00000000
0240 00000000_00000000_00000000_41102222
0241 00000000_00000000_00000000_00000000
0242 00000000_00000000_00000000_42900000
0243 00000000_00000000_00000000_00000000
0244 00000000_00000000_00000000_00000000
0245 00000000_00000000_90281400_00000000
0246 00000000_00000000_00000000_00000000
0247 00000000_00000000_4940000F_49006666
0248 00000000_00000000_00000000_00000000
0249 00000000_00000000_00000000_19114000
0250 00000000_00000000_00000000_00000000
0251 00000000_00000000_00000000_41103333
0252 00000000_00000000_00000000_00000000
0253 00000000_00000000_00000000_42900000
0254 00000000_00000000_00000000_00000000
0255 00000000_00000000_00000000_00000000
0256 00000000_00000000_90281800_00000000
0257 00000000_00000000_00000000_00000000
0258 00000000_00000000_4940000F_49008888
0259 00000000_00000000_00000000_00000000
0260 00000000_00000000_00000000_19114000
0261 00000000_00000000_00000000_00000000
0262 00000000_00000000_00000000_41104444
0263 00000000_00000000_00000000_00000000
0264 00000000_00000000_00000000_42900000
0265 00000000_00000000_00000000_00000000
0266 00000000_00000000_00000000_00000000
0267 00000000_00000000_90281C00_00000000
0268 00000000_00000000_94000000_00000000
0269 00000000_14040000_00000000_00000000
0270 00000000_00000000_00000000_00000000
0271 080000D0_00000000_00000000_00000000
0272 00000000_00000000_00000000_00000000
0273 050000DC_00000000_00000000_00000000
0274 054000E0_00000000_00000000_00000000
0275 00000000_00000000_00000000_00000000
0276 00000000_00000000_00000000_05114000
0277 00000000_00000000_00000000_49400001
0278 00000000_00000000_00000000_00000000
0279 00000000_00000000_00000000_09114000
0280 054000E0_00000000_00000000_00000000
0281 00000000_00000000_00000000_00000000
0282 00000000_04140282_00000000_00000000
0283 00000000_00000000_00000000_00000000
0284 00000000_00000000_06140000_05D00000
0285 00000000_00000000_00000000_00000000
0286 00000000_101C0004_00000000_00000000
0287 00000000_00000000_00000000_00000000
0288 00000000_00000000_00000000_09C1C000
0289 00000000_00000000_00000000_00000000
0290 00000000_10200004_00000000_00000000
0291 00000000_00000000_00000000_00000000
0292 00000000_00000000_00000000_0A020000
0293 00000000_00000000_00000000_00000000
0294 00000000_00000000_4A80001F_4AC00001
0295 00000000_00000000_49800000_4A400000
0296 00000000_00000000_00000000_1BAE8000
0297 00000000_00000000_00000000_4A800020
0298 00000000_00000000_00000000_00000000
0299 00000000_00000000_1A66C000_271C0000
0300 00000000_00000000_199AC000_19DEC000
0301 00000000_00000000_00000000_12670000
0302 00000000_00000000_00000000_17238000
0303 00000000_00000000_00000000_17678000
0304 00000000_00000000_00000000_00000000
0305 00000000_0F340004_00000000_00000000
0306 00000000_00000000_00000000_00000000
0307 00000000_00000000_41980001_0A660000
0308 00000000_00000000_00000000_00000000
0546 00000000_00000000_00000000_408BFFFC
0547 09000064_00000000_00000000_00000000
0548 05080000_00000000_00000000_00000000
0549 00000000_00000000_00000000_408BFFFC
0550 09000060_00000000_00000000_00000000
0551 05080000_00000000_00000000_00000000
0552 00000000_00000000_00000000_00000000
0553 0900005C_00000000_00000000_00000000
0554 08480000_00000000_00000000_00000000
0555 00000000_447FFE63_00000000_40880004
0556 00000000_00000000_00000000_00000000
0557 00000000_00000000_00000000_408BFFFC
0558 00000000_00000000_00000000_00000000
0559 04480000_00000000_00000000_00000000
0560 05000058_00000000_00000000_00000000
0561 058000F0_00000000_00000000_00000000
0562 00000000_00000000_00000000_41D00000
0563 00000000_00000000_00000000_00000000
0564 00000000_00000000_00000000_00000000
0565 00000000_00000000_00000000_00000000
0566 098000F0_00000000_00000000_00000000
0567 09C000F4_00000000_00000000_00000000
0568 050000EC_00000000_00000000_00000000
0569 058000F0_00000000_00000000_00000000
0570 05C000F4_00000000_00000000_42500000
0571 00000000_00000000_00000000_00000000
0572 00000000_00000000_00000000_00000000
0573 00000000_00000000_00000000_4A000000
0574 00000000_00000000_00000000_00000000
0575 00000000_1240000D_00000000_00000000
0576 05180000_00000000_00000000_00000000
0577 05580004_00000000_00000000_00000000
0578 00000000_00000000_00000000_41980008
0579 00000000_00000000_85115000_00000000
0580 00000000_00000000_00000000_42200001
0581 00000000_00000000_85551400_00000000
0582 00000000_00000000_00000000_00000000
0583 00000000_00000000_85115800_00000000
0584 00000000_00000000_00000000_00000000
0585 00000000_00000000_85551C00_00000000
0586 091C0000_00000000_00000000_00000000
0587 095C0004_0E63FFF5_00000000_41DC0008
0588 00000000_00000000_00000000_00000000
0589 050000E4_00000000_00000000_00000000
0590 00000000_00000000_00000000_00000000
0591 00000000_00000000_00000000_41100001
0592 00000000_00000000_00000000_00000000
0593 090000E4_403FFEF3_00000000_00000000
0594 00000000_00000000_00000000_00000000
0595 00000000_14040000_00000000_00000000
0596 00000000_00000000_00000000_00000000
0597 08000124_00000000_00000000_00000000
0598 00000000_00000000_00000000_00000000
0599 05000130_00000000_00000000_00000000
0600 05400134_00000000_00000000_00000000
0601 00000000_00000000_00000000_00000000
0602 00000000_00000000_00000000_05114000
0603 00000000_00000000_00000000_49400001
0604 00000000_00000000_00000000_00000000
0605 00000000_00000000_00000000_09114000
0606 05400134_00000000_00000000_00000000
0607 00000000_00000000_00000000_00000000
0608 00000000_0414013C_00000000_00000000
0609 00000000_00000000_00000000_00000000
0610 00000000_00000000_06140000_05D00000
0611 00000000_00000000_00000000_00000000
0612 00000000_101C0004_00000000_00000000
0613 00000000_00000000_00000000_00000000
0614 00000000_00000000_00000000_09C1C000
0615 00000000_00000000_00000000_00000000
0616 00000000_10200004_00000000_00000000
0617 00000000_00000000_00000000_00000000
0855 00000000_00000000_00000000_00000000
0856 00000000_00000000_00000000_00000000
0857 09800144_00000000_00000000_00000000
0858 0500012C_00000000_00000000_00000000
0859 00000000_00000000_00000000_00000000
0860 09080000_00000000_00000000_00000000
0861 05000134_00000000_00000000_40880004
0862 00000000_00000000_00000000_00000000
0863 09080000_00000000_00000000_00000000
0864 05000138_00000000_00000000_40880004
0865 00000000_00000000_00000000_00000000
0866 09080000_00000000_00000000_00000000
0867 00000000_00000000_00000000_40880004
0868 00000000_00000000_00000000_00000000
0869 00000000_00000000_00000000_408BFFFC
0870 00000000_00000000_00000000_00000000
0871 05080000_00000000_00000000_00000000
0872 00000000_00000000_00000000_408BFFFC
0873 09000064_00000000_00000000_00000000
0874 05080000_00000000_00000000_00000000
0875 00000000_00000000_00000000_408BFFFC
0876 09000060_00000000_00000000_00000000
0877 05080000_00000000_00000000_00000000
0878 00000000_00000000_00000000_00000000
0879 0900005C_00000000_00000000_00000000
0880 08480000_00000000_00000000_00000000
0881 00000000_447FFD1D_00000000_40880004
0882 00000000_00000000_00000000_00000000
0883 00000000_00000000_00000000_408BFFFC
0884 00000000_00000000_00000000_00000000
0885 04480000_00000000_00000000_00000000
0886 05000058_00000000_00000000_00000000
0887 05800144_00000000_00000000_00000000
0888 00000000_00000000_00000000_41D00000
0889 00000000_00000000_00000000_00000000
0890 00000000_00000000_00000000_00000000
0891 00000000_00000000_00000000_00000000
0892 09800144_00000000_00000000_00000000
0893 09C00148_00000000_00000000_00000000
0894 05000140_00000000_00000000_00000000
0895 05800144_00000000_00000000_00000000
0896 05C00148_00000000_00000000_42500000
0897 00000000_00000000_00000000_00000000
0898 00000000_00000000_00000000_00000000
0899 00000000_00000000_00000000_4A000000
0900 00000000_00000000_00000000_00000000
0901 00000000_1240000D_00000000_00000000
0902 05180004_00000000_00000000_00000000
0903 05580000_00000000_00000000_00000000
0904 00000000_00000000_00000000_41980008
0905 00000000_00000000_85115C00_00000000
0906 00000000_00000000_00000000_42200001
0907 00000000_00000000_85551800_00000000
0908 00000000_00000000_00000000_00000000
0909 00000000_00000000_85115400_00000000
0910 00000000_00000000_00000000_00000000
0911 00000000_00000000_85551000_00000000
0912 091C0004_00000000_00000000_00000000
0913 095C0000_0E63FFF5_00000000_41DC0008
0914 00000000_00000000_00000000_00000000
0915 05000138_00000000_00000000_00000000
0916 00000000_00000000_00000000_00000000
0917 00000000_00000000_00000000_41100001
0918 00000000_00000000_00000000_00000000
0919 09000138_403FFEF3_00000000_00000000
0920 00000000_00000000_00000000_00000000
0921 00000000_14040000_00000000_00000000
0922 00000000_00000000_00000000_00000000
0923 00000000_40000000_00000000_00000000
0924 00000000_40000000_00000000_00000000
```

## *F-E. Traza de ejecución*

```
# Traza de ejecución (enunciado CE1108 §8.4)
# Cada línea: [bundle N · línea L del .cls] evento.
# No se muestran las escrituras fuera de las variables del listado:
# la pila de llamadas (rx), las casillas ocultas del S y los temporales
# que se salvan en las llamadas.
# No se muestran los saltos hacia las rutinas internas
# (__ccom_arith_*, __ccom_array_*, .L_crypto_*); sí el salto a
# .L_arith_error. Los saltos condicionales no tomados tampoco.
[bundle 0004 · línea 60] n1 = 0
[bundle 0007 · línea 61] n2 = 0
[bundle 0019 · línea 62] auth
[bundle 0021 · línea 63] llamada → .L_func_preparar_boveda (bundle 178)
[bundle 0188 · línea 21] kv_write k=0 w=0
[bundle 0201 · línea 22] kv_write k=0 w=1
[bundle 0212 · línea 23] kv_write k=0 w=2
[bundle 0223 · línea 24] kv_write k=0 w=3
[bundle 0234 · línea 25] kv_write k=1 w=0
[bundle 0245 · línea 26] kv_write k=1 w=1
[bundle 0256 · línea 27] kv_write k=1 w=2
[bundle 0267 · línea 28] kv_write k=1 w=3
[bundle 0268 · línea 29] kv_lock
[bundle 0269 · línea 30] retorno → bundle 22
[bundle 0044 · línea 64] cifrar_archivo.tam = 16
[bundle 0047 · línea 64] cifrar_archivo.total = 128
[bundle 0050 · línea 64] cifrar_archivo.hacia = 8192
[bundle 0053 · línea 64] cifrar_archivo.desde = 4096
[bundle 0055 · línea 64] llamada → .L_func_cifrar_archivo (bundle 271)
[bundle 0271 · línea 33] cifrar_archivo.n = 0  (prólogo)
[bundle 0319 · línea 34] cifrar_archivo.n = 8
[bundle 0321 · línea 35] cifrar_archivo.i = 0
[bundle 0344 · línea 36] bloques_de.i = 0
[bundle 0347 · línea 36] bloques_de.tam = 16
[bundle 0350 · línea 36] bloques_de.total = 128
[bundle 0352 · línea 36] llamada → .L_func_bloques_de (bundle 104)
[bundle 0104 · línea 5] bloques_de.b = 0  (prólogo)
[bundle 0124 · línea 6] resto = 128
```

```
[bundle 0130 · línea 7] beq → .L_if_next_0 (bundle 136)
[bundle 0138 · línea 10] bloques_de.b = 16
[bundle 0140 · línea 12] retorno → bundle 353
[bundle 0359 · línea 36] cifrar_archivo.b = 16
[bundle 0417 · línea 38] direccion.i = 0
[bundle 0420 · línea 38] direccion.tam = 16
[bundle 0423 · línea 38] base = 4096
[bundle 0425 · línea 38] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 4096
[bundle 0176 · línea 17] retorno → bundle 426
[bundle 0452 · línea 38] direccion.i = 0
[bundle 0455 · línea 38] direccion.tam = 16
[bundle 0458 · línea 38] base = 8192
[bundle 0460 · línea 38] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 8192
[bundle 0176 · línea 17] retorno → bundle 461
[bundle 0493 · línea 38] cifrar: 16 bloques de 0x1000 a 0x2000
[bundle 0494 · línea 37] jef → .L_if_end_6 (bundle 589)
[bundle 0593 · línea 35] cifrar_archivo.i = 1
[bundle 0593 · línea 35] jef → .L_for_start_2 (bundle 324)
[bundle 0344 · línea 36] bloques_de.i = 1
[bundle 0347 · línea 36] bloques_de.tam = 16
[bundle 0350 · línea 36] bloques_de.total = 128
[bundle 0352 · línea 36] llamada → .L_func_bloques_de (bundle 104)
[bundle 0104 · línea 5] bloques_de.b = 0  (prólogo)
[bundle 0124 · línea 6] resto = 112
[bundle 0130 · línea 7] beq → .L_if_next_0 (bundle 136)
[bundle 0138 · línea 10] bloques_de.b = 16
[bundle 0140 · línea 12] retorno → bundle 353
[bundle 0359 · línea 36] cifrar_archivo.b = 16
[bundle 0399 · línea 37] beq → .L_if_next_5 (bundle 496)
[bundle 0512 · línea 40] direccion.i = 1
[bundle 0515 · línea 40] direccion.tam = 16
[bundle 0518 · línea 40] base = 4096
[bundle 0520 · línea 40] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 4224
[bundle 0176 · línea 17] retorno → bundle 521
[bundle 0547 · línea 40] direccion.i = 1
[bundle 0550 · línea 40] direccion.tam = 16
[bundle 0553 · línea 40] base = 8192
[bundle 0555 · línea 40] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 8320
[bundle 0176 · línea 17] retorno → bundle 556
[bundle 0588 · línea 40] cifrar: 16 bloques de 0x1080 a 0x2080
[bundle 0593 · línea 35] cifrar_archivo.i = 2
[bundle 0593 · línea 35] jef → .L_for_start_2 (bundle 324)
[bundle 0344 · línea 36] bloques_de.i = 2
[bundle 0347 · línea 36] bloques_de.tam = 16
[bundle 0350 · línea 36] bloques_de.total = 128
[bundle 0352 · línea 36] llamada → .L_func_bloques_de (bundle 104)
[bundle 0104 · línea 5] bloques_de.b = 0  (prólogo)
[bundle 0124 · línea 6] resto = 96
[bundle 0130 · línea 7] beq → .L_if_next_0 (bundle 136)
[bundle 0138 · línea 10] bloques_de.b = 16
[bundle 0140 · línea 12] retorno → bundle 353
[bundle 0359 · línea 36] cifrar_archivo.b = 16
[bundle 0417 · línea 38] direccion.i = 2
[bundle 0420 · línea 38] direccion.tam = 16
[bundle 0423 · línea 38] base = 4096
[bundle 0425 · línea 38] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 4352
[bundle 0176 · línea 17] retorno → bundle 426
[bundle 0452 · línea 38] direccion.i = 2
[bundle 0455 · línea 38] direccion.tam = 16
[bundle 0458 · línea 38] base = 8192
[bundle 0460 · línea 38] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 8448
[bundle 0176 · línea 17] retorno → bundle 461
[bundle 0493 · línea 38] cifrar: 16 bloques de 0x1100 a 0x2100
[bundle 0494 · línea 37] jef → .L_if_end_6 (bundle 589)
[bundle 0593 · línea 35] cifrar_archivo.i = 3
[bundle 0593 · línea 35] jef → .L_for_start_2 (bundle 324)
[bundle 0344 · línea 36] bloques_de.i = 3
[bundle 0347 · línea 36] bloques_de.tam = 16
[bundle 0350 · línea 36] bloques_de.total = 128
[bundle 0352 · línea 36] llamada → .L_func_bloques_de (bundle 104)
[bundle 0104 · línea 5] bloques_de.b = 0  (prólogo)
[bundle 0124 · línea 6] resto = 80
[bundle 0130 · línea 7] beq → .L_if_next_0 (bundle 136)
[bundle 0138 · línea 10] bloques_de.b = 16
[bundle 0140 · línea 12] retorno → bundle 353
[bundle 0359 · línea 36] cifrar_archivo.b = 16
[bundle 0399 · línea 37] beq → .L_if_next_5 (bundle 496)
[bundle 0512 · línea 40] direccion.i = 3
[bundle 0515 · línea 40] direccion.tam = 16
[bundle 0518 · línea 40] base = 4096
[bundle 0520 · línea 40] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 4480
[bundle 0176 · línea 17] retorno → bundle 521
[bundle 0547 · línea 40] direccion.i = 3
[bundle 0550 · línea 40] direccion.tam = 16
[bundle 0553 · línea 40] base = 8192
[bundle 0555 · línea 40] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 8576
[bundle 0176 · línea 17] retorno → bundle 556
[bundle 0588 · línea 40] cifrar: 16 bloques de 0x1180 a 0x2180
[bundle 0593 · línea 35] cifrar_archivo.i = 4
[bundle 0593 · línea 35] jef → .L_for_start_2 (bundle 324)
[bundle 0344 · línea 36] bloques_de.i = 4
[bundle 0347 · línea 36] bloques_de.tam = 16
[bundle 0350 · línea 36] bloques_de.total = 128
[bundle 0352 · línea 36] llamada → .L_func_bloques_de (bundle 104)
[bundle 0104 · línea 5] bloques_de.b = 0  (prólogo)
[bundle 0124 · línea 6] resto = 64
[bundle 0130 · línea 7] beq → .L_if_next_0 (bundle 136)
[bundle 0138 · línea 10] bloques_de.b = 16
[bundle 0140 · línea 12] retorno → bundle 353
```

```
[bundle 0359 · línea 36] cifrar_archivo.b = 16
[bundle 0417 · línea 38] direccion.i = 4
[bundle 0420 · línea 38] direccion.tam = 16
[bundle 0423 · línea 38] base = 4096
[bundle 0425 · línea 38] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 4608
[bundle 0176 · línea 17] retorno → bundle 426
[bundle 0452 · línea 38] direccion.i = 4
[bundle 0455 · línea 38] direccion.tam = 16
[bundle 0458 · línea 38] base = 8192
[bundle 0460 · línea 38] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 8704
[bundle 0176 · línea 17] retorno → bundle 461
[bundle 0493 · línea 38] cifrar: 16 bloques de 0x1200 a 0x2200
[bundle 0494 · línea 37] jef → .L_if_end_6 (bundle 589)
[bundle 0593 · línea 35] cifrar_archivo.i = 5
[bundle 0593 · línea 35] jef → .L_for_start_2 (bundle 324)
[bundle 0344 · línea 36] bloques_de.i = 5
[bundle 0347 · línea 36] bloques_de.tam = 16
[bundle 0350 · línea 36] bloques_de.total = 128
[bundle 0352 · línea 36] llamada → .L_func_bloques_de (bundle 104)
[bundle 0104 · línea 5] bloques_de.b = 0  (prólogo)
[bundle 0124 · línea 6] resto = 48
[bundle 0130 · línea 7] beq → .L_if_next_0 (bundle 136)
[bundle 0138 · línea 10] bloques_de.b = 16
[bundle 0140 · línea 12] retorno → bundle 353
[bundle 0359 · línea 36] cifrar_archivo.b = 16
[bundle 0399 · línea 37] beq → .L_if_next_5 (bundle 496)
[bundle 0512 · línea 40] direccion.i = 5
[bundle 0515 · línea 40] direccion.tam = 16
[bundle 0518 · línea 40] base = 4096
[bundle 0520 · línea 40] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 4736
[bundle 0176 · línea 17] retorno → bundle 521
[bundle 0547 · línea 40] direccion.i = 5
[bundle 0550 · línea 40] direccion.tam = 16
[bundle 0553 · línea 40] base = 8192
[bundle 0555 · línea 40] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 8832
[bundle 0176 · línea 17] retorno → bundle 556
[bundle 0588 · línea 40] cifrar: 16 bloques de 0x1280 a 0x2280
[bundle 0593 · línea 35] cifrar_archivo.i = 6
[bundle 0593 · línea 35] jef → .L_for_start_2 (bundle 324)
[bundle 0344 · línea 36] bloques_de.i = 6
[bundle 0347 · línea 36] bloques_de.tam = 16
[bundle 0350 · línea 36] bloques_de.total = 128
[bundle 0352 · línea 36] llamada → .L_func_bloques_de (bundle 104)
[bundle 0104 · línea 5] bloques_de.b = 0  (prólogo)
[bundle 0124 · línea 6] resto = 32
[bundle 0130 · línea 7] beq → .L_if_next_0 (bundle 136)
[bundle 0138 · línea 10] bloques_de.b = 16
[bundle 0140 · línea 12] retorno → bundle 353
[bundle 0359 · línea 36] cifrar_archivo.b = 16
[bundle 0417 · línea 38] direccion.i = 6
[bundle 0420 · línea 38] direccion.tam = 16
[bundle 0423 · línea 38] base = 4096
[bundle 0425 · línea 38] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 4864
[bundle 0176 · línea 17] retorno → bundle 426
[bundle 0452 · línea 38] direccion.i = 6
[bundle 0455 · línea 38] direccion.tam = 16
[bundle 0458 · línea 38] base = 8192
[bundle 0460 · línea 38] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 8960
[bundle 0176 · línea 17] retorno → bundle 461
[bundle 0493 · línea 38] cifrar: 16 bloques de 0x1300 a 0x2300
[bundle 0494 · línea 37] jef → .L_if_end_6 (bundle 589)
[bundle 0593 · línea 35] cifrar_archivo.i = 7
[bundle 0593 · línea 35] jef → .L_for_start_2 (bundle 324)
[bundle 0344 · línea 36] bloques_de.i = 7
[bundle 0347 · línea 36] bloques_de.tam = 16
[bundle 0350 · línea 36] bloques_de.total = 128
[bundle 0352 · línea 36] llamada → .L_func_bloques_de (bundle 104)
[bundle 0104 · línea 5] bloques_de.b = 0  (prólogo)
[bundle 0124 · línea 6] resto = 16
[bundle 0130 · línea 7] beq → .L_if_next_0 (bundle 136)
[bundle 0138 · línea 10] bloques_de.b = 16
[bundle 0140 · línea 12] retorno → bundle 353
[bundle 0359 · línea 36] cifrar_archivo.b = 16
[bundle 0399 · línea 37] beq → .L_if_next_5 (bundle 496)
[bundle 0512 · línea 40] direccion.i = 7
[bundle 0515 · línea 40] direccion.tam = 16
[bundle 0518 · línea 40] base = 4096
[bundle 0520 · línea 40] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 4992
[bundle 0176 · línea 17] retorno → bundle 521
[bundle 0547 · línea 40] direccion.i = 7
[bundle 0550 · línea 40] direccion.tam = 16
[bundle 0553 · línea 40] base = 8192
[bundle 0555 · línea 40] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 9088
[bundle 0176 · línea 17] retorno → bundle 556
[bundle 0588 · línea 40] cifrar: 16 bloques de 0x1380 a 0x2380
[bundle 0593 · línea 35] cifrar_archivo.i = 8
[bundle 0593 · línea 35] jef → .L_for_start_2 (bundle 324)
[bundle 0327 · línea 35] bge → .L_for_end_4 (bundle 595)
[bundle 0595 · línea 43] retorno → bundle 56
[bundle 0062 · línea 64] n1 = 8
[bundle 0081 · línea 65] descifrar_archivo.tam = 16
[bundle 0084 · línea 65] descifrar_archivo.total = 128
[bundle 0087 · línea 65] descifrar_archivo.hacia = 12288
[bundle 0090 · línea 65] descifrar_archivo.desde = 8192
[bundle 0092 · línea 65] llamada → .L_func_descifrar_archivo (bundle 597)
[bundle 0597 · línea 46] descifrar_archivo.n = 0  (prólogo)
[bundle 0645 · línea 47] descifrar_archivo.n = 8
[bundle 0647 · línea 48] descifrar_archivo.i = 0
[bundle 0670 · línea 49] bloques_de.i = 0
```

```
[bundle 0673 · línea 49] bloques_de.tam = 16
[bundle 0676 · línea 49] bloques_de.total = 128
[bundle 0678 · línea 49] llamada → .L_func_bloques_de (bundle 104)
[bundle 0104 · línea 5] bloques_de.b = 0  (prólogo)
[bundle 0124 · línea 6] resto = 128
[bundle 0130 · línea 7] beq → .L_if_next_0 (bundle 136)
[bundle 0138 · línea 10] bloques_de.b = 16
[bundle 0140 · línea 12] retorno → bundle 679
[bundle 0685 · línea 49] descifrar_archivo.b = 16
[bundle 0743 · línea 51] direccion.i = 0
[bundle 0746 · línea 51] direccion.tam = 16
[bundle 0749 · línea 51] base = 8192
[bundle 0751 · línea 51] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 8192
[bundle 0176 · línea 17] retorno → bundle 752
[bundle 0778 · línea 51] direccion.i = 0
[bundle 0781 · línea 51] direccion.tam = 16
[bundle 0784 · línea 51] base = 12288
[bundle 0786 · línea 51] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 12288
[bundle 0176 · línea 17] retorno → bundle 787
[bundle 0819 · línea 51] descifrar: 16 bloques de 0x2000 a 0x3000
[bundle 0820 · línea 50] jef → .L_if_end_11 (bundle 915)
[bundle 0919 · línea 48] descifrar_archivo.i = 1
[bundle 0919 · línea 48] jef → .L_for_start_7 (bundle 650)
[bundle 0670 · línea 49] bloques_de.i = 1
[bundle 0673 · línea 49] bloques_de.tam = 16
[bundle 0676 · línea 49] bloques_de.total = 128
[bundle 0678 · línea 49] llamada → .L_func_bloques_de (bundle 104)
[bundle 0104 · línea 5] bloques_de.b = 0  (prólogo)
[bundle 0124 · línea 6] resto = 112
[bundle 0130 · línea 7] beq → .L_if_next_0 (bundle 136)
[bundle 0138 · línea 10] bloques_de.b = 16
[bundle 0140 · línea 12] retorno → bundle 679
[bundle 0685 · línea 49] descifrar_archivo.b = 16
[bundle 0725 · línea 50] beq → .L_if_next_10 (bundle 822)
[bundle 0838 · línea 53] direccion.i = 1
[bundle 0841 · línea 53] direccion.tam = 16
[bundle 0844 · línea 53] base = 8192
[bundle 0846 · línea 53] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 8320
[bundle 0176 · línea 17] retorno → bundle 847
[bundle 0873 · línea 53] direccion.i = 1
[bundle 0876 · línea 53] direccion.tam = 16
[bundle 0879 · línea 53] base = 12288
[bundle 0881 · línea 53] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 12416
[bundle 0176 · línea 17] retorno → bundle 882
[bundle 0914 · línea 53] descifrar: 16 bloques de 0x2080 a 0x3080
[bundle 0919 · línea 48] descifrar_archivo.i = 2
[bundle 0919 · línea 48] jef → .L_for_start_7 (bundle 650)
[bundle 0670 · línea 49] bloques_de.i = 2
[bundle 0673 · línea 49] bloques_de.tam = 16
[bundle 0676 · línea 49] bloques_de.total = 128
[bundle 0678 · línea 49] llamada → .L_func_bloques_de (bundle 104)
[bundle 0104 · línea 5] bloques_de.b = 0  (prólogo)
[bundle 0124 · línea 6] resto = 96
[bundle 0130 · línea 7] beq → .L_if_next_0 (bundle 136)
[bundle 0138 · línea 10] bloques_de.b = 16
[bundle 0140 · línea 12] retorno → bundle 679
[bundle 0685 · línea 49] descifrar_archivo.b = 16
[bundle 0743 · línea 51] direccion.i = 2
[bundle 0746 · línea 51] direccion.tam = 16
[bundle 0749 · línea 51] base = 8192
[bundle 0751 · línea 51] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 8448
[bundle 0176 · línea 17] retorno → bundle 752
[bundle 0778 · línea 51] direccion.i = 2
[bundle 0781 · línea 51] direccion.tam = 16
[bundle 0784 · línea 51] base = 12288
[bundle 0786 · línea 51] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 12544
[bundle 0176 · línea 17] retorno → bundle 787
[bundle 0819 · línea 51] descifrar: 16 bloques de 0x2100 a 0x3100
[bundle 0820 · línea 50] jef → .L_if_end_11 (bundle 915)
[bundle 0919 · línea 48] descifrar_archivo.i = 3
[bundle 0919 · línea 48] jef → .L_for_start_7 (bundle 650)
[bundle 0670 · línea 49] bloques_de.i = 3
[bundle 0673 · línea 49] bloques_de.tam = 16
[bundle 0676 · línea 49] bloques_de.total = 128
[bundle 0678 · línea 49] llamada → .L_func_bloques_de (bundle 104)
[bundle 0104 · línea 5] bloques_de.b = 0  (prólogo)
[bundle 0124 · línea 6] resto = 80
[bundle 0130 · línea 7] beq → .L_if_next_0 (bundle 136)
[bundle 0138 · línea 10] bloques_de.b = 16
[bundle 0140 · línea 12] retorno → bundle 679
[bundle 0685 · línea 49] descifrar_archivo.b = 16
[bundle 0725 · línea 50] beq → .L_if_next_10 (bundle 822)
[bundle 0838 · línea 53] direccion.i = 3
[bundle 0841 · línea 53] direccion.tam = 16
[bundle 0844 · línea 53] base = 8192
[bundle 0846 · línea 53] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 8576
[bundle 0176 · línea 17] retorno → bundle 847
[bundle 0873 · línea 53] direccion.i = 3
[bundle 0876 · línea 53] direccion.tam = 16
[bundle 0879 · línea 53] base = 12288
[bundle 0881 · línea 53] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 12672
[bundle 0176 · línea 17] retorno → bundle 882
[bundle 0914 · línea 53] descifrar: 16 bloques de 0x2180 a 0x3180
[bundle 0919 · línea 48] descifrar_archivo.i = 4
[bundle 0919 · línea 48] jef → .L_for_start_7 (bundle 650)
[bundle 0670 · línea 49] bloques_de.i = 4
[bundle 0673 · línea 49] bloques_de.tam = 16
[bundle 0676 · línea 49] bloques_de.total = 128
[bundle 0678 · línea 49] llamada → .L_func_bloques_de (bundle 104)
```

```
[bundle 0104 · línea 5] bloques_de.b = 0  (prólogo)
[bundle 0124 · línea 6] resto = 64
[bundle 0130 · línea 7] beq → .L_if_next_0 (bundle 136)
[bundle 0138 · línea 10] bloques_de.b = 16
[bundle 0140 · línea 12] retorno → bundle 679
[bundle 0685 · línea 49] descifrar_archivo.b = 16
[bundle 0743 · línea 51] direccion.i = 4
[bundle 0746 · línea 51] direccion.tam = 16
[bundle 0749 · línea 51] base = 8192
[bundle 0751 · línea 51] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 8704
[bundle 0176 · línea 17] retorno → bundle 752
[bundle 0778 · línea 51] direccion.i = 4
[bundle 0781 · línea 51] direccion.tam = 16
[bundle 0784 · línea 51] base = 12288
[bundle 0786 · línea 51] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 12800
[bundle 0176 · línea 17] retorno → bundle 787
[bundle 0819 · línea 51] descifrar: 16 bloques de 0x2200 a 0x3200
[bundle 0820 · línea 50] jef → .L_if_end_11 (bundle 915)
[bundle 0919 · línea 48] descifrar_archivo.i = 5
[bundle 0919 · línea 48] jef → .L_for_start_7 (bundle 650)
[bundle 0670 · línea 49] bloques_de.i = 5
[bundle 0673 · línea 49] bloques_de.tam = 16
[bundle 0676 · línea 49] bloques_de.total = 128
[bundle 0678 · línea 49] llamada → .L_func_bloques_de (bundle 104)
[bundle 0104 · línea 5] bloques_de.b = 0  (prólogo)
[bundle 0124 · línea 6] resto = 48
[bundle 0130 · línea 7] beq → .L_if_next_0 (bundle 136)
[bundle 0138 · línea 10] bloques_de.b = 16
[bundle 0140 · línea 12] retorno → bundle 679
[bundle 0685 · línea 49] descifrar_archivo.b = 16
[bundle 0725 · línea 50] beq → .L_if_next_10 (bundle 822)
[bundle 0838 · línea 53] direccion.i = 5
[bundle 0841 · línea 53] direccion.tam = 16
[bundle 0844 · línea 53] base = 8192
[bundle 0846 · línea 53] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 8832
[bundle 0176 · línea 17] retorno → bundle 847
[bundle 0873 · línea 53] direccion.i = 5
[bundle 0876 · línea 53] direccion.tam = 16
[bundle 0879 · línea 53] base = 12288
[bundle 0881 · línea 53] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 12928
[bundle 0176 · línea 17] retorno → bundle 882
[bundle 0914 · línea 53] descifrar: 16 bloques de 0x2280 a 0x3280
[bundle 0919 · línea 48] descifrar_archivo.i = 6
[bundle 0919 · línea 48] jef → .L_for_start_7 (bundle 650)
[bundle 0670 · línea 49] bloques_de.i = 6
[bundle 0673 · línea 49] bloques_de.tam = 16
[bundle 0676 · línea 49] bloques_de.total = 128
[bundle 0678 · línea 49] llamada → .L_func_bloques_de (bundle 104)
[bundle 0104 · línea 5] bloques_de.b = 0  (prólogo)
[bundle 0124 · línea 6] resto = 32
[bundle 0130 · línea 7] beq → .L_if_next_0 (bundle 136)
[bundle 0138 · línea 10] bloques_de.b = 16
[bundle 0140 · línea 12] retorno → bundle 679
[bundle 0685 · línea 49] descifrar_archivo.b = 16
[bundle 0743 · línea 51] direccion.i = 6
[bundle 0746 · línea 51] direccion.tam = 16
[bundle 0749 · línea 51] base = 8192
[bundle 0751 · línea 51] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 8960
[bundle 0176 · línea 17] retorno → bundle 752
[bundle 0778 · línea 51] direccion.i = 6
[bundle 0781 · línea 51] direccion.tam = 16
[bundle 0784 · línea 51] base = 12288
[bundle 0786 · línea 51] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 13056
[bundle 0176 · línea 17] retorno → bundle 787
[bundle 0819 · línea 51] descifrar: 16 bloques de 0x2300 a 0x3300
[bundle 0820 · línea 50] jef → .L_if_end_11 (bundle 915)
[bundle 0919 · línea 48] descifrar_archivo.i = 7
[bundle 0919 · línea 48] jef → .L_for_start_7 (bundle 650)
[bundle 0670 · línea 49] bloques_de.i = 7
[bundle 0673 · línea 49] bloques_de.tam = 16
[bundle 0676 · línea 49] bloques_de.total = 128
[bundle 0678 · línea 49] llamada → .L_func_bloques_de (bundle 104)
[bundle 0104 · línea 5] bloques_de.b = 0  (prólogo)
[bundle 0124 · línea 6] resto = 16
[bundle 0130 · línea 7] beq → .L_if_next_0 (bundle 136)
[bundle 0138 · línea 10] bloques_de.b = 16
[bundle 0140 · línea 12] retorno → bundle 679
[bundle 0685 · línea 49] descifrar_archivo.b = 16
[bundle 0725 · línea 50] beq → .L_if_next_10 (bundle 822)
[bundle 0838 · línea 53] direccion.i = 7
[bundle 0841 · línea 53] direccion.tam = 16
[bundle 0844 · línea 53] base = 8192
[bundle 0846 · línea 53] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 9088
[bundle 0176 · línea 17] retorno → bundle 847
[bundle 0873 · línea 53] direccion.i = 7
[bundle 0876 · línea 53] direccion.tam = 16
[bundle 0879 · línea 53] base = 12288
[bundle 0881 · línea 53] llamada → .L_func_direccion (bundle 142)
[bundle 0142 · línea 15] d = 0  (prólogo)
[bundle 0176 · línea 16] d = 13184
[bundle 0176 · línea 17] retorno → bundle 882
[bundle 0914 · línea 53] descifrar: 16 bloques de 0x2380 a 0x3380
[bundle 0919 · línea 48] descifrar_archivo.i = 8
[bundle 0919 · línea 48] jef → .L_for_start_7 (bundle 650)
[bundle 0653 · línea 48] bge → .L_for_end_9 (bundle 921)
[bundle 0921 · línea 56] retorno → bundle 93
[bundle 0099 · línea 65] n2 = 8
[bundle 0101 · línea 66] logout
[bundle 0102 · línea 67] jef → fin_main (bundle 923)
[bundle 0923] jef → fin_main (bundle 923)
```

```
Estado final de las variables:
  origen = 4096
  cifrado = 8192
  vuelta = 12288
  bloques_de.b = 16
  bloques_de.total = 128
  bloques_de.tam = 16
  bloques_de.i = 7
  resto = 16
  d = 13184
  base = 12288
  direccion.tam = 16
  direccion.i = 7
  cifrar_archivo.n = 8
  cifrar_archivo.desde = 4096
  cifrar_archivo.hacia = 8192
  cifrar_archivo.total = 128
  cifrar_archivo.tam = 16
  cifrar_archivo.i = 8
  cifrar_archivo.b = 16
  descifrar_archivo.n = 8
  descifrar_archivo.desde = 8192
  descifrar_archivo.hacia = 12288
  descifrar_archivo.total = 128
  descifrar_archivo.tam = 16
  descifrar_archivo.i = 8
  descifrar_archivo.b = 16
  n1 = 8
  n2 = 8
```

## *F-F. Resultado de la ejecución*

```
Ejecucion de ejercicio3 con tools/sim_asm.py
Programa: ejercicios/ejercicio3_cifrado/ejercicio3.asm (925 bundles)
Fin: salto a sí mismo; bundles ejecutados: 14770; ciclos: 17083
Avisos de separación: 0
CRYPTO: 1035 instrucciones; autenticación de prueba (sin Ascon; token 999 falla)
Estado: AUTH=0 KVLOCK=1 EXC=0

Valores finales de las variables:
  origen = 4096
  cifrado = 8192
  vuelta = 12288
  bloques_de.b = 16
  bloques_de.total = 128
  bloques_de.tam = 16
  bloques_de.i = 7
  resto = 16
  d = 13184
  base = 12288
  direccion.tam = 16
  direccion.i = 7
  cifrar_archivo.n = 8
  cifrar_archivo.desde = 4096
  cifrar_archivo.hacia = 8192
  cifrar_archivo.total = 128
  cifrar_archivo.tam = 16
  cifrar_archivo.i = 8
  cifrar_archivo.b = 16
  descifrar_archivo.n = 8
  descifrar_archivo.desde = 8192
  descifrar_archivo.hacia = 12288
  descifrar_archivo.total = 128
  descifrar_archivo.tam = 16
  descifrar_archivo.i = 8
  descifrar_archivo.b = 16
  n1 = 8
  n2 = 8

Resultado esperado contra obtenido:
  n1: esperado 8, obtenido 8  OK
  n2: esperado 8, obtenido 8  OK

Archivo de entrada: datos.txt (1024 bytes) cargado en 0x1000.
Primeros 64 bytes de cada region (hexadecimal y texto):

0x1000, original:
  0x1000  43 61 6c 73 65 72 20 65 73 20 75 6e 20 6c 65 6e  |Calser es un len|
  0x1010  67 75 61 6a 65 20 70 65 71 75 65 6e 6f 20 79 20  |guaje pequeno y |
  0x1020  65 73 74 72 69 63 74 6f 2c 20 70 65 6e 73 61 64  |estricto, pensad|
  0x1030  6f 20 70 61 72 61 20 65 6e 73 65 6e 61 72 20 63  |o para ensenar c|

0x2000, cifrado:
  0x2000  0e 3d 78 e6 de ba 90 3c 76 0c dc 30 c9 2a e1 9d  |.=x....<v..0.*..|
  0x2010  95 37 dc 8d 61 12 a3 c2 71 10 ec 2e 28 b2 14 89  |.7..a...q...(...|
  0x2020  c2 bc 33 8b 1a c9 16 04 ec 87 40 01 c8 14 e8 95  |..3.......@.....|
  0x2030  26 8c 1e 4c 1e 68 a6 bd 26 6e fe a9 07 5a 0c f3  |&..L.h..&n...Z..|

0x3000, descifrado:
  0x3000  43 61 6c 73 65 72 20 65 73 20 75 6e 20 6c 65 6e  |Calser es un len|
  0x3010  67 75 61 6a 65 20 70 65 71 75 65 6e 6f 20 79 20  |guaje pequeno y |
```

```
  0x3020  65 73 74 72 69 63 74 6f 2c 20 70 65 6e 73 61 64  |estricto, pensad|
  0x3030  6f 20 70 61 72 61 20 65 6e 73 65 6e 61 72 20 63  |o para ensenar c|

Comprobaciones:
  0x3000-0x33FF == 0x1000-0x13FF: si
  0x2000-0x23FF distinto del original: si
  0x1000-0x13FF igual a datos.txt (no se altero): si

Veredicto: OK
```